

SP-3a — Multi-Tracker SDK

Foundation Implementation Plan

For agentic workers: REQUIRED SUB-SKILL: Use superpowers:subagent-driven-development (recommended) or superpowers:executing-plans to implement this plan task-by-task. Steps use checkbox (– []) syntax for tracking.

Goal: Decouple Lava's RuTracker implementation behind a tracker-agnostic SDK contract, ship RuTor as the first additional tracker, extract reusable mirror/registry/testing primitives to a new vasic-digital/Tracker-SDK submodule, and enforce three new constitutional anti-bluff clauses (6.D/6.E/6.F).

Architecture: Capability-based SDK (TrackerClient + per-feature interfaces) replaces the monolithic 14-method NetworkApi. Generic primitives (mirror manager, plugin registry, testing harness) live in vasic-digital/Tracker-SDK mounted at submodules/tracker_sdk/. Tracker-domain contract and impls (:core:tracker:api, :core:tracker:rutracker, :core:tracker:rutor) stay in this repo. SwitchingNetworkApi is rewired to delegate to LavaTrackerSdk underneath, preserving feature-module backward compatibility throughout SP-3a.

Tech Stack: Kotlin 2.1.0, Jetpack Compose, Dagger Hilt, Orbit MVI 7.1.0, Room 2.7.2, Ktor 2.3.1, OkHttp, Jsoup 1.15.3, kotlinx-serialization, JUnit 4, kotlinx-coroutines-test, PITest (mutation), WorkManager (periodic health probes).

Spec: docs/superpowers/specs/2026-04-30-sp3a-multi-tracker-sdk-foundation-design.md

Companion materials: docs/refactoring/decoupling/Lava_Multi_Tracker_SDK_Architecture_Plan.pdf

Estimated total: ~8.5 weeks across 6 phases (Phase 0 + Phases 1–5).

Phase Overview

Phase	Name	Duration	Tasks
0	Pre-flight Bluff Audit (Task 1.0)	0.5w	7
1	Foundation: Tracker-SDK + :core:tracker:* modules	2w	38
2	RuTracker decoupling (Kotlin only)	2.5w	36
3	RuTor implementation	2w	41
4	Mirror health + cross-tracker fallback + tracker_settings UI	1w	20

Phase	Name	Duration	Tasks
5	Constitutional updates + Challenge Tests + tag gate	0.5w	25

Phases are sequential except where explicitly noted (Phase 3 may begin after Phase 2 Task 2.5; Phase 5 runs concurrent with Phase 4).

File Structure (after SP-3a)

New repository: vasic-digital/Tracker-SDK (mounted at submodules/tracker_sdk/)

submodules/tracker_sdk/	
├─ README.md	
├─ LICENSE	MIT (matching
vasic-digital convention)	
├─ CLAUDE.md	inherits Lava
root + adds "no domain shape"	
├─ CONSTITUTION.md	same content as
CLAUDE.md, formatted as constitution	
├─ AGENTS.md	submodule-
specific agent guide	
├─ settings.gradle.kts	
├─ build.gradle.kts	root convention
├─ gradle/libs.versions.toml	pinned versions
for SDK	
├─ api/	
├─├─ build.gradle.kts	Kotlin library
├─├─ src/main/kotlin/lava/sdk/api/	
├─├─├─ MirrorUrl.kt	
├─├─├─ Protocol.kt	
├─├─├─ HealthState.kt	
├─├─├─ MirrorState.kt	
├─├─├─ FallbackPolicy.kt	
├─├─├─ MirrorUnavailableException.kt	
├─├─├─ PluginConfig.kt	
├─├─├─ HasId.kt	
├─ mirror/	
├─├─ build.gradle.kts	depends on :api
├─├─ src/main/kotlin/lava/sdk/mirror/	
├─├─├─ MirrorManager.kt	interface
├─├─├─ DefaultMirrorManager.kt	
├─├─├─ HealthProbe.kt	interface
├─├─├─ DefaultHealthProbe.kt	OkHttp-based
HEAD/GET probe	
├─ registry/	
├─├─ build.gradle.kts	depends on :api
├─├─ src/main/kotlin/lava/sdk/registry/	
├─├─├─ PluginRegistry.kt	
├─├─├─ PluginFactory.kt	
├─├─├─ DefaultPluginRegistry.kt	

```

├─ testing/
│   └─ build.gradle.kts
on :api, :mirror, :registry
│   └─ src/main/kotlin/lava/sdk/testing/
│       ├── FakeMirrorManager.kt
│       ├── FakeHealthProbe.kt
│       ├── HtmlFixtureLoader.kt
│       ├── JsonFixtureLoader.kt
│       └─ FalsifiabilityRehearsal.kt
protocol helper
│   └─ RehearsalRecord.kt
├─ docs/
│   ├── adding-a-primitive.md
│   └─ falsifiability-protocol.md
├─ scripts/
│   └─ ci.sh
gate (subset relevant to SDK)
│   └─ sync-mirrors.sh
sync

```

depends

the load-bearing

local-only CI

four-upstream

New modules in this repo

```

core/tracker/
grouping
├─ api/
│   └─ build.gradle.kts
lava.kotlin.library
│   └─ src/main/kotlin/lava/tracker/api/
│       ├── TrackerClient.kt
│       ├── TrackerFeature.kt
│       ├── TrackerDescriptor.kt
│       ├── TrackerCapability.kt
│       ├── AuthType.kt
│       └─ feature/
│           ├── SearchableTracker.kt
│           ├── BrowsableTracker.kt
│           ├── TopicTracker.kt
│           ├── CommentsTracker.kt
│           ├── FavoritesTracker.kt
│           ├── AuthenticatableTracker.kt
│           └─ DownloadableTracker.kt
│       └─ model/
│           ├── TorrentItem.kt
│           ├── TorrentFile.kt
│           ├── SearchRequest.kt
│           ├── SearchResult.kt
│           ├── BrowseResult.kt
│           ├── SortField.kt
│           ├── SortOrder.kt
│           ├── TimePeriod.kt
│           ├── TopicDetail.kt
│           ├── TopicPage.kt
│           └─ CommentsPage.kt

```

NEW top-level

marker

enum (13 values)
enum

```

|
|   |— Comment.kt
|   |— ForumTree.kt
|   |— ForumCategory.kt
|   |— LoginRequest.kt
|   |— LoginResult.kt
|   |— AuthState.kt
|   |— CaptchaChallenge.kt
|   |— CaptchaSolution.kt
|— client/
|   |— build.gradle.kts
|— lava.kotlin.library
|   |— src/main/kotlin/lava/tracker/client/
|       |— LavaTrackerSdk.kt                facade
|       |— CrossTrackerFallbackPolicy.kt
|       |— SearchOutcome.kt                sealed
|       |— BrowseOutcome.kt                sealed
|       |— TopicOutcome.kt                 sealed
|       |— DownloadOutcome.kt              sealed
|       |— di/
|           |— TrackerClientModule.kt        Hilt module
|       |— persistence/
|           |— MirrorHealthDao.kt
|           |— MirrorHealthEntity.kt
|           |— UserMirrorDao.kt
|           |— UserMirrorEntity.kt
|— rutracker/                               git mv from core/
|— network/rutracker
|   |— build.gradle.kts
|— lava.kotlin.tracker.module
|   |— src/main/kotlin/lava/tracker/rutracker/
|       |— RuTrackerClient.kt
|       |— RuTrackerDescriptor.kt
|       |— RuTrackerClientFactory.kt
|       |— feature/                         feature impls
|           |— RuTrackerSearch.kt
|           |— RuTrackerBrowse.kt
|           |— RuTrackerTopic.kt
|           |— RuTrackerComments.kt
|           |— RuTrackerFavorites.kt
|           |— RuTrackerAuth.kt
|           |— RuTrackerDownload.kt
|       |— mapper/                         DTO ↔ model
|           |— RuTrackerDtoMappers.kt      reverse mappers
|— used by SwitchingNetworkApi
|   |— ForumDtoMapper.kt
|   |— SearchPageMapper.kt
|   |— TopicMapper.kt
|   |— CommentsMapper.kt
|   |— TorrentMapper.kt
|   |— AuthMapper.kt
|   |— FavoritesMapper.kt
|   |— (existing parsers + scrapers, package paths updated to
|— lava.tracker.rutracker.*)
|— rutor/                                   NEW

```

```

|   |— build.gradle.kts
lava.kotlin.tracker.module
|   |— src/main/kotlin/lava/tracker/rutor/
|   |   |— RuTorClient.kt
|   |   |— RuTorDescriptor.kt
|   |   |— RuTorClientFactory.kt
|   |   |— http/
|   |   |   |— RuTorHttpClient.kt
|   |   |— feature/
|   |   |   |— RuTorSearch.kt
|   |   |   |— RuTorBrowse.kt
|   |   |   |— RuTorTopic.kt
|   |   |   |— RuTorComments.kt
|   |   |   |— RuTorAuth.kt
|   |   |   |— RuTorDownload.kt
|   |   |— parser/
|   |   |   |— RuTorSearchParser.kt
|   |   |   |— RuTorBrowseParser.kt
|   |   |   |— RuTorTopicParser.kt
|   |   |   |— RuTorCommentsParser.kt
|   |   |   |— RuTorLoginParser.kt
|   |   |   |— RuTorDateParser.kt
|   |   |   |— RuTorSizeParser.kt
|   |— src/test/
|   |   |— kotlin/lava/tracker/rutor/parser/
|   |   |   |— RuTorSearchParserTest.kt
|   |   |   |— RuTorBrowseParserTest.kt
|   |   |   |— RuTorTopicParserTest.kt
|   |   |   |— RuTorLoginParserTest.kt
|   |   |   |— RuTorDateParserTest.kt
|   |   |— resources/fixtures/rutor/
|   |   |   |— search/
|   |   |   |   |— search-normal-2026-04-30.html
|   |   |   |   |— search-empty-2026-04-30.html
|   |   |   |   |— search-edge-columns-2026-04-30.html
|   |   |   |   |— search-cyrillic-2026-04-30.html
|   |   |   |   |— search-malformed-2026-04-30.html
|   |   |   |— browse/ (5 fixtures)
|   |   |   |— topic/ (5 fixtures)
|   |   |   |— login/ (5 fixtures)
|— testing/
test helpers
|   |— build.gradle.kts
lava.kotlin.library
|   |— src/main/kotlin/lava/tracker/testing/
|   |   |— FakeTrackerClient.kt
|   |   |— TorrentItemBuilder.kt
|   |   |— SearchRequestBuilder.kt
|   |   |— LavaFixtureLoader.kt

feature/tracker_settings/
feature module
|— build.gradle.kts
lava.android.feature

```

Lava-specific

NEW Compose

```

└─ src/main/kotlin/lava/feature/tracker_settings/
    ├── TrackerSettingsScreen.kt
    ├── TrackerSettingsViewModel.kt
    ├── TrackerSettingsState.kt
    ├── TrackerSettingsAction.kt
    ├── TrackerSettingsSideEffect.kt
    ├── components/
    │   ├── TrackerSelectorList.kt
    │   ├── MirrorListSection.kt
    │   ├── AddCustomMirrorDialog.kt
    │   └── HealthIndicator.kt
    └─ navigation/
        └─ TrackerSettingsNavigation.kt

```

Modified existing files

buildSrc/src/main/kotlin/	
└─ KotlinTrackerModuleConventionPlugin.kt	NEW convention
plugin	
buildSrc/build.gradle.kts	register the new
plugin	
settings.gradle.kts	add new modules,
remove :core:network:rutracker	
core/network/rutracker/	REMOVED (git mv
to core/tracker/rutracker)	
core/network/api/build.gradle.kts	
add :core:tracker:api dependency for SwitchingNetworkApi rewire	
core/network/impl/src/main/kotlin/lava/network/impl/	
SwitchingNetworkApi.kt rewired to delegate to LavaTrackerSdk	
core/data/	feature
integrations updated to call LavaTrackerSdk	
feature/login/	ViewModel updated
to use AuthenticatableTracker via SDK	
feature/search_result/	ViewModel updated
to use SearchableTracker via SDK + cross-tracker fallback handling	
feature/topic/	ViewModel updated
to use TopicTracker via SDK	
feature/forum/	ViewModel updated
to use BrowsableTracker via SDK	
feature/favorites/	ViewModel updated
to use FavoritesTracker via SDK	
feature/menu/	Add tracker
selector entry point	
core/database/	Add Room
migrations for mirror health + user mirrors	
core/database/schemas/lava.database.LavaDatabase/<next>.json	New
schema file	
app/src/main/AndroidManifest.xml	Register

MirrorHealthCheckWorker if needed app/src/main/assets/mirrors.json configuration	NEW bundled mirror
CLAUDE.md	Add clauses 6.D,
6.E, 6.F	
core/CLAUDE.md	Scoped clause
about tracker capability honesty	
feature/CLAUDE.md	Scoped clause
about Challenge Tests for SDK consumers	
lava-api-go/CLAUDE.md	SP-3a-bridge
expectations	
lava-api-go/AGENTS.md	Mirror the above
AGENTS.md	Updated module
map and Tracker-SDK pin policy	
scripts/ci.sh	NEW local-only CI
gate	
scripts/tag.sh	Updated with new
gate (.lava-ci-evidence/<tag>/ requirements)	
scripts/check-fixture-freshness.sh	NEW
scripts/sync-tracker-sdk-mirrors.sh	NEW four-upstream
sync	
.github/hooks/pre-push	NEW non-
bypassable pre-push gate	
docs/sdk-developer-guide.md	NEW (partial draft
for SP-3a; full version in Spec 2)	
docs/superpowers/specs/2026-04-30-sp3a-coverage-exemptions.md	NEW
exemption ledger	
docs/refactoring/decoupling/refresh-fixtures.md	NEW fixture-
refresh runbook	
CHANGELOG.md	Lava-
Android-1.2.0-1020 entry	
.lava-ci-evidence/sp3a-bluff-audit/	NEW directory
.lava-ci-evidence/sp3a-challenges/	NEW directory

Conventions Used in This Plan

- **File paths** are repo-relative (e.g., core/tracker/api/...).
- **Run commands** start at the repo root unless noted.
- **Commit messages** follow the existing project style (lowercase prefix, short imperative summary). Co-author footer is appended automatically by scripts/commit.sh if present, otherwise added manually.
- **Falsifiability evidence** lives at .lava-ci-evidence/sp3a-<phase>/<task>-<sha>.json and is committed alongside the task.
- **Test runs** use ./gradlew :module:test for unit tests;
:module:integrationTest for integration;
:module:connectedAndroidTest for instrumented Compose tests.
- **All new Kotlin files** must run ./gradlew spotlessApply before commit.

Phase 0: Pre-flight Bluff Audit (Task 1.0)

Duration: 0.5 weeks ($\approx$ 2.5 working days). **Tasks:** 7. **Goal:** Apply the falsifiability protocol (Sixth Law clause 6.A) to every Kotlin test fake that SP-3a code will consume, plus re-rehearsal of two existing contract tests. Records evidence in `.lava-ci-evidence/sp3a-bluff-audit/`.

Acceptance gate: `.lava-ci-evidence/sp3a-bluff-audit/<commit-sha>.summary.json` exists with one entry per audited fake/contract, each entry containing the Test/Mutation/Observed/Reverted quartet. No SP-3a Phase 1 work begins until this gate is green.

Task 0.1: Audit `TestEndpointsRepository` for duplicate-rejection equivalence

Files:

- Read: `core/testing/src/main/kotlin/lava/testing/repository/TestEndpointsRepository.kt`
- Read: `core/data/src/main/kotlin/lava/data/EndpointsRepositoryImpl.kt` (find via `grep` — actual path may differ; the real impl is the one Hilt provides for `EndpointsRepository`)
- Create: `core/testing/src/test/kotlin/lava/testing/repository/TestEndpointsRepositoryEquivalenceTest.kt`
- Create: `.lava-ci-evidence/sp3a-bluff-audit/0.1-test-endpoints-repository-<sha>.json`



Step 1: Read the fake's current contract

Run: `cat core/testing/src/main/kotlin/lava/testing/repository/TestEndpointsRepository.kt` Expected: in-memory list of endpoints with add/remove/list operations.



Step 2: Locate the real `EndpointsRepositoryImpl`

Run: `grep -rn "class EndpointsRepositoryImpl" core/` Expected: One file printed. Read that file with the Read tool to identify the duplicate-rejection mechanism (likely a `Room @Insert(onConflict = OnConflictStrategy.ABORT)` or an explicit `if (exists()) throw`).



Step 3: Write equivalence test that asserts duplicate rejection

Create `core/testing/src/test/kotlin/lava/testing/repository/TestEndpointsRepositoryEquivalenceTest.kt`:

```
package lava.testing.repository
```

```
import kotlinx.coroutines.test.runTest
```



```

import org.junit.Assert.assertThrows
import org.junit.Test

class TestEndpointsRepositoryEquivalenceTest {

    @Test
    fun fake_rejects_duplicate_endpoint_like_real_impl() =
        runTest {
            val fake = TestEndpointsRepository()
            val endpoint = endpointFixture(name = "primary")
            fake.add(endpoint)

            val ex = assertThrows(IllegalStateException::class.java) {
                kotlinx.coroutines.runBlocking { fake.add(endpoint) }
            }

            assert(ex.message?.contains("duplicate") == true) {
                "fake threw '${ex.message}' but real
                EndpointsRepositoryImpl rejects duplicates with " +
                "an exception mentioning 'duplicate'"
            }
        }
}

```

If the fake doesn't currently reject duplicates, the test will fail — that's the **expected first run**, proving the fake was a bluff.



Step 4: Run the test, confirm it fails (deliberate-break observation)

```

Run: ./gradlew :core:testing:test --tests
"lava.testing.repository.TestEndpointsRepositoryEquivalenceTest"
-i Expected: FAIL with AssertionError: fake threw 'null' but real
EndpointsRepositoryImpl rejects duplicates...

```

Capture the failure output verbatim — it goes into the evidence file.



Step 5: Make the fake match the real contract

Edit core/testing/src/main/kotlin/lava/testing/repository/TestEndpointsRepository.kt. Add duplicate rejection to the add method:

```

override suspend fun add(endpoint: Endpoint) {
    val existing = _state.value.any { it == endpoint }
    if (existing) error("duplicate endpoint: $endpoint")
    _state.update { it + endpoint }
}

```

(Adjust to match the actual current method signature in the file — name and types may differ.)



Step 6: Re-run the test, confirm it passes

Run: `./gradlew :core:testing:test --tests "lava.testing.repository.TestEndpointsRepositoryEquivalenceTest"`
-i Expected: PASS.



Step 7: Write evidence file

Create `.lava-ci-evidence/sp3a-bluff-audit/0.1-test-endpoints-repository.json`:

```
{
  "task": "0.1",
  "subject": "TestEndpointsRepository",
  "real_counterpart": "EndpointsRepositoryImpl",
  "test":
    "TestEndpointsRepositoryEquivalenceTest.fake_rejects_duplicate_endpoint_l",
  "mutation": "Initial state: fake had no duplicate-rejection
    logic (the bluff)",
  "observed": "AssertionError: fake threw 'null' but real
    EndpointsRepositoryImpl rejects duplicates",
  "reverted": "Added duplicate-rejection check in add() matching
    real impl's onConflict=ABORT semantics",
  "verified_at": "<commit sha after step 8>",
  "ledger_clause": "6.A"
}
```



Step 8: Commit

```
git add core/testing/src/main/kotlin/lava/testing/repository/
TestEndpointsRepository.kt \
core/testing/src/test/kotlin/lava/testing/repository/
TestEndpointsRepositoryEquivalenceTest.kt \
.lava-ci-evidence/sp3a-bluff-audit/0.1-test-endpoints-
repository.json
git commit -m
"sp3a-0.1: bluff audit + fix TestEndpointsRepository
duplicate rejection"
```

Real EndpointsRepositoryImpl rejects duplicates via Room PK
conflict; the
fake silently accepted them. Falsifiability rehearsal recorded;
fake now
matches the real behavioral contract.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Then update the evidence file's `verified_at` field with the resulting commit SHA
and amend:

```
COMMIT=$(git rev-parse HEAD)
sed -i "s/<commit sha after step 8>/${COMMIT}/" .lava-ci-evidence/
sp3a-bluff-audit/0.1-test-endpoints-repository.json
```

```
git add .lava-ci-evidence/sp3a-bluff-audit/0.1-test-endpoints-  
repository.json  
git commit --amend --no-edit
```

Task 0.2: Audit TestBookmarksRepository for PK-conflict equivalence

Files:

- Read: core/testing/src/main/kotlin/lava/testing/repository/TestBookmarksRepository.kt
- Read: real BookmarksRepositoryImpl (locate via grep)
- Create: core/testing/src/test/kotlin/lava/testing/repository/TestBookmarksRepositoryEquivalenceTest.kt
- Create: .lava-ci-evidence/sp3a-bluff-audit/0.2-test-bookmarks-repository.json



Step 1: Locate the real impl

Run: `grep -rn "class BookmarksRepositoryImpl" core/` Read the file. Confirm Room DAO uses `onConflict = OnConflictStrategy.ABORT` (or equivalent).



Step 2: Write the equivalence test

```
package lava.testing.repository
```

```
import kotlinx.coroutines.test.runTest  
import org.junit.Assert.assertThrows  
import org.junit.Test
```

```
class TestBookmarksRepositoryEquivalenceTest {  
    @Test  
    fun fake_rejects_duplicate_bookmark_like_real_impl() =  
        runTest {  
            val fake = TestBookmarksRepository()  
            fake.add("topic-1")  
            val ex = assertThrows(IllegalStateException::class.java) {  
                kotlinx.coroutines.runBlocking { fake.add("topic-1") }  
            }  
            assert(ex.message?.contains("duplicate") == true) {  
                "fake threw '${ex.message}' but real impl rejects via  
                PK conflict"  
            }  
        }  
}
```



Step 3: Run, confirm fails (deliberate-break observation)

Run: `./gradlew :core:testing:test --tests "lava.testing.repository.TestBookmarksRepositoryEquivalenceTest"`
Expected: FAIL with the assertion message.



Step 4: Patch the fake

Edit `TestBookmarksRepository.kt` to reject duplicates. Match method signatures from the existing file.



Step 5: Re-run, confirm passes

Run: `./gradlew :core:testing:test --tests "lava.testing.repository.TestBookmarksRepositoryEquivalenceTest"`
Expected: PASS.



Step 6: Write evidence

`.lava-ci-evidence/sp3a-bluff-audit/0.2-test-bookmarks-repository.json` — same shape as 0.1.



Step 7: Commit + SHA stamp (same procedure as Task 0.1 Step 8).

Task 0.3: Audit `TestAuthService` for token-persistence equivalence

Files:

- Read: `core/testing/src/main/kotlin/lava/testing/service/TestAuthService.kt`
- Read: real `AuthServiceImpl` (likely in `core/auth/impl/`)
- Create: `core/testing/src/test/kotlin/lava/testing/service/TestAuthServiceEquivalenceTest.kt`
- Create: `.lava-ci-evidence/sp3a-bluff-audit/0.3-test-auth-service.json`



Step 1: Locate real impl

Run: `grep -rn "class AuthServiceImpl" core/`. Read file. Note what gets persisted on `login()` (likely a session token to `EncryptedSharedPreferences`).



Step 2: Write equivalence test asserting that, after `login()`, `currentUser` reflects the stored auth state and survives a fresh service instance

package `lava.testing.service`

import `kotlinx.coroutines.test.runTest`
import `org.junit.Assert.assertEquals`

```
import org.junit.Test
```

```
class TestAuthServiceEquivalenceTest {
    @Test
    fun fake_persists_login_state_like_real_impl() = runTest {
        val store = mutableMapOf<String, String>() // simulated
            shared backing store
        val a = TestAuthService(store)
        a.login("<redacted-per-$6.H>", "<redacted-per-$6.H>")
        val b = TestAuthService(store)
        assertEquals(
            "fake: a fresh instance backed by the same store
            should report Authenticated " +
            "(real AuthServiceImpl reads from
            EncryptedSharedPreferences on construction)",
            true, b.isAuthenticated()
        )
    }
}
```



Step 3: Run, confirm fails

Run: `./gradlew :core:testing:test --tests "lava.testing.service.TestAuthServiceEquivalenceTest"` Expected: FAIL
— fake holds state in instance field, not in shared store.



Step 4: Patch the fake to accept an injected backing store and read from it on construction.

If the existing `TestAuthService` constructor doesn't take a store argument, add one with a default `mutableMapOf()` so existing call sites still compile. Then in `login()`, write to the store; in `isAuthenticated()/init`, read from it.



Step 5: Re-run, confirm passes

Run: `./gradlew :core:testing:test --tests "lava.testing.service.TestAuthServiceEquivalenceTest"` Expected: PASS.



Step 6: Write evidence file (same shape).



Step 7: Commit + SHA stamp.

Task 0.4: Audit TestLocalNetworkDiscoveryService for `_lava._tcp.local.` suffix handling

Files:

- Read: `core/testing/src/main/kotlin/lava/testing/service/TestLocalNetworkDiscoveryService.kt`
- Read: real `LocalNetworkDiscoveryServiceImpl` (likely under `core/data/`)
- Create: `core/testing/src/test/kotlin/lava/testing/service/TestLocalNetworkDiscoveryServiceEquivalenceTest.kt`
- Create: `.lava-ci-evidence/sp3a-bluff-audit/0.4-test-local-network-discovery.json`



Step 1: Locate the real impl

Run: `grep -rn "_lava._tcp" core/` Read `LocalNetworkDiscoveryServiceImpl`. Confirm it filters or trims the `.local.` suffix when emitting service-name results.



Step 2: Write equivalence test

package `lava.testing.service`

```
import kotlinx.coroutines.test.runTest
import org.junit.Assert.assertTrue
import org.junit.Test
```

```
class TestLocalNetworkDiscoveryServiceEquivalenceTest {
    @Test
    fun fake_emits_only_lava_tcp_local_services() = runTest {
        val fake = TestLocalNetworkDiscoveryService()
        fake.simulateAdvertisedService(
            name = "lava-prod._lava._tcp.local.", host =
            "192.168.1.10", port = 8443
        )
        fake.simulateAdvertisedService(
            name = "printer._ipp._tcp.local.", host =
            "192.168.1.20", port = 631
        )
        val results =
            fake.discover().toList() // Flow.toList() – adjust to
            match actual API
        assertTrue(
            "fake should filter to _lava._tcp services (real impl
            uses NsdManager filter)",
            results.size == 1 && results[0].host == "192.168.1.10"
        )
    }
}
```



Step 3: Run, confirm fails (if fake doesn't filter)

Run: `./gradlew :core:testing:test --tests "lava.testing.service.TestLocalNetworkDiscoveryServiceEquivalenceTest"` Expected: FAIL.

☐

Step 4: Patch the fake to filter on `_lava._tcp.local.` substring (or equivalent filter the real impl uses — verify).

☐

Step 5: Re-run, confirm passes.

☐

Step 6: Evidence file.

☐

Step 7: Commit + SHA stamp.

Task 0.5: Re-rehearse `EndpointConverterTest`

Files:

- Read: `core/preferences/src/test/kotlin/lava/securestorage/EndpointConverterTest.kt`
- Read: the converter under test (find via `grep class EndpointConverter`)
- Create: `.lava-ci-evidence/sp3a-bluff-audit/0.5-endpoint-converter.json`

☐

Step 1: Read the existing test

Run: `cat core/preferences/src/test/kotlin/lava/securestorage/EndpointConverterTest.kt` Identify all `Endpoint` variants the test currently covers.

☐

Step 2: Read the converter

Run: `grep -rn "class EndpointConverter" core/`. Read the file.

☐

Step 3: Apply a deliberate break to the converter

Temporarily edit the converter to drop one endpoint variant from its when (e.g., remove the `Endpoint.LAN` branch). Save.

☐

Step 4: Run the existing test, confirm it fails

Run: `./gradlew :core:preferences:test --tests "lava.securestorage.EndpointConverterTest"` Expected: FAIL with a message naming the dropped variant.

Capture the message verbatim.



Step 5: Revert the break

Restore the converter file to its pre-step-3 state. Either via `git checkout -- <path>` or manually.



Step 6: Re-run the test, confirm it passes

Run: `./gradlew :core:preferences:test --tests "lava.securestorage.EndpointConverterTest"` Expected: PASS.



Step 7: Evidence file

`.lava-ci-evidence/sp3a-bluff-audit/0.5-endpoint-converter.json`:

```
{
  "task": "0.5",
  "subject": "EndpointConverterTest (re-rehearsal of pre-existing
    test)",
  "real_counterpart": "EndpointConverter",
  "test": "EndpointConverterTest (existing)",
  "mutation": "Removed Endpoint.LAN branch from
    EndpointConverter.when block",
  "observed": "<verbatim failure message from step 4>",
  "reverted": "git checkout -- core/preferences/src/main/
    kotlin/.../EndpointConverter.kt",
  "verified_at": "<commit sha>",
  "ledger_clause": "6.A"
}
```



Step 8: Commit evidence file (no code change in this task — the rehearsal validates an existing test).

```
git add .lava-ci-evidence/sp3a-bluff-audit/0.5-endpoint-
  converter.json
git commit -m "sp3a-0.5: re-rehearse EndpointConverterTest"
```

Falsifiability rehearsal recorded; existing test catches the deliberate break of removing Endpoint.LAN from the converter's when block.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 0.6: Re-rehearse healthcheck_contract_test

Files:

- Read: lava-api-go/tests/contract/healthcheck_contract_test.go
- Read: docker-compose.yml (the healthcheck declaration)
- Read: lava-api-go/cmd/healthprobe/main.go
- Create: .lava-ci-evidence/sp3a-bluff-audit/0.6-healthprobe-contract.json



Step 1: Read the contract test

Run: `cat lava-api-go/tests/contract/healthcheck_contract_test.go`
Note the falsifiability sub-test that exists per clause 6.A. Confirm it re-introduces the historical buggy `--http3` flag in the compose file fragment used by the test.



Step 2: Run the test as-is, confirm it passes

Run (from repo root): `cd lava-api-go && go test ./tests/contract/... -run TestHealthcheckContract -v Expected: PASS`, including the falsifiability sub-test.



Step 3: Apply a stronger deliberate break: introduce a second unknown flag in the live docker-compose.yml healthcheck (e.g., add `--http2` after `healthprobe`)

Edit `docker-compose.yml` and add `--http2` to the healthcheck command in the `lava-api-go` service. Do NOT commit this — it's a temporary deliberate break.



Step 4: Re-run the contract test

Run: `cd lava-api-go && go test ./tests/contract/... -run TestHealthcheckContract -v Expected: FAIL` — the contract checker rejects `--http2` (not registered in the binary's flag set).

Capture the failure output.



Step 5: Revert the break

Run: `git checkout -- docker-compose.yml` Verify: `git diff docker-compose.yml` is empty.



Step 6: Re-run, confirm passes

Run: `cd lava-api-go && go test ./tests/contract/... -run TestHealthcheckContract -v Expected: PASS`.



Step 7: Evidence file

.lava-ci-evidence/sp3a-bluff-audit/0.6-healthprobe-contract.json
— same shape as 0.5, mutation = "introduced --http2 in docker-compose.yml
healthcheck".



Step 8: Commit.

```
git add .lava-ci-evidence/sp3a-bluff-audit/0.6-healthprobe-
contract.json
git commit -m "sp3a-0.6: re-rehearse healthprobe contract test

Falsifiability re-rehearsal of forensic-anchor 6.A test using a
stronger
deliberate break (--http2) than the historical bug.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"
```

Task 0.7: Seed coverage exemption ledger and aggregate audit summary

Files:

- Create: docs/superpowers/specs/2026-04-30-sp3a-coverage-exemptions.md
- Create: .lava-ci-evidence/sp3a-bluff-audit/_summary.json



Step 1: Create the exemption ledger

```
cat > docs/superpowers/specs/2026-04-30-sp3a-coverage-
exemptions.md <<'EOF'
# SP-3a Coverage Exemption Ledger

Per Constitutional clause 6.D (Behavioral Coverage Contract),
every uncovered
line in code authored under SP-3a is listed here with reason,
reviewer, and
date. Blanket waivers are forbidden.

## Format

| File | Lines | Reason | Reviewer | Date | PR |
|---|---|---|---|---|---|

## Entries

(none yet – populated as Phase 1+ ships)

EOF
```



Step 2: Aggregate Phase 0 evidence into a summary

```
cat > .lava-ci-evidence/sp3a-bluff-audit/_summary.json <<'EOF'
{
  "phase": "SP-3a Phase 0 – Pre-flight Bluff Audit",
  "completed_at": "<set after commit>",
  "audited": [
    {"task": "0.1", "subject": "TestEndpointsRepository",
      "evidence": "0.1-test-endpoints-repository.json"},
    {"task": "0.2", "subject": "TestBookmarksRepository",
      "evidence": "0.2-test-bookmarks-repository.json"},
    {"task": "0.3", "subject": "TestAuthService", "evidence":
      "0.3-test-auth-service.json"},
    {"task": "0.4", "subject": "TestLocalNetworkDiscoveryService",
      "evidence": "0.4-test-local-network-discovery.json"},
    {"task": "0.5", "subject": "EndpointConverterTest (existing)",
      "evidence": "0.5-endpoint-converter.json"},
    {"task": "0.6", "subject": "healthcheck_contract_test
      (existing)", "evidence": "0.6-healthprobe-contract.json"}
  ],
  "deferred_to_sp3a_bridge": [
    "Go-side rutracker test fakes (audit deferred until SP-2
      ships)",
    "Go-side parity test framework (audit deferred until SP-2
      ships)"
  ],
  "ledger_clause": "6.A",
  "next_phase_unblocked": true
}
EOF
```



Step 3: Commit

```
git add docs/superpowers/specs/2026-04-30-sp3a-coverage-
  exemptions.md \
  .lava-ci-evidence/sp3a-bluff-audit/_summary.json
git commit -m "sp3a-0.7: seed coverage exemption ledger + Phase 0
  audit summary"
```

Phase 0 (pre-flight bluff audit) complete. All four touched fakes
plus
two existing contract tests have recorded falsifiability
rehearsals.
Phase 1 unblocked.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Then stamp _summary.json's completed_at field with the resulting commit SHA
via `git commit --amend` (same pattern as Task 0.1 Step 8).



Step 4: Phase 0 acceptance check

Run: `ls -la .lava-ci-evidence/sp3a-bluff-audit/` Expected: `_summary.json` plus 6 task-evidence files (0.1 through 0.6). All committed.

Run: `git log --oneline -8` Expected: 7 commits, one per task in Phase 0.

Phase 0 done. Phase 1 unblocked.

Phase 1: Foundation — Tracker-SDK + :core:tracker:* modules

Duration: 2 weeks. **Tasks:** 38. **Goal:** Stand up the new `vasic-digital/Tracker-SDK` repo on all four upstreams, build its three primitive packages (`:api`, `:mirror`, `:registry`, `:testing`) with full falsifiability evidence, mount it as a submodule, then create the in-repo `:core:tracker:api`, `:core:tracker:client`, `:core:tracker:registry`, `:core:tracker:mirror` modules.

Acceptance gate: `./`

`gradlew :core:tracker:api:test :core:tracker:client:test :core:tracker:registry:test :core:tracker:mirror:test :core:tracker:testing:test` all green. Tracker-SDK tagged `0.1.0` on all four upstreams with verified per-mirror SHA convergence. Constitutional clauses 6.D/6.E/6.F drafted in root `CLAUDE.md` (final wording locked in Phase 5).

Section A — Create `vasic-digital/Tracker-SDK` on all four upstreams

Task 1.1: Create `vasic-digital/Tracker-SDK` on GitHub

Files:

- Read: existing `gh repo create` patterns from any project script (e.g., `scripts/sync-mirrors.sh`).
- No files created in this repo for this task — it creates a remote.



Step 1: Verify `gh` authentication

Run: `gh auth status` Expected: shows authenticated to `github.com` as the user owning the `vasic-digital` org.

If not authenticated, run `gh auth login` (interactive — user types `!` `gh auth login` in prompt to run in this session).



Step 2: Create the repo

Run:

```
gh repo create vasic-digital/Tracker-SDK \
  --public \
  --description "Generic, tracker-agnostic SDK primitives: mirror
    manager, plugin registry, testing harness. Used by the
    Lava project to support multiple torrent trackers." \
  --license MIT \
  --add-readme=false
```

Expected: prints the new repo URL.



Step 3: Verify the repo exists

Run: `gh repo view vasic-digital/Tracker-SDK --json name,isPrivate,description` Expected: JSON showing "name": "Tracker-SDK", "isPrivate": false.



Step 4: Apply branch protection on master

Run:

```
gh api repos/vasic-digital/Tracker-SDK/branches/master/protection \
  --method PUT \
  --input - <<'EOF'
{
  "required_status_checks": null,
  "enforce_admins": false,
  "required_pull_request_reviews": null,
  "restrictions": null,
  "allow_force_pushes": false,
  "allow_deletions": false
}
EOF
```

Note: branch may not exist yet — that's OK; this command is re-issued in Task 1.25 after the initial commit. If it errors with "branch not found," skip; the Task 1.25 retry handles it.



Step 5: Record the upstream URL for Task 1.5's sync script:

```
echo "github=git@github.com:vasic-digital/Tracker-SDK.git" >> /
  tmp/tracker-sdk-upstreams.txt
```

(This file is consumed by the sync script in Task 1.5 and discarded after.)

This task creates a remote artifact only — no commit in the Lava repo.

Task 1.2: Create vasic-digital/Tracker-SDK on GitFlic



Step 1: Verify glab/GitFlic CLI auth

GitFlic uses a GitLab-compatible API. Confirm the existing project tooling — run `grep -rn "gitflic" scripts/`, look at `scripts/tag.sh` or `scripts/sync-mirrors.sh` for the established pattern.

If the tooling is `curl`-based against GitFlic's REST API, follow the existing pattern. If `glab` is configured for GitFlic, use:

Run: `glab --gitlab-host=https://gitflic.ru auth status`



Step 2: Create the repo

Following the existing project pattern. Typical command:

```
curl -X POST https://gitflic.ru/api/v1/project \
  -H "Authorization: token $GITFLIC_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Tracker-SDK","ownerAlias":"vasic-digital","description":"Generic SDK primitives","visibility":"PUBLIC"}'
```

Adjust headers and body to match the existing `scripts/sync-mirrors.sh` invocations exactly.



Step 3: Verify

Run: `curl -s -H "Authorization: token $GITFLIC_TOKEN" https://gitflic.ru/api/v1/project/vasic-digital/Tracker-SDK | jq .` Expected: project metadata including the description.



Step 4: Record upstream URL

```
echo "gitflic=git@gitflic.ru:vasic-digital/Tracker-SDK.git" >> /
tmp/tracker-sdk-upstreams.txt
```

Task 1.3: Create vasic-digital/Tracker-SDK on GitLab



Step 1: Verify glab auth

Run: `glab auth status` Expected: authenticated to gitlab.com.



Step 2: Create the repo

Run:

```
glab repo create vasic-digital/Tracker-SDK \
  --public \
  --description "Generic, tracker-agnostic SDK primitives: mirror
    manager, plugin registry, testing harness."
```



Step 3: Verify

Run: `glab repo view vasic-digital/Tracker-SDK` Expected: project page details with the description.



Step 4: Disable hosted CI/CD pipelines on the repo

Per Local-Only CI/CD constitution. GitLab projects ship with CI enabled by default — disable:

```
glab api projects/vasic-digital%2FTracker-SDK \
  --method PUT \
  --field "builds_access_level=disabled" \
  --field "shared_runners_enabled=false"
```

Expected: HTTP 200 with updated project JSON.



Step 5: Record upstream URL

```
echo "gitlab=git@gitlab.com:vasic-digital/Tracker-SDK.git" >> /
  tmp/tracker-sdk-upstreams.txt
```

Task 1.4: Create vasic-digital/Tracker-SDK on GitVerse



Step 1: Verify GitVerse access

GitVerse (Russian Sber-backed git host) uses a custom API. Confirm via existing project tooling — look at `scripts/sync-mirrors.sh` for the pattern.



Step 2: Create the repo via GitVerse's API

Use the same pattern as the existing Lava sync script. Typical:

```
curl -X POST https://api.gitverse.ru/v1/repo \
  -H "Authorization: Bearer $GITVERSE_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"name":"Tracker-SDK","owner":"vasic-
    digital","description":"Generic SDK
    primitives","public":true}'
```

(Adapt to the actual GitVerse API contract used by scripts/sync-mirrors.sh.)



Step 3: Verify

Run: `curl -s -H "Authorization: Bearer $GITVERSE_TOKEN" https://api.gitverse.ru/v1/repo/vasic-digital/Tracker-SDK | jq .` Expected: project metadata.



Step 4: Record upstream URL

```
echo "gitverse=git@gitverse.ru:vasic-digital/Tracker-SDK.git" >> /tmp/tracker-sdk-upstreams.txt
```

Task 1.5: Create the four-upstream sync script for Tracker-SDK

Files:

- Create: scripts/sync-tracker-sdk-mirrors.sh



Step 1: Read the existing main-repo sync script for the pattern

Run: `cat scripts/sync-mirrors.sh 2>&1 | head -80` (Adjust path if the existing script has a different name — `grep -l "GitFlic\|GitVerse" scripts/.).`



Step 2: Create the new script

Create scripts/sync-tracker-sdk-mirrors.sh:

```
#!/usr/bin/env bash
# scripts/sync-tracker-sdk-mirrors.sh
# Pushes the submodules/tracker_sdk/ working tree to all four
# upstreams.
# Local-only – no hosted CI invoked. Operator-controlled.

set -euo pipefail

cd "$(dirname "$0")/../../submodules/tracker_sdk"

UPSTREAMS=(
    "github git@github.com:vasic-digital/Tracker-SDK.git"
    "gitflic git@gitflic.ru:vasic-digital/Tracker-SDK.git"
    "gitlab git@gitlab.com:vasic-digital/Tracker-SDK.git"
    "gitverse git@gitverse.ru:vasic-digital/Tracker-SDK.git"
)

# Ensure remotes exist (idempotent)
for entry in "${UPSTREAMS[@]}; do
```



```

    read -r name url <<<"$entry"
    if ! git remote get-url "$name" >/dev/null 2>&1; then
        git remote add "$name" "$url"
    else
        git remote set-url "$name" "$url"
    fi
done

# Push to each
declare -A SHAS
for entry in "${UPSTREAMS[@]}; do
    read -r name url <<<"$entry"
    echo ">>> Pushing to $name ($url)..."
    git push "$name" --tags --force-with-lease
    git push "$name" master --force-with-lease
    SHAS[$name]=$ (git ls-remote "$name" master | awk '{print $1}')
done

# Verify per-mirror SHA convergence (Sixth Law clause 6.C)
EXPECTED=$(git rev-parse master)
for entry in "${UPSTREAMS[@]}; do
    read -r name _ <<<"$entry"
    if [[ "${SHAS[$name]}" != "$EXPECTED" ]]; then
        echo "MIRROR MISMATCH: $name reports ${SHAS[$name]}, expected
            $EXPECTED" >&2
        exit 1
    fi
done

echo "All four upstreams converged on $EXPECTED"

```



Step 3: Make executable

Run: `chmod +x scripts/sync-tracker-sdk-mirrors.sh`



Step 4: Commit

```

git add scripts/sync-tracker-sdk-mirrors.sh
git commit -m "sp3a-1.5: add Tracker-SDK four-upstream sync script

```

Mirrors submodules/tracker_sdk/ to GitHub + GitFlic + GitLab + GitVerse,
 verifies per-mirror SHA convergence per clause 6.C, exits non-zero
 on
 divergence. Used after every commit to the SDK submodule.

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Section B — Initialize the Tracker-SDK submodule repo

For Tasks 1.6 onward, work happens INSIDE the new (empty) vasic-digital/Tracker-SDK repo. Clone it locally first:

```
mkdir -p /tmp/tracker-sdk-init
cd /tmp/tracker-sdk-init
git clone git@github.com:vasic-digital/Tracker-SDK.git
cd Tracker-SDK
```

All subsequent file paths in Tasks 1.6–1.27 are relative to this clone unless prefixed with Lava/.

Task 1.6: Initialize the SDK Gradle project

Files (in /tmp/tracker-sdk-init/Tracker-SDK):

- Create: settings.gradle.kts
- Create: build.gradle.kts
- Create: gradle.properties
- Create: gradle/libs.versions.toml
- Create: gradlew, gradlew.bat, gradle/wrapper/
- Create: .gitignore



Step 1: Run gradle init to scaffold the wrapper

Run (inside /tmp/tracker-sdk-init/Tracker-SDK):

```
gradle init --type basic --dsl kotlin --project-name Tracker-SDK
--no-incubating <<< $'\n'
```

Expected: creates gradle/wrapper/, gradlew, gradlew.bat, basic settings.gradle.kts.



Step 2: Replace settings.gradle.kts

Overwrite with:

```
@file:Suppress("UnstableApiUsage")

pluginManagement {
    repositories {
        gradlePluginPortal()
        mavenCentral()
        google()
    }
}

dependencyResolutionManagement {
    repositoriesMode.set(RepositoriesMode.FAIL_ON_PROJECT_REPOS)
    repositories {
```

```

        mavenCentral()
        google()
    }
}

rootProject.name = "Tracker-SDK"

include(":api")
include(":mirror")
include(":registry")
include(":testing")

```



Step 3: Create the version catalog

Create gradle/libs.versions.toml:

[versions]

```

kotlin = "2.1.0"
kotlinx-coroutines = "1.8.1"
kotlinx-serialization = "1.6.3"
kotlinx-datetime = "0.6.0"
okhttp = "4.12.0"
junit = "4.13.2"
mockk = "1.13.10"
truth = "1.4.2"

```

[libraries]

```

kotlinx-coroutines-core = { module =
    "org.jetbrains.kotlinx:kotlinx-coroutines-core",
    version.ref = "kotlinx-coroutines" }
kotlinx-coroutines-test = { module =
    "org.jetbrains.kotlinx:kotlinx-coroutines-test",
    version.ref = "kotlinx-coroutines" }
kotlinx-serialization-json = { module =
    "org.jetbrains.kotlinx:kotlinx-serialization-json",
    version.ref = "kotlinx-serialization" }
kotlinx-datetime = { module = "org.jetbrains.kotlinx:kotlinx-
    datetime", version.ref = "kotlinx-datetime" }
okhttp-core = { module = "com.squareup.okhttp3:okhttp",
    version.ref = "okhttp" }
junit = { module = "junit:junit", version.ref = "junit" }
mockk = { module = "io.mockk:mockk", version.ref = "mockk" }
truth = { module = "com.google.truth:truth", version.ref =
    "truth" }

```

[plugins]

```

kotlin-jvm = { id = "org.jetbrains.kotlin.jvm", version.ref =
    "kotlin" }
kotlin-serialization = { id =
    "org.jetbrains.kotlin.plugin.serialization", version.ref
    = "kotlin" }

```



Step 4: Create root build.gradle.kts

Create:

```
plugins {  
    alias(libs.plugins.kotlin.jvm) apply false  
    alias(libs.plugins.kotlin.serialization) apply false  
}  
  
allprojects {  
    group = "lava.sdk"  
    version = "0.1.0-SNAPSHOT"  
}
```



Step 5: Create gradle.properties

Create:

```
org.gradle.jvmargs=-Xmx2g -Dfile.encoding=UTF-8  
org.gradle.parallel=true  
org.gradle.caching=true  
kotlin.code.style=official
```



Step 6: Create .gitignore

Create:

```
.gradle/  
build/  
.idea/  
*.iml  
local.properties  
out/  
.lava-ci-evidence/
```



Step 7: Verify the wrapper works

Run: `./gradlew tasks` Expected: lists default tasks without errors.



Step 8: Initial commit (do NOT push yet — pushing happens in Task 1.25 after the SDK content is fleshed out)

Run:

```
git add settings.gradle.kts build.gradle.kts gradle.properties \  
      gradle/libs.versions.toml gradle/wrapper/ gradlew  
      gradlew.bat .gitignore  
git commit -m "sdk-init: gradle scaffold, version catalog, wrapper"
```

Task 1.7: Submodule constitutional documents

Files (in Tracker-SDK clone):

- Create: README.md
- Create: LICENSE (MIT)
- Create: CLAUDE.md
- Create: CONSTITUTION.md
- Create: AGENTS.md



Step 1: Create LICENSE

```
curl -s https://raw.githubusercontent.com/licenses/license-templates/master/templates/mit.txt > LICENSE
```

Then edit the placeholders to fill in [year]=2026 and [fullname]=Vasić Digital / vasic-digital contributors.



Step 2: Create README.md

Tracker-SDK

Generic, tracker-agnostic SDK primitives extracted from the [Lava project](<https://github.com/milos85vasic/Lava>). Used to build multi-tracker clients without coupling to any one tracker's quirks.

Modules

- `**`api`**` – generic types: ``MirrorUrl``, ``Protocol``, ``HealthState``, ``MirrorState``, ``FallbackPolicy``, ``MirrorUnavailableException``, ``PluginConfig``, ``HasId``. No tracker-specific shape.
- `**`mirror`**` – ``MirrorManager`` + health-probe engine. Manages a list of mirror URLs per logical endpoint group, tracks per-mirror health, and provides waterfall fallback execution.
- `**`registry`**` – generic plugin-registry pattern. ``ConcurrentHashMap`` of factories keyed by string ID, type-parameterized over descriptor and plugin types.
- `**`testing`**` – fakes, fixture loaders, and the ``falsifiabilityRehearsal`` helper used to enforce Sixth Law clause 6.A

in consuming projects.

Constitutional inheritance

This submodule inherits all constitutional rules from the parent [Lava project](<https://github.com/milos85vasic/Lava/blob/master/CLAUDE.md>):

Sixth Law (clauses 6.A–6.F), Local-Only CI/CD, Decoupled Reusable Architecture, Host Machine Stability Directive. **Stricter rule added

here:** "no domain shape" – no class, function, file, or test resource

in this submodule may name a specific tracker, torrent site, or other

Lava-domain entity. CI gate enforces this.

Adding a primitive

See ``docs/adding-a-primitive.md``.

License

MIT (see LICENSE).



Step 3: Create CLAUDE.md (referencing the Lava root)

CLAUDE.md (Tracker-SDK submodule)

This file inherits all constitutional rules from the parent Lava project

at <https://github.com/milos85vasic/Lava/blob/master/CLAUDE.md>.

Read that file first; this document only adds rules specific to this submodule.

Inherited rules (binding)

- **Sixth Law (clauses 6.A–6.F)** – all anti-bluff testing requirements.
Tests in this submodule must traverse the same surfaces a consumer touches; falsifiability rehearsals are recorded; capability honesty applies (a primitive that exposes a feature must work).
- **Local-Only CI/CD** – no ``.github/workflows/*``, no ``.gitlab-ci.yml``, no GitVerse pipelines, no GitFlic CI. The ``.scripts/ci.sh`` in this repo is the only quality gate; it runs locally only.
- **Decoupled Reusable Architecture** – applies recursively. If this submodule grows code that another vasic-digital project would reasonably reuse, that code is extracted to a deeper submodule.

Submodule-specific rule: NO DOMAIN SHAPE

This submodule MUST NOT contain any class, function, file, source-set package name, or test resource that names a specific tracker, torrent site, scraper target, or other Lava-domain entity. The submodule is the **generic** layer; consuming projects (e.g. Lava) bring their own domain shape on top.

CI gate (in `scripts/ci.sh`):

```
\\\`bash
forbidden=$(grep -rEi --exclude-dir=.git --exclude-dir=build \
  '(rutracker|rutor|magnet|tracker\.org|torrent\.com)' . | \
  grep -v 'docs/.example' | grep -v 'README.md.*Lava project' ||
  true)
if [[ -n "$forbidden" ]]; then
  echo "Domain-shape violation:"
  echo "$forbidden"
  exit 1
fi
\\\`
```

The `tracker` word itself is permitted in **generic** contexts (e.g., "this primitive is useful for any tracker-style client"). Specific tracker names are not. The CI gate is calibrated accordingly.

Versioning

This submodule uses semantic versioning. The pin in any consuming repo is **frozen by default**; updating the pin is a deliberate PR. Breaking changes require a major version bump and explicit consumer migration notes in `CHANGELOG.md`.

Mirroring

This submodule is mirrored to GitHub, GitFlic, GitLab, and GitVerse (per Lava's recursive mirror policy). The `scripts/sync-mirrors.sh` in this repo handles all four; see Lava's `scripts/sync-tracker-sdk-mirrors.sh` for the consumer-side counterpart.



Step 4: Create CONSTITUTION.md

Same content as CLAUDE.md but formatted as a constitutional document with section numbers (matching the Lava project's convention). Copy the content; format the headings as `## Article 1: Inherited Rules`, `## Article 2: No Domain Shape`, etc.



Step 5: Create AGENTS.md

AGENTS.md (Tracker-SDK submodule)

Agent guide for working in this submodule. Read this before making any changes.

Quick orientation

- This submodule is **generic SDK primitives** – no tracker-specific code. If you find yourself typing "rutracker", "rutor", "magnet" or similar, you're in the wrong repo. Submit the change to the consuming project (Lava) instead.
- All quality gates run locally via `./scripts/ci.sh`.
- All four upstreams must converge on the same SHA before a release tag is cut. `./scripts/sync-mirrors.sh` enforces this.

Common tasks

Add a new primitive

1. Pick the right module: `:api` (types), `:mirror` (mirror logic), `:registry` (plugin discovery), or `:testing` (fakes/fixtures).
2. Create the new file under the module's `src/main/kotlin/lava/sdk/<module>/`.
3. Write a real-stack test under `src/test/kotlin/...`. Tests must exercise the public API; do not mock internal implementation.
4. If the primitive can be misused or is contract-shaped, add a `falsifiabilityRehearsal` test in `:testing`.
5. Run `./scripts/ci.sh` locally. All gates must pass.
6. Commit. Push to all four upstreams via `./scripts/sync-mirrors.sh`.
7. If the change is API-breaking, bump major version and add a `CHANGELOG.md` entry.

Run the full local CI gate

```
\\ \\ \\ bash
./scripts/ci.sh
\\ \\ \\
```

This runs: `spotless / ktlint`, unit tests, no-domain-shape grep, `forbidden hosted-CI files` check, license header check, mutation

tests on `:api` and `:mirror`.

Add a new module

1. Create the directory under the repo root.
2. Add `include(":<module>")` to `settings.gradle.kts`.
3. Create `<module>/build.gradle.kts` matching the pattern of an existing module.
4. Document the module in `README.md` and `CLAUDE.md`.

Things to avoid

- Importing OkHttp, Ktor, or any Android-specific library into `:api`.
- `:api` is pure Kotlin/JVM with no I/O dependencies.
- Adding a tracker-specific type or method anywhere.
- Hardcoded URLs of any kind. Even examples in tests use placeholder hosts (`example.com`, `mirror1.example`, etc.).
- Bypassing `./scripts/ci.sh` with `--no-verify`. The pre-push hook is not optional.



Step 6: Commit

```
git add README.md LICENSE CLAUDE.md CONSTITUTION.md AGENTS.md
git commit -m
    "sdk-init: constitution + agent guide (inherits Lava root
    + adds no-domain-shape)"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.8: Submodule `scripts/ci.sh` and pre-push hook

Files (in Tracker-SDK clone):

- Create: `scripts/ci.sh`
- Create: `scripts/sync-mirrors.sh`
- Create: `.githubhooks/pre-push`



Step 1: Create `scripts/ci.sh`

```
mkdir -p scripts
cat > scripts/ci.sh <<'CIEOF'
#!/usr/bin/env bash
# scripts/ci.sh – local-only CI gate for Tracker-SDK
set -euo pipefail
cd "$(dirname "$0")/.."
```

```

echo "==> No-domain-shape check"
violations=$(grep -rEi --exclude-dir=.git --exclude-dir=build --
    exclude-dir=.gradle \
    '(rutracker|rutor|magnet|tracker\.org|torrent\.com)' . \
    | grep -v 'README.md.*Lava project' \
    | grep -v 'CLAUDE.md.*example' \
    | grep -v 'AGENTS.md.*example' \
    || true)
if [[ -n "$violations" ]]; then
    echo "DOMAIN SHAPE VIOLATION:"
    echo "$violations"
    exit 1
fi
echo "    ok"

echo "==> Forbidden hosted-CI files check"
hosted=$(find . -path './.git' -prune -o \
    \( -path '*.github/workflows/*' \
    -o -name '*.gitlab-ci.yml' \
    -o -name 'azure-pipelines.yml' \
    -o -name 'bitbucket-pipelines.yml' \
    -o -name 'Jenkinsfile' \
    -o -name '*.circleci' -type d \
    \) -print 2>/dev/null || true)
if [[ -n "$hosted" ]]; then
    echo "HOSTED CI FILE FORBIDDEN: $hosted"
    exit 1
fi
echo "    ok"

echo "==> Spotless / ktlint"
./gradlew spotlessCheck

echo "==> Unit tests"
./gradlew test

echo "==> All gates passed"
CIEOF
chmod +x scripts/ci.sh

```



Step 2: Create scripts/sync-mirrors.sh

(Same shape as the Lava-side scripts/sync-tracker-sdk-mirrors.sh from Task 1.5 but inverted — pushes the local SDK clone to all four upstreams.)

```

cat > scripts/sync-mirrors.sh <<'SYNCEOF'
#!/usr/bin/env bash
# scripts/sync-mirrors.sh – push this repo to all four upstreams
set -euo pipefail
cd "$(dirname "$0")/.."

UPSTREAMS=(

```

```

"github git@github.com:vasic-digital/Tracker-SDK.git"
"gitflic git@gitflic.ru:vasic-digital/Tracker-SDK.git"
"gitlab git@gitlab.com:vasic-digital/Tracker-SDK.git"
"gitverse git@gitverse.ru:vasic-digital/Tracker-SDK.git"
)

for entry in "${UPSTREAMS[@]"}; do
    read -r name url <<<"$entry"
    if ! git remote get-url "$name" >/dev/null 2>&1; then
        git remote add "$name" "$url"
    else
        git remote set-url "$name" "$url"
    fi
done

declare -A SHAS
for entry in "${UPSTREAMS[@]"}; do
    read -r name url <<<"$entry"
    echo ">>> Pushing to $name"
    git push "$name" --tags --force-with-lease
    git push "$name" master --force-with-lease
    SHAS[$name]=$(git ls-remote "$name" master | awk '{print $1}')
done

EXPECTED=$(git rev-parse master)
for entry in "${UPSTREAMS[@]"}; do
    read -r name _ <<<"$entry"
    if [[ "${SHAS[$name]}" != "$EXPECTED" ]]; then
        echo "MIRROR MISMATCH: $name=${SHAS[$name]},
            expected=$EXPECTED" >&2
        exit 1
    fi
done
echo "All four upstreams converged on $EXPECTED"
SYNCEOF
chmod +x scripts/sync-mirrors.sh

```



Step 3: Create .github/hooks/pre-push

```

mkdir -p .github/hooks
cat > .github/hooks/pre-push <<'HOOKEOF'
#!/usr/bin/env bash
# .github/hooks/pre-push - non-bypassable local CI gate
set -euo pipefail
exec ./scripts/ci.sh
HOOKEOF
chmod +x .github/hooks/pre-push
git config core.hooksPath .github/hooks

```



Step 4: Commit

```
git add scripts/ci.sh scripts/sync-mirrors.sh .github/hooks/pre-push
git commit -m "sdk-init: scripts/ci.sh, sync-mirrors.sh, pre-push
hook"
```

Local-only CI gate: domain-shape check, hosted-CI forbidden-file check,
spotless, unit tests. Pre-push hook enabled via core.hooksPath.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section C — Tracker-SDK :api module

Task 1.9: Create :api module skeleton

Files (in Tracker-SDK clone):

- Create: api/build.gradle.kts
- Create: api/src/main/kotlin/lava/sdk/api/.gitkeep (placeholder until real files arrive)
- Create: api/src/test/kotlin/lava/sdk/api/.gitkeep



Step 1: Create api/build.gradle.kts

```
plugins {
    alias(libs.plugins.kotlin.jvm)
    alias(libs.plugins.kotlin.serialization)
}

dependencies {
    api(libs.kotlinx-coroutines.core)
    api(libs.kotlinx-serialization.json)
    api(libs.kotlinx-datetime)

    testImplementation(libs.junit)
    testImplementation(libs.kotlinx-coroutines.test)
    testImplementation(libs.truth)
    testImplementation(libs.mockk)
}
```



Step 2: Create source directories with .gitkeep

```
mkdir -p api/src/main/kotlin/lava/sdk/api
mkdir -p api/src/test/kotlin/lava/sdk/api
touch api/src/main/kotlin/lava/sdk/api/.gitkeep
touch api/src/test/kotlin/lava/sdk/api/.gitkeep
```



Step 3: Verify the module builds

Run: `./gradlew :api:compileKotlin` Expected: BUILD SUCCESSFUL.



Step 4: Commit

```
git add api/build.gradle.kts api/src/
git commit -m "sdk-1.9: scaffold :api module"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.10: MirrorUrl, Protocol, HealthState + tests

Files:

- Create: `api/src/main/kotlin/lava/sdk/api/Protocol.kt`
- Create: `api/src/main/kotlin/lava/sdk/api/HealthState.kt`
- Create: `api/src/main/kotlin/lava/sdk/api/MirrorUrl.kt`
- Create: `api/src/test/kotlin/lava/sdk/api/MirrorUrlTest.kt`
- Create: `api/src/test/kotlin/lava/sdk/api/ProtocolTest.kt`



Step 1: Write the failing tests first (TDD)

`api/src/test/kotlin/lava/sdk/api/MirrorUrlTest.kt`:

```
package lava.sdk.api
```

```
import com.google.common.truth.Truth.assertThat
import kotlinx.serialization.json.Json
import org.junit.Test
```

```
class MirrorUrlTest {
    private val json = Json { prettyPrint = false }

    @Test
    fun `constructs with sensible defaults`() {
        val m = MirrorUrl(url = "https://example.com")
        assertThat(m.url).isEqualTo("https://example.com")
        assertThat(m.isPrimary).isFalse()
        assertThat(m.priority).isEqualTo(0)
        assertThat(m.protocol).isEqualTo(Protocol.HTTPS)
        assertThat(m.region).isNull()
    }

    @Test
    fun `serializes to JSON round-trip identical`() {
        val original = MirrorUrl(
            url = "https://primary.example",
            isPrimary = true,
            priority = 0,
            protocol = Protocol.HTTPS,
        )
    }
}
```

```

        region = "us-east"
    )
    val encoded = json.encodeToString(MirrorUrl.serializer(),
        original)
    val decoded =
        json.decodeFromString(MirrorUrl.serializer(), encoded)
    assertThat(decoded).isEqualTo(original)
}

@Test
fun `equality is structural`() {
    val a = MirrorUrl(url = "https://x")
    val b = MirrorUrl(url = "https://x")
    assertThat(a).isEqualTo(b)
    assertThat(a.hashCode()).isEqualTo(b.hashCode())
}
}

```

api/src/test/kotlin/lava/sdk/api/ProtocolTest.kt:

```
package lava.sdk.api
```

```
import com.google.common.truth.Truth.assertThat
import org.junit.Test
```

```
class ProtocolTest {
    @Test
    fun `enum has exactly three values`() {
        assertThat(Protocol.entries).hasSize(3)
        assertThat(Protocol.entries.map { it.name })
            .containsExactly("HTTP", "HTTPS", "HTTP3")
    }
}

```



Step 2: Run tests, confirm they fail (compile error: undefined types)

Run: `./gradlew :api:test` Expected: FAIL with "Unresolved reference: MirrorUrl" / "Protocol".



Step 3: Implement Protocol

api/src/main/kotlin/lava/sdk/api/Protocol.kt:

```
package lava.sdk.api
```

```
import kotlinx.serialization.Serializable
```

```
@Serializable
enum class Protocol { HTTP, HTTPS, HTTP3 }
```



Step 4: Implement HealthState

api/src/main/kotlin/lava/sdk/api/HealthState.kt:

```
package lava.sdk.api
```

```
import kotlinx.serialization.Serializable
```

```
@Serializable
```

```
enum class HealthState { HEALTHY, DEGRADED, UNHEALTHY, UNKNOWN }
```



Step 5: Implement MirrorUrl

api/src/main/kotlin/lava/sdk/api/MirrorUrl.kt:

```
package lava.sdk.api
```

```
import kotlinx.serialization.Serializable
```

```
/**
```

```
 * A single mirror endpoint URL with metadata. Lower [priority] =  
 * higher precedence.
```

```
*/
```

```
@Serializable
```

```
data class MirrorUrl(  
    val url: String,  
    val isPrimary: Boolean = false,  
    val priority: Int = 0,  
    val protocol: Protocol = Protocol.HTTPS,  
    val region: String? = null,  
)
```



Step 6: Run tests, confirm pass

Run: ./gradlew :api:test Expected: BUILD SUCCESSFUL with 4 tests passing (3 in MirrorUrlTest + 1 in ProtocolTest).



Step 7: Commit

```
git add api/src/main/kotlin/lava/sdk/api/Protocol.kt \  
        api/src/main/kotlin/lava/sdk/api/HealthState.kt \  
        api/src/main/kotlin/lava/sdk/api/MirrorUrl.kt \  
        api/src/test/kotlin/lava/sdk/api/MirrorUrlTest.kt \  
        api/src/test/kotlin/lava/sdk/api/ProtocolTest.kt \  
        api/src/main/kotlin/lava/sdk/api/.gitkeep \  
        api/src/test/kotlin/lava/sdk/api/.gitkeep
```

```
# Remove .gitkeep stubs (the dirs now have real files)
```

```
git rm api/src/main/kotlin/lava/sdk/api/.gitkeep api/src/test/  
      kotlin/lava/sdk/api/.gitkeep
```

```
git commit -m "sdk-1.10: MirrorUrl, Protocol, HealthState + tests"
```

TDD: tests written first, observed failing, then types implemented.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.11: MirrorState and FallbackPolicy + tests

Files:

- Create: api/src/main/kotlin/lava/sdk/api/MirrorState.kt
- Create: api/src/main/kotlin/lava/sdk/api/FallbackPolicy.kt
- Create: api/src/test/kotlin/lava/sdk/api/MirrorStateTest.kt
- Create: api/src/test/kotlin/lava/sdk/api/FallbackPolicyTest.kt



Step 1: Write failing tests

MirrorStateTest.kt:

```
package lava.sdk.api
```

```
import com.google.common.truth.Truth.assertThat
import kotlinx.datetime.Instant
import org.junit.Test
```

```
class MirrorStateTest {
    @Test
    fun `constructs with required fields`() {
        val mirror = MirrorUrl("https://x")
        val now = Instant.fromEpochSeconds(1000)
        val s = MirrorState(mirror = mirror, health =
            HealthState.HEALTHY,
                           lastCheck = now, consecutiveFailures
            = 0)
        assertThat(s.mirror).isEqualTo(mirror)
        assertThat(s.health).isEqualTo(HealthState.HEALTHY)
        assertThat(s.consecutiveFailures).isEqualTo(0)
    }

    @Test
    fun `lastCheck may be null for never-probed mirror`() {
        val s = MirrorState(MirrorUrl("https://x"),
            HealthState.UNKNOWN, null, 0)
        assertThat(s.lastCheck).isNull()
    }
}
```

FallbackPolicyTest.kt:


```

package lava.sdk.api

import com.google.common.truth.Truth.assertThat
import kotlin.time.Duration.Companion.seconds
import org.junit.Test

class FallbackPolicyTest {
    @Test
    fun `defaults match documented values`() {
        val p = FallbackPolicy()
        assertThat(p.maxAttempts).isEqualTo(3)
        assertThat(p.perAttemptTimeout).isEqualTo(10.seconds)
        assertThat(p.degradeAfter).isEqualTo(1)
        assertThat(p.unhealthyAfter).isEqualTo(3)
    }

    @Test
    fun `custom values are honored`() {
        val p = FallbackPolicy(maxAttempts = 5, perAttemptTimeout = 30.seconds,
                               degradeAfter = 2, unhealthyAfter = 4)
        assertThat(p.maxAttempts).isEqualTo(5)
        assertThat(p.perAttemptTimeout).isEqualTo(30.seconds)
    }
}

```



Step 2: Run tests, confirm fail

Run: `./gradlew :api:test --tests "*MirrorStateTest*" --tests "*FallbackPolicyTest*" Expected: FAIL — unresolved references.`



Step 3: Implement MirrorState

`api/src/main/kotlin/lava/sdk/api/MirrorState.kt:`

```

package lava.sdk.api

import kotlinx.datetime.Instant
import kotlinx.serialization.Serializable

@Serializable
data class MirrorState(
    val mirror: MirrorUrl,
    val health: HealthState,
    val lastCheck: Instant?,
    val consecutiveFailures: Int,
)

```



Step 4: Implement FallbackPolicy

api/src/main/kotlin/lava/sdk/api/FallbackPolicy.kt:

```
package lava.sdk.api
```

```
import kotlin.time.Duration
```

```
import kotlin.time.Duration.Companion.seconds
```

```
data class FallbackPolicy(  
    val maxAttempts: Int = 3,  
    val perAttemptTimeout: Duration = 10.seconds,  
    val degradeAfter: Int = 1,  
    val unhealthyAfter: Int = 3,  
)
```



Step 5: Run tests, confirm pass

Run: ./gradlew :api:test Expected: BUILD SUCCESSFUL.



Step 6: Commit

```
git add api/src/main/kotlin/lava/sdk/api/MirrorState.kt \  
      api/src/main/kotlin/lava/sdk/api/FallbackPolicy.kt \  
      api/src/test/kotlin/lava/sdk/api/MirrorStateTest.kt \  
      api/src/test/kotlin/lava/sdk/api/FallbackPolicyTest.kt  
git commit -m "sdk-1.11: MirrorState + FallbackPolicy + tests"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.12: MirrorUnavailableException, PluginConfig, HasId + tests

Files:

- Create: api/src/main/kotlin/lava/sdk/api/MirrorUnavailableException.kt
- Create: api/src/main/kotlin/lava/sdk/api/PluginConfig.kt
- Create: api/src/main/kotlin/lava/sdk/api/HasId.kt
- Create: api/src/test/kotlin/lava/sdk/api/MirrorUnavailableExceptionTest.kt



Step 1: Write failing test

MirrorUnavailableExceptionTest.kt:

```
package lava.sdk.api
```

```
import com.google.common.truth.Truth.assertThat
```

```
import org.junit.Test
```

```

class MirrorUnavailableExceptionTest {
    @Test
    fun `carries list of attempted mirrors`() {
        val tried = listOf(MirrorUrl("https://a"),
            MirrorUrl("https://b"))
        val ex = MirrorUnavailableException(tried)
        assertThat(ex.tried).containsExactlyElementsIn(tried).inOrder()
        assertThat(ex.message).contains("2 mirror(s) attempted")
    }

    @Test
    fun `is a Throwable subtype usable in coroutine
        cancellation`() {
        val ex: Throwable =
            MirrorUnavailableException(emptyList())
        assertThat(ex).assertInstanceOf(Exception::class.java)
    }
}

```



Step 2: Run, confirm fails

Run: `./gradlew :api:test --tests "*MirrorUnavailableExceptionTest"`
 Expected: FAIL.



Step 3: Implement the three types

MirrorUnavailableException.kt:

```
package lava.sdk.api
```

```

class MirrorUnavailableException(
    val tried: List<MirrorUrl>,
    cause: Throwable? = null,
) : Exception("MirrorUnavailable: ${tried.size} mirror(s)
    attempted", cause)

```

PluginConfig.kt:

```
package lava.sdk.api
```

```

/**
 * Opaque config bag passed to a [PluginFactory.create] call.
 * Implementations may extend with their own typed accessors.
 */
interface PluginConfig {
    val raw: Map<String, Any?>
}

```

```

/** Trivial implementation backed by a map. */
class MapPluginConfig(override val raw: Map<String, Any?> =
    emptyMap()) : PluginConfig

```

HasId.kt:

```
package lava.sdk.api
```

```
/** Anything keyed by a stable string identifier. */  
interface HasId {  
    val id: String  
}
```



Step 4: Run tests, confirm pass

Run: `./gradlew :api:test` Expected: PASS, all api tests green.



Step 5: Commit

```
git add api/src/main/kotlin/lava/sdk/api/  
MirrorUnavailableException.kt \  
api/src/main/kotlin/lava/sdk/api/PluginConfig.kt \  
api/src/main/kotlin/lava/sdk/api/HasId.kt \  
api/src/test/kotlin/lava/sdk/api/  
MirrorUnavailableExceptionTest.kt  
git commit -m  
"sdk-1.12: MirrorUnavailableException + PluginConfig +  
HasId"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.13: :api falsifiability rehearsal — confirm MirrorState equality is structural

This task validates that the test suite catches a deliberate equals regression — proving the tests aren't tautological.

Files:

- Create: `api/src/test/kotlin/lava/sdk/api/MirrorStateFalsifiabilityTest.kt`
- Create: `.lava-ci-evidence/sdk-falsifiability/1.13-mirror-state-equality.json` (in the SDK clone)



Step 1: Apply the deliberate break — temporarily change MirrorState from data class to plain class

Edit `api/src/main/kotlin/lava/sdk/api/MirrorState.kt`, change data class to class. Save.



Step 2: Run the equality assertion, confirm it fails

Run: `./gradlew :api:test --tests "*MirrorStateTest*" Expected: FAIL`
— without data, structural equality is gone.

Actually, wait — `MirrorStateTest` doesn't currently assert equality. Let's add one in the falsifiability test rather than mutating the existing test file:

Create `api/src/test/kotlin/lava/sdk/api/MirrorStateFalsifiabilityTest.kt`:

```
package lava.sdk.api
```

```
import com.google.common.truth.Truth.assertThat
import kotlinx.datetime.Instant
import org.junit.Test
```

```
class MirrorStateFalsifiabilityTest {
    @Test
    fun `equality is structural – proves data class shape`() {
        val mirror = MirrorUrl("https://x")
        val now = Instant.fromEpochSeconds(1000)
        val a = MirrorState(mirror, HealthState.HEALTHY, now, 0)
        val b = MirrorState(mirror, HealthState.HEALTHY, now, 0)
        assertThat(a).isEqualTo(b)
        assertThat(a.hashCode()).isEqualTo(b.hashCode())
    }
}
```



Step 3: Run with `MirrorState` mutated to plain class, confirm fails

Run: `./gradlew :api:test --tests "*MirrorStateFalsifiabilityTest*" Expected: FAIL` — expected <a> to be equal to (Truth assertion message).

Capture the failure verbatim.



Step 4: Revert `MirrorState` to data class

Either via git checkout -- `api/src/main/kotlin/lava/sdk/api/MirrorState.kt` or by manually re-adding the data keyword. Verify with git diff that the file matches its committed state.



Step 5: Re-run the test, confirm passes

Run: `./gradlew :api:test --tests "*MirrorStateFalsifiabilityTest*" Expected: PASS.`



Step 6: Write evidence file

```
mkdir -p .lava-ci-evidence/sdk-falsifiability
cat > .lava-ci-evidence/sdk-falsifiability/1.13-mirror-state-
equality.json <<'EOF'
{
  "task": "sdk-1.13",
  "subject": "MirrorState equality contract",
  "test": "MirrorStateFalsifiabilityTest.equality is structural",
  "mutation": "Changed `data class MirrorState` to `class
    MirrorState`",
  "observed": "AssertionError: expected <a> to be equal to <b>",
  "reverted": "Restored `data` keyword via git checkout",
  "ledger_clause": "6.A"
}
EOF
```



Step 7: Commit

```
git add api/src/test/kotlin/lava/sdk/api/
    MirrorStateFalsifiabilityTest.kt \
    .lava-ci-evidence/sdk-falsifiability/1.13-mirror-state-
    equality.json
git commit -m "sdk-1.13: falsifiability rehearsal – MirrorState
    equality contract"
```

Confirms test suite catches loss of structural equality on
MirrorState.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section D — Tracker-SDK :mirror module

Task 1.14: Create :mirror module skeleton

Files (in Tracker-SDK clone):

- Create: mirror/build.gradle.kts
- Create: mirror/src/main/kotlin/lava/sdk/mirror/.gitkeep
- Create: mirror/src/test/kotlin/lava/sdk/mirror/.gitkeep



Step 1: Create mirror/build.gradle.kts

```
plugins {
    alias(libs.plugins.kotlin.jvm)
}

dependencies {
    api(project(":api"))
    api(libs.kotlinx.coroutines.core)
    implementation(libs.okhttp.core)
```

```

        testImplementation(libs.junit)
        testImplementation(libs.kotlinx.coroutines.test)
        testImplementation(libs.truth)
        testImplementation(libs.mockk)
        testImplementation(testFixtures(project(":api")))
    }

```



Step 2: Verify

Run: `./gradlew :mirror:compileKotlin` Expected: BUILD SUCCESSFUL.



Step 3: Commit

```

mkdir -p mirror/src/main/kotlin/lava/sdk/mirror mirror/src/test/
    kotlin/lava/sdk/mirror
touch mirror/src/main/kotlin/lava/sdk/mirror/.gitkeep mirror/src/
    test/kotlin/lava/sdk/mirror/.gitkeep
git add mirror/
git commit -m "sdk-1.14: scaffold :mirror module"

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 1.15: HealthProbe interface + DefaultHealthProbe

Files:

- Create: `mirror/src/main/kotlin/lava/sdk/mirror/HealthProbe.kt`
- Create: `mirror/src/main/kotlin/lava/sdk/mirror/DefaultHealthProbe.kt`
- Create: `mirror/src/test/kotlin/lava/sdk/mirror/DefaultHealthProbeTest.kt`



Step 1: Write failing test

`DefaultHealthProbeTest.kt`:

```

package lava.sdk.mirror

import com.google.common.truth.Truth.assertThat
import kotlinx.coroutines.test.runTest
import lava.sdk.api.HealthState
import lava.sdk.api.MirrorUrl
import okhttp3.mockwebserver.MockResponse
import okhttp3.mockwebserver.MockWebServer
import org.junit.After
import org.junit.Before
import org.junit.Test

```

```

import kotlin.time.Duration.Companion.seconds

class DefaultHealthProbeTest {
    private lateinit var server: MockWebServer

    @Before fun setUp() { server = MockWebServer().also {
        it.start() } }
    @After fun tearDown() { server.shutdown() }

    @Test
    fun `200 with body containing marker - HEALTHY`() = runTest {
        server.enqueue(MockResponse().setBody("Welcome to
        ExpectedMarker site"))
        val probe = DefaultHealthProbe(
            expectedMarker = "ExpectedMarker", timeout = 5.seconds
        )
        val result =
            probe.probe(MirrorUrl(server.url("/").toString()))
        assertThat(result).isEqualTo(HealthState.HEALTHY)
    }

    @Test
    fun `200 without marker - UNHEALTHY (captive-portal
    hazard)`() = runTest {
        server.enqueue(MockResponse().setBody("This site is
        blocked"))
        val probe = DefaultHealthProbe("ExpectedMarker",
            5.seconds)
        val result =
            probe.probe(MirrorUrl(server.url("/").toString()))
        assertThat(result).isEqualTo(HealthState.UNHEALTHY)
    }

    @Test
    fun `500 - UNHEALTHY`() = runTest {
        server.enqueue(MockResponse().setResponseCode(500).setBody("Internal
        Error"))
        val probe = DefaultHealthProbe("ExpectedMarker",
            5.seconds)
        val result =
            probe.probe(MirrorUrl(server.url("/").toString()))
        assertThat(result).isEqualTo(HealthState.UNHEALTHY)
    }

    @Test
    fun `slow response (5-10s) - DEGRADED`() = runTest {
        server.enqueue(MockResponse().setBody("ExpectedMarker
        present").setHeadersDelay(7,
            java.util.concurrent.TimeUnit.SECONDS))
        val probe = DefaultHealthProbe("ExpectedMarker",
            15.seconds)
        val result =
            probe.probe(MirrorUrl(server.url("/").toString()))
        assertThat(result).isEqualTo(HealthState.DEGRADED)
    }
}

```



```
}  
}
```

(Note: MockWebServer requires okhttp.mockwebserver test dependency. Add to mirror/build.gradle.kts testImplementation:
"com.squareup.okhttp3:mockwebserver:4.12.0".)



Step 2: Add MockWebServer dependency, run test, confirm fail

Edit mirror/build.gradle.kts testImplementation to add
"com.squareup.okhttp3:mockwebserver:4.12.0".

Run: ./gradlew :mirror:test --tests "*DefaultHealthProbeTest*"
Expected: FAIL — unresolved references.



Step 3: Implement HealthProbe interface

HealthProbe.kt:

```
package lava.sdk.mirror  
  
import lava.sdk.api.HealthState  
import lava.sdk.api.MirrorUrl  
  
interface HealthProbe {  
    suspend fun probe(endpoint: MirrorUrl): HealthState  
}
```



Step 4: Implement DefaultHealthProbe

DefaultHealthProbe.kt:

```
package lava.sdk.mirror  
  
import kotlin.time.Duration  
import kotlinx.coroutines.Dispatchers  
import kotlinx.coroutines.withContext  
import kotlinx.coroutines.withTimeoutOrNull  
import lava.sdk.api.HealthState  
import lava.sdk.api.MirrorUrl  
import lava.sdk.api.Protocol  
import okhttp3.OkHttpClient  
import okhttp3.Request  
import java.util.concurrent.TimeUnit  
  
class DefaultHealthProbe(  
    private val expectedMarker: String,  
    private val timeout: Duration,  
    private val degradedThresholdMs: Long = 5_000L,  
    private val unhealthyThresholdMs: Long = 10_000L,  
    private val client: OkHttpClient = defaultClient(timeout),
```

```

) : HealthProbe {

    override suspend fun probe(endpoint: MirrorUrl): HealthState
        = withContext(Dispatchers.IO) {
            val started = System.currentTimeMillis()
            val request =
                Request.Builder().url(endpoint.url).get().build()
            val outcome = withTimeoutOrNull(timeout) {
                runCatching { client.newCall(request).execute() }
            } ?: return@withContext HealthState.UNHEALTHY // timed
                out

            outcome.fold(
                onSuccess = { response ->
                    response.use { r ->
                        val elapsed = System.currentTimeMillis() -
                            started
                        val body = r.body?.string().orEmpty()
                        when {
                            !r.isSuccessful -> HealthState.UNHEALTHY
                            !body.contains(expectedMarker, ignoreCase
                                = true) -> HealthState.UNHEALTHY
                            elapsed >= unhealthyThresholdMs ->
                                HealthState.UNHEALTHY
                            elapsed >= degradedThresholdMs ->
                                HealthState.DEGRADED
                            else -> HealthState.HEALTHY
                        }
                    }
                },
                onFailure = { HealthState.UNHEALTHY }
            )
        }

    private companion object {
        fun defaultClient(timeout: Duration) =
            OkHttpClient.Builder()
                .connectTimeout(timeout.inWholeMilliseconds,
                    TimeUnit.MILLISECONDS)
                .readTimeout(timeout.inWholeMilliseconds,
                    TimeUnit.MILLISECONDS)
                .build()
    }
}

```



Step 5: Run tests, confirm pass

Run: `./gradlew :mirror:test --tests "*DefaultHealthProbeTest"`
 Expected: BUILD SUCCESSFUL — all 4 tests pass.



Step 6: Commit

```
git add mirror/build.gradle.kts \
    mirror/src/main/kotlin/lava/sdk/mirror/HealthProbe.kt \
    mirror/src/main/kotlin/lava/sdk/mirror/
    DefaultHealthProbe.kt \
    mirror/src/test/kotlin/lava/sdk/mirror/
    DefaultHealthProbeTest.kt
git commit -m "sdk-1.15: HealthProbe interface +
    DefaultHealthProbe + tests"
```

Probes return UNHEALTHY for: non-2xx, body missing expected marker, timeout. DEGRADED for response time >5s. Marker check catches captive-portal-style 200-OK-but-blocked responses.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.16: MirrorManager interface + group-state types

Files:

- Create: mirror/src/main/kotlin/lava/sdk/mirror/MirrorManager.kt
- Create: mirror/src/main/kotlin/lava/sdk/mirror/MirrorGroup.kt



Step 1: Implement the interface (no test yet — interface alone has nothing to test)

MirrorManager.kt:

```
package lava.sdk.mirror
```

```
import kotlinx.coroutines.flow.Flow
import lava.sdk.api.MirrorState
import lava.sdk.api.MirrorUrl
```

```
interface MirrorManager {
```

```
    /** Returns the highest-priority mirror with health HEALTHY or
        DEGRADED, or null if none. */
    suspend fun getHealthyMirror(groupId: String): MirrorUrl?
```

```
    /**
        * Executes [op] against mirrors in priority order, skipping
        UNHEALTHY ones,
        * recording success/failure, and falling back on per-mirror
        failure.
        * Throws [lava.sdk.api.MirrorUnavailableException] if all
        attempts fail.
        */
```

```
suspend fun <T> executeWithFallback(
    groupId: String,
    op: suspend (MirrorUrl) -> T,
```

```

): T

/** Stream of current state for the group's mirrors. */
fun observeHealth(groupId: String): Flow<List<MirrorState>>

/** Probe all registered mirrors for the group right now (out-
    of-band). */
suspend fun probeAll(groupId: String)

/** Mark a successful op against [endpoint] (clears
    consecutiveFailures, ensures HEALTHY/DEGRADED). */
suspend fun reportSuccess(endpoint: MirrorUrl)

/** Mark a failure against [endpoint] (increments counter, may
    transition state). */
suspend fun reportFailure(endpoint: MirrorUrl, cause:
    Throwable)
}

```

MirrorGroup.kt:

```

package lava.sdk.mirror

import lava.sdk.api.MirrorUrl

/** Configuration for a single mirror group registered with a
    [MirrorManager]. */
data class MirrorGroup(
    val groupId: String,
    val mirrors: List<MirrorUrl>,
    val expectedMarker: String,
)

```



Step 2: Verify compile

Run: `./gradlew :mirror:compileKotlin` Expected: BUILD SUCCESSFUL.



Step 3: Commit

```

git add mirror/src/main/kotlin/lava/sdk/mirror/MirrorManager.kt \
    mirror/src/main/kotlin/lava/sdk/mirror/MirrorGroup.kt
git commit -m "sdk-1.16: MirrorManager interface + MirrorGroup
    config

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 1.17: DefaultMirrorManager impl + falsifiability rehearsal for fallback

Files:

- Create: mirror/src/main/kotlin/lava/sdk/mirror/DefaultMirrorManager.kt
- Create: mirror/src/test/kotlin/lava/sdk/mirror/DefaultMirrorManagerTest.kt
- Create: mirror/src/test/kotlin/lava/sdk/mirror/DefaultMirrorManagerFalsifiabilityTest.kt
- Create: .lava-ci-evidence/sdk-falsifiability/1.17-mirror-manager-fallback.json



Step 1: Write failing tests for the contract

DefaultMirrorManagerTest.kt:

```
package lava.sdk.mirror

import com.google.common.truth.Truth.assertThat
import kotlinx.coroutines.flow.first
import kotlinx.coroutines.test.runTest
import lava.sdk.api.HealthState
import lava.sdk.api.MirrorUnavailableException
import lava.sdk.api.MirrorUrl
import org.junit.Test

class DefaultMirrorManagerTest {

    private fun groupOf(vararg urls: String) = MirrorGroup(
        groupId = "g",
        mirrors = urls.mapIndexed { i, u -> MirrorUrl(u,
            isPrimary = i == 0, priority = i) },
        expectedMarker = "marker",
    )

    @Test
    fun `executeWithFallback returns first successful mirror result`() = runTest {
        val mgr = DefaultMirrorManager(
            initialGroups = listOf(groupOf("https://a", "https://b")),
            healthProbe = FakeProbe { _ -> HealthState.HEALTHY },
        )
        val tried = mutableListOf<String>()
        val result = mgr.executeWithFallback("g") { mirror ->
            tried += mirror.url
            "result-from-${mirror.url}"
        }
        assertThat(result).isEqualTo("result-from-https://a")
        assertThat(tried).containsExactly("https://a")
    }
}
```

```

}

@Test
fun `executeWithFallback skips UNHEALTHY mirror and tries
next`() = runTest {
    val mgr = DefaultMirrorManager(
        initialGroups = listOf(groupOf("https://a", "https://
b")),
        healthProbe = FakeProbe { mirror ->
            if (mirror.url == "https://a")
                HealthState.UNHEALTHY else HealthState.HEALTHY
        },
    )
    mgr.probeAll("g")
    val tried = mutableListOf<String>()
    val result = mgr.executeWithFallback("g") { mirror ->
        tried += mirror.url
        "result-from-${mirror.url}"
    }
    assertThat(tried).containsExactly("https://b")
    assertThat(result).isEqualTo("result-from-https://b")
}

@Test
fun `executeWithFallback retries on per-attempt failure`() =
    runTest {
        val mgr = DefaultMirrorManager(
            initialGroups = listOf(groupOf("https://a", "https://
b")),
            healthProbe = FakeProbe { _ -> HealthState.HEALTHY },
        )
        var calls = 0
        val result = mgr.executeWithFallback("g") { mirror ->
            calls++
            if (mirror.url == "https://a") error("a-failed") else
                "ok"
        }
        assertThat(calls).isEqualTo(2)
        assertThat(result).isEqualTo("ok")
    }

@Test
fun `executeWithFallback throws MirrorUnavailable when all
fail`() = runTest {
    val mgr = DefaultMirrorManager(
        initialGroups = listOf(groupOf("https://a", "https://
b")),
        healthProbe = FakeProbe { _ -> HealthState.HEALTHY },
    )
    try {
        mgr.executeWithFallback("g") { _ -> error("always-
fail") }
        error("expected MirrorUnavailableException")
    } catch (e: MirrorUnavailableException) {

```

```

        assertThat(e.tried).hasSize(2)
    }
}

private class FakeProbe(private val states: (MirrorUrl) ->
    HealthState) : HealthProbe {
    override suspend fun probe(endpoint: MirrorUrl) =
        states(endpoint)
}

```



Step 2: Implement DefaultMirrorManager

DefaultMirrorManager.kt:

```

package lava.sdk.mirror

import kotlinx.coroutines.flow.Flow
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.asStateFlow
import kotlinx.coroutines.flow.update
import kotlinx.coroutines.sync.Mutex
import kotlinx.coroutines.sync.withLock
import kotlinx.datetime.Clock
import lava.sdk.api.FallbackPolicy
import lava.sdk.api.HealthState
import lava.sdk.api.MirrorState
import lava.sdk.api.MirrorUnavailableException
import lava.sdk.api.MirrorUrl

class DefaultMirrorManager(
    initialGroups: List<MirrorGroup>,
    private val healthProbe: HealthProbe,
    private val policy: FallbackPolicy = FallbackPolicy(),
    private val clock: Clock = Clock.System,
) : MirrorManager {

    private data class Group(
        val config: MirrorGroup,
        val states: MutableStateFlow<List<MirrorState>>,
    )

    private val groups: Map<String, Group> =
        initialGroups.associate { g ->
            g.groupId to Group(
                config = g,
                states = MutableStateFlow(g.mirrors.map { mirror ->
                    MirrorState(mirror, HealthState.UNKNOWN,
                        lastCheck = null, consecutiveFailures = 0)
                })
            )
        }
}

```

```

private val mutex = Mutex()

override suspend fun getHealthyMirror(groupId: String):
    MirrorUrl? {
    val g = groups[groupId] ?: return null
    return g.states.value
        .filter { it.health == HealthState.HEALTHY ||
            it.health == HealthState.DEGRADED }
        .sortedWith(compareBy({ healthRank(it.health) }, {
            it.mirror.priority })))
        .firstOrNull()
        ?.mirror
}

override suspend fun <T> executeWithFallback(
    groupId: String,
    op: suspend (MirrorUrl) -> T,
): T {
    val g = groups[groupId] ?: throw
        IllegalArgumentException("Unknown group: $groupId")
    val ordered = g.states.value
        .sortedWith(compareBy({ healthRank(it.health) }, {
            it.mirror.priority })))
        .filter { it.health != HealthState.UNHEALTHY }
    val tried = mutableListOf<MirrorUrl>()
    for (state in ordered) {
        tried += state.mirror
        try {
            val result = op(state.mirror)
            reportSuccess(state.mirror)
            return result
        } catch (t: Throwable) {
            reportFailure(state.mirror, t)
        }
    }
    throw MirrorUnavailableException(tried)
}

override fun observeHealth(groupId: String):
    Flow<List<MirrorState>> =
    groups[groupId]?.states?.asStateFlow() ?: error("Unknown
    group: $groupId")

override suspend fun probeAll(groupId: String) {
    val g = groups[groupId] ?: return
    val updated = g.states.value.map { current ->
        val newHealth = runCatching {
            healthProbe.probe(current.mirror) }
            .getOrElse(HealthState.UNHEALTHY)
        MirrorState(
            current.mirror, newHealth,
            lastCheck = clock.now(),

```



```

        consecutiveFailures = if (newHealth ==
HealthState.HEALTHY) 0 else current.consecutiveFailures
    )
}
g.states.value = updated
}

override suspend fun reportSuccess(endpoint: MirrorUrl) =
    mutex.withLock {
        val g = groupContaining(endpoint) ?: return
        g.states.update { list ->
            list.map { s ->
                if (s.mirror == endpoint) s.copy(health =
HealthState.HEALTHY, consecutiveFailures = 0, lastCheck =
clock.now())
                else s
            }
        }
    }

override suspend fun reportFailure(endpoint: MirrorUrl,
    cause: Throwable) = mutex.withLock {
        val g = groupContaining(endpoint) ?: return
        g.states.update { list ->
            list.map { s ->
                if (s.mirror != endpoint) s
                else {
                    val newCount = s.consecutiveFailures + 1
                    val newHealth = when {
                        newCount >= policy.unhealthyAfter ->
HealthState.UNHEALTHY
                        newCount >= policy.degradeAfter ->
HealthState.DEGRADED
                        else -> s.health
                    }
                    s.copy(health = newHealth,
consecutiveFailures = newCount, lastCheck = clock.now())
                }
            }
        }
    }

private fun groupContaining(endpoint: MirrorUrl): Group? =
    groups.values.firstOrNull { g -> g.states.value.any {
        it.mirror == endpoint } }

private fun healthRank(h: HealthState): Int = when (h) {
    HealthState.HEALTHY -> 0
    HealthState.DEGRADED -> 1
    HealthState.UNKNOWN -> 2
    HealthState.UNHEALTHY -> 3
}
}

```



Step 3: Run tests, confirm pass

Run: `./gradlew :mirror:test --tests "*DefaultMirrorManagerTest*"`
 Expected: PASS — all 4 tests green.



Step 4: Write the falsifiability rehearsal

DefaultMirrorManagerFalsifiabilityTest.kt:

```
package lava.sdk.mirror

import kotlinx.coroutines.test.runTest
import lava.sdk.api.HealthState
import lava.sdk.api.MirrorUrl
import org.junit.Test
import org.junit.Assert.assertThrows

class DefaultMirrorManagerFalsifiabilityTest {

    @Test
    fun `fallback skips UNHEALTHY – proves test catches
    deliberate misordering`() = runTest {
        // Setup: mirror A is UNHEALTHY, mirror B is HEALTHY. We
        // expect B to be tried, A skipped.
        val mgr = DefaultMirrorManager(
            initialGroups = listOf(MirrorGroup(
                groupId = "g",
                mirrors = listOf(
                    MirrorUrl("https://a", isPrimary = true,
                        priority = 0),
                    MirrorUrl("https://b", priority = 1),
                ),
                expectedMarker = "marker",
            )),
            healthProbe = object : HealthProbe {
                override suspend fun probe(endpoint: MirrorUrl) =
                    if (endpoint.url == "https://a")
                        HealthState.UNHEALTHY else HealthState.HEALTHY
            },
        )
        mgr.probeAll("g")

        val touched = mutableListof<String>()
        mgr.executeWithFallback("g") { m -> touched += m.url; "ok-
        ${m.url}" }

        check(touched == listOf("https://b")) {

            "expected only https://b to be tried (a is UNHEALTHY);
            got $touched"
        }
    }
}
```

```
}  
}
```



Step 5: Apply the deliberate break — break the UNHEALTHY filter in `executeWithFallback`

Edit `DefaultMirrorManager.kt`. In `executeWithFallback`, comment out the `.filter { it.health != HealthState.UNHEALTHY }` line. Save.



Step 6: Run the falsifiability test, confirm fails

Run: `./gradlew :mirror:test --tests "*DefaultMirrorManagerFalsifiabilityTest*" Expected: FAIL` — expected only `https://b` to be tried (a is UNHEALTHY); got `[https://a, https://b]` (because op throws when called against an UNHEALTHY mirror, but the touched list captures the attempt).

Capture the failure verbatim.



Step 7: Revert the break

Restore `DefaultMirrorManager.kt` to its committed state. `git diff` should be empty.



Step 8: Re-run, confirm passes

Run: `./gradlew :mirror:test --tests "*DefaultMirrorManagerFalsifiabilityTest*" Expected: PASS`.



Step 9: Write evidence file

`.lava-ci-evidence/sdk-falsifiability/1.17-mirror-manager-fallback.json`:

```
{  
  "task": "sdk-1.17",  
  "subject": "DefaultMirrorManager fallback chain skips  
             UNHEALTHY",  
  "test": "DefaultMirrorManagerFalsifiabilityTest.fallback skips  
           UNHEALTHY",  
  "mutation": "Removed `.filter { it.health !=  
              HealthState.UNHEALTHY }` from executeWithFallback",  
  "observed": "expected only https://b to be tried (a is  
              UNHEALTHY); got [https://a, https://b]",  
  "reverted": "Restored filter line via git checkout",  
  "ledger_clause": "6.A"  
}
```



Step 10: Commit

```
git add mirror/src/main/kotlin/lava/sdk/mirror/
      DefaultMirrorManager.kt \
mirror/src/test/kotlin/lava/sdk/mirror/
      DefaultMirrorManagerTest.kt \
mirror/src/test/kotlin/lava/sdk/mirror/
      DefaultMirrorManagerFalsifiabilityTest.kt \
.lava-ci-evidence/sdk-falsifiability/1.17-mirror-manager-
      fallback.json
git commit -m "sdk-1.17: DefaultMirrorManager + falsifiability
      rehearsal"
```

Implements fallback chain executor that orders by (health,
priority),
skips UNHEALTHY, and transitions states based on
consecutiveFailures.
Falsifiability rehearsal proves the test catches removal of the
UNHEALTHY filter.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Section E — Tracker-SDK :registry module

Task 1.18: Create :registry module skeleton

Files:

- Create: registry/build.gradle.kts



Step 1: Create registry/build.gradle.kts

```
plugins { alias(libs.plugins.kotlin.jvm) }
```

```
dependencies {
    api(project(":api"))
    testImplementation(libs.junit)
    testImplementation(libs.kotlinx-coroutines.test)
    testImplementation(libs.truth)
    testImplementation(libs.mockk)
}
```



Step 2: Verify and commit

Run: ./gradlew :registry:compileKotlin Expected: BUILD SUCCESSFUL.

```
mkdir -p registry/src/main/kotlin/lava/sdk/registry registry/src/
      test/kotlin/lava/sdk/registry
touch registry/src/main/kotlin/lava/sdk/registry/.gitkeep
      registry/src/test/kotlin/lava/sdk/registry/.gitkeep
```

```
git add registry/
git commit -m "sdk-1.18: scaffold :registry module"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.19: PluginRegistry, PluginFactory, DefaultPluginRegistry

Files:

- Create: registry/src/main/kotlin/lava/sdk/registry/PluginFactory.kt
- Create: registry/src/main/kotlin/lava/sdk/registry/PluginRegistry.kt
- Create: registry/src/main/kotlin/lava/sdk/registry/DefaultPluginRegistry.kt
- Create: registry/src/test/kotlin/lava/sdk/registry/DefaultPluginRegistryTest.kt



Step 1: Write failing test

DefaultPluginRegistryTest.kt:

```
package lava.sdk.registry
```

```
import com.google.common.truth.Truth.assertThat
import lava.sdk.api.HasId
import lava.sdk.api.MapPluginConfig
import org.junit.Test
```

```
class DefaultPluginRegistryTest {
    private data class TestDescriptor(override val id: String,
        val displayName: String) : HasId
    private interface TestPlugin { val descriptor:
        TestDescriptor; fun greet(): String }

    private fun factory(d: TestDescriptor):
        PluginFactory<TestDescriptor, TestPlugin> =
    object : PluginFactory<TestDescriptor, TestPlugin> {
        override val descriptor = d
        override fun create(config:
            lava.sdk.api.PluginConfig) =
            object : TestPlugin {
                override val descriptor = d
                override fun greet() = "hi from ${d.id}"
            }
    }
}
```

```
@Test
```

```

fun `register then get returns same plugin instance per
call`() {
    val reg = DefaultPluginRegistry<TestDescriptor,
    TestPlugin>()
    val d = TestDescriptor("alpha", "Alpha")
    reg.register(factory(d))
    val plugin = reg.get("alpha", MapPluginConfig())
    assertThat(plugin.greet()).isEqualTo("hi from alpha")
}

@Test
fun `list returns all registered descriptors`() {
    val reg = DefaultPluginRegistry<TestDescriptor,
    TestPlugin>()
    reg.register(factory(TestDescriptor("a", "A")))
    reg.register(factory(TestDescriptor("b", "B")))
    assertThat(reg.list().map { it.id }).containsExactly("a",
    "b")
}

@Test
fun `get on unknown id throws IllegalArgumentException`() {
    val reg = DefaultPluginRegistry<TestDescriptor,
    TestPlugin>()
    try {
        reg.get("missing", MapPluginConfig())
        error("expected exception")
    } catch (e: IllegalArgumentException) {
        assertThat(e.message).contains("missing")
    }
}

@Test
fun `unregister removes the factory`() {
    val reg = DefaultPluginRegistry<TestDescriptor,
    TestPlugin>()
    val d = TestDescriptor("x", "X")
    reg.register(factory(d))
    reg.unregister("x")
    assertThat(reg.list()).isEmpty()
}
}

```



Step 2: Run, confirm fail (compile error).



Step 3: Implement the three types.

PluginFactory.kt:

```
package lava.sdk.registry
```

```
import lava.sdk.api.HasId
```

```
import lava.sdk.api.PluginConfig
```

```
interface PluginFactory<D : HasId, P> {  
    val descriptor: D  
    fun create(config: PluginConfig): P  
}
```

PluginRegistry.kt:

```
package lava.sdk.registry
```

```
import lava.sdk.api.HasId  
import lava.sdk.api.PluginConfig
```

```
interface PluginRegistry<D : HasId, P> {  
    fun register(factory: PluginFactory<D, P>)  
    fun unregister(id: String)  
    fun get(id: String, config: PluginConfig): P  
    fun list(): List<D>  
    fun isRegistered(id: String): Boolean  
}
```

DefaultPluginRegistry.kt:

```
package lava.sdk.registry
```

```
import java.util.concurrent.ConcurrentHashMap  
import lava.sdk.api.HasId  
import lava.sdk.api.PluginConfig
```

```
class DefaultPluginRegistry<D : HasId, P> : PluginRegistry<D, P> {  
  
    private val factories = ConcurrentHashMap<String,  
        PluginFactory<D, P>>()  
  
    override fun register(factory: PluginFactory<D, P>) {  
        factories[factory.descriptor.id] = factory  
    }  
  
    override fun unregister(id: String) {  
        factories.remove(id)  
    }  
  
    override fun get(id: String, config: PluginConfig): P {  
        val f = factories[id]  
        ?: throw IllegalArgumentException("Unknown plugin id:  
            $id")  
        return f.create(config)  
    }  
  
    override fun list(): List<D> = factories.values.map {  
        it.descriptor }  
}
```

```

        override fun isRegistered(id: String): Boolean =
            factories.containsKey(id)
    }

```



Step 4: Run tests, confirm pass

Run: `./gradlew :registry:test` Expected: BUILD SUCCESSFUL — 4 tests pass.



Step 5: Commit

```

git add registry/
git commit -m "sdk-1.19: PluginRegistry + PluginFactory +
    DefaultPluginRegistry

```

Generic plugin registration keyed by descriptor.id,
ConcurrentHashMap-backed.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Task 1.20: :registry falsifiability rehearsal — duplicate-registration semantics

Files:

- Create: `registry/src/test/kotlin/lava/sdk/registry/DefaultPluginRegistryFalsifiabilityTest.kt`
- Create: `.lava-ci-evidence/sdk-falsifiability/1.20-registry-duplicate.json`

The semantics decision: re-registering the same id replaces the factory (last-write-wins). We assert this via test, then prove the test catches the alternative behavior (silent ignore, or throw).



Step 1: Write the falsifiability test asserting last-write-wins

```

package lava.sdk.registry

```

```

import com.google.common.truth.Truth.assertThat
import lava.sdk.api.HasId
import lava.sdk.api.MapPluginConfig
import lava.sdk.api.PluginConfig
import org.junit.Test

```

```

class DefaultPluginRegistryFalsifiabilityTest {

```

```

    private data class D(override val id: String) : HasId

```



```

private fun factoryReturning(value: String) = object :
    PluginFactory<D, String> {
        override val descriptor = D("x")
        override fun create(config: PluginConfig) = value
    }

@Test
fun `re-registering same id replaces the factory
    (last-write-wins)`() {
    val reg = DefaultPluginRegistry<D, String>()
    reg.register(factoryReturning("first"))
    reg.register(factoryReturning("second"))
    val result = reg.get("x", MapPluginConfig())
    assertThat(result).isEqualTo("second")
}
}

```



Step 2: Apply deliberate break — change register to ignore duplicates

Edit DefaultPluginRegistry.kt:

```

override fun register(factory: PluginFactory<D, P>) {
    if (!factories.containsKey(factory.descriptor.id)) {
        factories[factory.descriptor.id] = factory
    }
}

```



Step 3: Run the test, confirm fails

Run: `./gradlew :registry:test --tests`
`"*DefaultPluginRegistryFalsifiabilityTest*" Expected: FAIL —`
`expected "second" but was "first".`



Step 4: Revert the break

Restore DefaultPluginRegistry.kt. `git diff clean.`



Step 5: Re-run, confirm passes.



Step 6: Evidence

```

{
    "task": "sdk-1.20",
    "subject": "DefaultPluginRegistry duplicate-id semantics",
    "test": "DefaultPluginRegistryFalsifiabilityTest.re-registering
        same id replaces",
    "mutation": "Changed register() to silently ignore duplicates",
    "observed": "expected \"second\" but was \"first\"",
}

```

```
"reverted": "Restored last-write-wins via git checkout",  
"ledger_clause": "6.A"  
}
```



Step 7: Commit.

Section F — Tracker-SDK :testing module

Task 1.21: Create :testing module skeleton

Files:

- Create: testing/build.gradle.kts



Step 1:

```
plugins { alias(libs.plugins.kotlin.jvm) }  
  
dependencies {  
    api(project(":api"))  
    api(project(":mirror"))  
    api(project(":registry"))  
    api(libs.kotlinx.coroutines.core)  
    api(libs.junit) // intentionally api: consumers use JUnit  
                    assertions in tests built on this module  
    implementation(libs.truth)  
}
```



Step 2: Verify, commit.

```
mkdir -p testing/src/main/kotlin/lava/sdk/testing  
git add testing/  
git commit -m "sdk-1.21: scaffold :testing module"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.22: FakeMirrorManager and FakeHealthProbe

Files:

- Create: testing/src/main/kotlin/lava/sdk/testing/
FakeHealthProbe.kt
- Create: testing/src/main/kotlin/lava/sdk/testing/
FakeMirrorManager.kt

- Create: testing/src/test/kotlin/lava/sdk/testing/
FakeMirrorManagerTest.kt



Step 1: Implement FakeHealthProbe

```
package lava.sdk.testing

import lava.sdk.api.HealthState
import lava.sdk.api.MirrorUrl
import lava.sdk.mirror.HealthProbe

/** Test probe that returns the configured state per mirror,
    defaulting to HEALTHY. */
class FakeHealthProbe(
    initial: Map<String, HealthState> = emptyMap(),
    private val default: HealthState = HealthState.HEALTHY,
) : HealthProbe {
    private val states: MutableMap<String, HealthState> =
        initial.toMutableMap()
    fun setState(url: String, state: HealthState) { states[url] =
        state }
    override suspend fun probe(endpoint: MirrorUrl) =
        states[endpoint.url] ?: default
}
```



Step 2: Implement FakeMirrorManager (delegates to DefaultMirrorManager with FakeHealthProbe, exposes hooks for tests)

```
package lava.sdk.testing

import lava.sdk.api.MirrorState
import lava.sdk.api.MirrorUrl
import lava.sdk.mirror.DefaultMirrorManager
import lava.sdk.mirror.MirrorGroup
import lava.sdk.mirror.MirrorManager

class FakeMirrorManager(groups: List<MirrorGroup>) :
    MirrorManager {
    val probe = FakeHealthProbe()
    private val delegate = DefaultMirrorManager(initialGroups =
        groups, healthProbe = probe)

    suspend fun setHealth(url: String, state:
        lava.sdk.api.HealthState) {
        probe.setState(url, state)
        // Trigger probeAll so the state flow updates immediately
        groups().forEach { delegate.probeAll(it) }
    }

    private fun groups() = delegate.run {
        // No public groups() accessor; introspect via
        observeHealth – for the fake we expose names
    }
}
```

```

        listOf("g") // FakeMirrorManager assumes single-group
                      scenarios; multi-group fakes extend.
    }

    override suspend fun getHealthyMirror(groupId: String):
        MirrorUrl? = delegate.getHealthyMirror(groupId)
    override suspend fun <T> executeWithFallback(groupId: String,
        op: suspend (MirrorUrl) -> T): T =
        delegate.executeWithFallback(groupId, op)
    override fun observeHealth(groupId: String) =
        delegate.observeHealth(groupId)
    override suspend fun probeAll(groupId: String) =
        delegate.probeAll(groupId)
    override suspend fun reportSuccess(endpoint: MirrorUrl) =
        delegate.reportSuccess(endpoint)
    override suspend fun reportFailure(endpoint: MirrorUrl,
        cause: Throwable) =
        delegate.reportFailure(endpoint, cause)
}

```



Step 3: Test

```
package lava.sdk.testing
```

```

import com.google.common.truth.Truth.assertThat
import kotlinx.coroutines.test.runTest
import lava.sdk.api.HealthState
import lava.sdk.api.MirrorUrl
import lava.sdk.mirror.MirrorGroup
import org.junit.Test

class FakeMirrorManagerTest {
    @Test
    fun `setHealth toggles mirror state visible to
        executeWithFallback`() = runTest {
        val mgr = FakeMirrorManager(listOf(MirrorGroup(
            groupId = "g",
            mirrors = listOf(MirrorUrl("https://a", priority =
                0), MirrorUrl("https://b", priority = 1)),
            expectedMarker = "marker",
        )))
        mgr.setHealth("https://a", HealthState.UNHEALTHY)
        val tried = mutableListOf<String>()
        mgr.executeWithFallback("g") { m -> tried += m.url; "ok" }
        assertThat(tried).containsExactly("https://b")
    }
}

```



Step 4: Run, confirm pass; commit.

```

git add testing/src/
git commit -m "sdk-1.22: FakeMirrorManager + FakeHealthProbe

```

Task 1.23: HtmlFixtureLoader and JsonFixtureLoader

Files:

- Create: testing/src/main/kotlin/lava/sdk/testing/HtmlFixtureLoader.kt
- Create: testing/src/main/kotlin/lava/sdk/testing/JsonFixtureLoader.kt
- Create: testing/src/test/kotlin/lava/sdk/testing/HtmlFixtureLoaderTest.kt
- Create: testing/src/test/resources/fixtures/sample.html



Step 1: Test fixture file

testing/src/test/resources/fixtures/sample.html:

```
<!DOCTYPE html>
<html><body><h1>Sample</h1></body></html>
```



Step 2: Test

```
package lava.sdk.testing
```

```
import com.google.common.truth.Truth.assertThat
import org.junit.Test
```

```
class HtmlFixtureLoaderTest {
    @Test
    fun `loads fixture from classpath resource`() {
        val loader = HtmlFixtureLoader(resourceRoot = "fixtures")
        val html = loader.load("sample.html")
        assertThat(html).contains("<h1>Sample</h1>")
    }

    @Test
    fun `throws on missing fixture with helpful message`() {
        val loader = HtmlFixtureLoader(resourceRoot = "fixtures")
        try {
            loader.load("missing.html")
            error("expected")
        } catch (e: IllegalArgumentException) {
            assertThat(e.message).contains("missing.html")
            assertThat(e.message).contains("fixtures")
        }
    }
}
```

```
}
}
```

☐

Step 3: Implement

```
package lava.sdk.testing
```

```
class HtmlFixtureLoader(private val resourceRoot: String) {
    fun load(relativePath: String): String {
        val full = "$resourceRoot/$relativePath"
        val stream =
            HtmlFixtureLoader::class.java.getResourceAsStream(full)
            ?: throw IllegalArgumentException(
                "Fixture not found: $relativePath under
                $resourceRoot (looked for $full)"
            )
        return stream.bufferedReader().use { it.readText() }
    }
}
```

```
package lava.sdk.testing
```

```
import kotlinx.serialization.json.Json
import kotlinx.serialization.json.JsonElement

class JsonFixtureLoader(private val resourceRoot: String) {
    private val json = Json { ignoreUnknownKeys = true }
    fun loadElement(relativePath: String): JsonElement =
        json.parseToJsonElement(loadString(relativePath))
    fun loadString(relativePath: String): String =
        JsonFixtureLoader::class.java.getResourceAsStream("/
        $resourceRoot/$relativePath")
        ?.bufferedReader()?.use { it.readText() }
        ?: throw IllegalArgumentException("Fixture not found:
        $relativePath")
}
```

☐

Step 4: Run, commit.

```
git add testing/src/main/kotlin/lava/sdk/testing/
      HtmlFixtureLoader.kt \
      testing/src/main/kotlin/lava/sdk/testing/
      JsonFixtureLoader.kt \
      testing/src/test/kotlin/lava/sdk/testing/
      HtmlFixtureLoaderTest.kt \
      testing/src/test/resources/fixtures/sample.html
git commit -m "sdk-1.23: HtmlFixtureLoader + JsonFixtureLoader"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.24: falsifiabilityRehearsal helper + RehearsalRecord

This is the **load-bearing testing primitive** — every consumer of this submodule will use this helper to record their own bluff-audit evidence per Sixth Law clause 6.A.

Files:

- Create: testing/src/main/kotlin/lava/sdk/testing/RehearsalRecord.kt
- Create: testing/src/main/kotlin/lava/sdk/testing/FalsifiabilityRehearsal.kt
- Create: testing/src/test/kotlin/lava/sdk/testing/FalsifiabilityRehearsalTest.kt



Step 1: Write RehearsalRecord

```
package lava.sdk.testing
```

```
import kotlinx.datetime.Clock
import kotlinx.datetime.Instant
import kotlinx.serialization.Serializable
```

```
@Serializable
```

```
data class RehearsalRecord(
    val testName: String,
    val mutationDescription: String,
    val observedFailureMessage: String,
    val revertedSuccessfully: Boolean,
    val recordedAt: Instant = Clock.System.now(),
    val ledgerClause: String = "6.A",
)
```



Step 2: Write FalsifiabilityRehearsal

```
package lava.sdk.testing
```

```
/**
 * Run a test, with a temporary mutation applied, asserting the
 * test fails;
 * then revert the mutation and assert the test passes.
 *
 * Usage:
 * ```
 * val rec = falsifiabilityRehearsal(
 *     name = "fallback skips UNHEALTHY",
 *     mutationDescription = "removed UNHEALTHY filter from
 *         executeWithFallback",
 *     applyMutation = { /* temp edit; return AutoCloseable that
 *         reverts */ },
 *     runTest = { runUnderTest() }
 * )
```

```

* evidenceWriter.write(rec)
* ```
*
* The runner is given the mutation lifecycle as code, not as
  comment, so
* the rehearsal is repeatable, deterministic, and auditable.
*/
inline fun falsifiabilityRehearsal(
    name: String,
    mutationDescription: String,
    applyMutation: () -> AutoCloseable,
    runTest: () -> Unit,
): RehearsalRecord {
    val handle = applyMutation()
    val observedFailure = try {
        runTest()
        // If we reach here, the test passed under the mutation –
        // that's a bluff.
        return RehearsalRecord(
            testName = name,
            mutationDescription = mutationDescription,
            observedFailureMessage = "FALSIFIABILITY VIOLATION:
test passed under mutation",
            revertedSuccessfully = false,
        )
    } catch (t: Throwable) {
        t.message ?: t::class.simpleName ?: "(no message)"
    } finally {
        handle.close()
    }

    // Now run with mutation reverted; this should pass.
    val passedAfterRevert = try { runTest(); true } catch (_:
    Throwable) { false }

    return RehearsalRecord(
        testName = name,
        mutationDescription = mutationDescription,
        observedFailureMessage = observedFailure,
        revertedSuccessfully = passedAfterRevert,
    )
}

```



Step 3: Test the helper

```

package lava.sdk.testing

import com.google.common.truth.Truth.assertThat
import org.junit.Test

class FalsifiabilityRehearsalTest {

```



```

@Test
fun `rehearsal returns record when mutation causes failure
and revert restores`() {
    var brokenFlag = false
    val rec = falsifiabilityRehearsal(
        name = "demo",
        mutationDescription = "set brokenFlag",
        applyMutation = {
            brokenFlag = true
            AutoCloseable { brokenFlag = false }
        },
        runTest = {
            if (brokenFlag) error("system is broken")
            // else: pass
        }
    )
    assertThat(rec.testName).isEqualTo("demo")
    assertThat(rec.mutationDescription).isEqualTo("set
brokenFlag")

    assertThat(rec.observedFailureMessage).contains("system is
broken")
    assertThat(rec.revertedSuccessfully).isTrue()
}

@Test
fun `rehearsal flags violation when mutation does not cause
failure`() {
    val rec = falsifiabilityRehearsal(
        name = "tautology",
        mutationDescription = "set a flag nobody reads",
        applyMutation = { AutoCloseable { } },
        runTest = { /* always passes */ }
    )
    assertThat(rec.observedFailureMessage).contains("FALSIFIABILITY
VIOLATION")
    assertThat(rec.revertedSuccessfully).isFalse()
}
}

```



Step 4: Run, commit.

```

git add testing/src/main/kotlin/lava/sdk/testing/
    RehearsalRecord.kt \
    testing/src/main/kotlin/lava/sdk/testing/
    FalsifiabilityRehearsal.kt \
    testing/src/test/kotlin/lava/sdk/testing/
    FalsifiabilityRehearsalTest.kt
git commit -m "sdk-1.24: falsifiabilityRehearsal helper +
    RehearsalRecord

```

The load-bearing primitive Sixth Law clause 6.A consumers depend on.

Catches tautological tests by detecting when a mutation does NOT cause the test to fail.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Section G — Mirror SDK to four upstreams + tag 0.1.0

Task 1.25: Initial four-upstream mirror push of Tracker-SDK



Step 1: Run `./scripts/ci.sh` from inside the SDK clone, confirm passes

```
cd /tmp/tracker-sdk-init/Tracker-SDK
./scripts/ci.sh
```

Expected: all gates pass.



Step 2: Run `./scripts/sync-mirrors.sh`

```
./scripts/sync-mirrors.sh
```

Expected: pushes to all four upstreams. Final line: "All four upstreams converged on <sha>".

If any upstream rejects the push (branch protection, missing remote, auth issue), resolve and re-run. Common fix: branch protection from Task 1.1 Step 4 may need to be temporarily disabled for the very first push. Re-enable after.



Step 3: Re-apply branch protection on the now-existing master branch

```
gh api repos/vasic-digital/Tracker-SDK/branches/master/protection
\
--method PUT --input - <<'EOF'
{
  "required_status_checks": null,
  "enforce_admins": false,
  "required_pull_request_reviews": null,
  "restrictions": null,
  "allow_force_pushes": false,
  "allow_deletions": false
}
EOF
```

(Repeat the equivalent for GitLab and GitVerse via their APIs.)



Step 4: Verify by re-running sync — should be a no-op

```
./scripts/sync-mirrors.sh
```

Expected: prints "All four upstreams converged on <same sha>" without pushing new commits.

Task 1.26: Verify per-mirror SHA convergence and record evidence

Files (Lava repo side):

- Create: `.lava-ci-evidence/sdk-mirror-sync/initial-push-<sha>.json`



Step 1: Capture per-upstream SHAs

```
cd /tmp/tracker-sdk-init/Tracker-SDK
EXPECTED=$(git rev-parse master)
for r in github gitflic gitlab gitverse; do
    ACTUAL=$(git ls-remote "$r" master | awk '{print $1}')
    echo "$r=$ACTUAL"
done
```

Expected: all four print the same SHA == EXPECTED.



Step 2: Write evidence in the Lava repo

In the Lava repo (`/run/media/milosvasic/DATA4TB/Projects/Lava/`):

```
mkdir -p .lava-ci-evidence/sdk-mirror-sync
SHA=<sha from step 1>
cat > .lava-ci-evidence/sdk-mirror-sync/initial-push-{SHA}.json
    <<EOF
{
  "task": "sdk-1.26",
  "subject": "Initial four-upstream push of vasic-digital/Tracker-
    SDK",
  "expected_sha": "{SHA}",
  "github": "{SHA}",
  "gitflic": "{SHA}",
  "gitlab": "{SHA}",
  "gitverse": "{SHA}",
  "ledger_clause": "6.C"
}
EOF
git add .lava-ci-evidence/sdk-mirror-sync/initial-push-{SHA}.json
git commit -m "sp3a-1.26: SDK initial push - per-mirror SHA
    convergence verified"
```

All four upstreams converged on `{SHA}`.

Task 1.27: Tag Tracker-SDK 0.1.0 on all four upstreams



Step 1: Tag in the SDK clone

```
cd /tmp/tracker-sdk-init/Tracker-SDK
git tag -a v0.1.0 -m "Tracker-SDK 0.1.0 - initial release"
```

```
Modules: :api, :mirror, :registry, :testing.
Consumers: Lava (SP-3a Phase 1)."
```



Step 2: Push tag to all four upstreams

```
./scripts/sync-mirrors.sh # already pushes --tags
```

Verify: `git ls-remote --tags github | grep v0.1.0` shows the tag on all four.



Step 3: Record evidence in Lava repo

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
TAG_SHA=$(cd /tmp/tracker-sdk-init/Tracker-SDK && git rev-list -n
1 v0.1.0)
cat > .lava-ci-evidence/sdk-mirror-sync/v0.1.0-tag.json <<EOF
{
  "task": "sdk-1.27",
  "tag": "v0.1.0",
  "tag_sha": "${TAG_SHA}",
  "ledger_clause": "6.C"
}
EOF
git add .lava-ci-evidence/sdk-mirror-sync/v0.1.0-tag.json
git commit -m "sp3a-1.27: Tracker-SDK v0.1.0 tag - record SHA $
(echo ${TAG_SHA} | head -c 7)"
```

Section H — Pin submodule in Lava

Task 1.28: Add submodules/tracker_sdk/ as a git submodule

Files (Lava repo):

- Create: submodules/tracker_sdk/ (via git submodule add)
- Modify: .gitmodules



Step 1: Pre-flight check

```
cd /run/media/milosvasic/DATA4TB/Projects/Lava
ls submodules/
```

Expected: existing 15 submodules (Auth, Cache, Challenges, etc.) listed; no Tracker-SDK yet.



Step 2: Add the submodule pinned at v0.1.0

```
git submodule add git@github.com:vasic-digital/Tracker-SDK.git
                  submodules/tracker_sdk
cd submodules/tracker_sdk
git checkout v0.1.0
cd ../../
```



Step 3: Verify pin

Run: cd submodules/tracker_sdk && git describe --tags Expected: v0.1.0.

Run: cat .gitmodules | tail -5 Expected: contains a stanza for submodules/tracker_sdk.



Step 4: Commit the submodule pin

```
git add .gitmodules submodules/tracker_sdk
git commit -m "sp3a-1.28: pin vasic-digital/Tracker-SDK at v0.1.0"
```

Submodule mounted at submodules/tracker_sdk/, frozen by default.
Updating the pin is a deliberate PR per the Decoupled Reusable Architecture rule.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.29: Verify the submodule fetches cleanly on a fresh clone



Step 1: Test in a temp directory

```
cd /tmp
git clone --recurse-submodules <Lava-upstream-url> lava-fresh
cd lava-fresh
ls submodules/tracker_sdk/api/src/main/kotlin/lava/sdk/api/
    MirrorUrl.kt
```

Expected: file exists.



Step 2: Verify the submodule's CI script runs from inside the Lava clone

```
cd submodules/tracker_sdk
./scripts/ci.sh
```

Expected: all gates pass (means the SDK is fetched + buildable from the consumer side).



Step 3: Cleanup

```
rm -rf /tmp/lava-fresh
```

No commit in this task — verification only.

Section I — New convention plugin in Lava buildSrc/

Task 1.30: Create KotlinTrackerModuleConventionPlugin

Files (Lava repo):

- Create: buildSrc/src/main/kotlin/
KotlinTrackerModuleConventionPlugin.kt



Step 1: Read existing convention plugin pattern

Run: cat buildSrc/src/main/kotlin/KotlinLibraryConventionPlugin.kt
Note the structure: applies java-library, kotlin.jvm, configures JVM 17.



Step 2: Create the new plugin

```
// buildSrc/src/main/kotlin/KotlinTrackerModuleConventionPlugin.kt
import lava.conventions.StaticAnalysisConventionPlugin
import org.gradle.api.JavaVersion
import org.gradle.api.Plugin
```

```

import org.gradle.api.Project
import org.gradle.api.artifacts.VersionCatalogsExtension
import org.gradle.api.tasks.compile.JavaCompile
import org.gradle.kotlin.dsl.dependencies
import org.gradle.kotlin.dsl.getByType
import org.gradle.kotlin.dsl.withType
import org.jetbrains.kotlin.gradle.dsl.JvmTarget
import org.jetbrains.kotlin.gradle.tasks.KotlinJvmCompile

/**
 * Convention plugin for tracker plugin modules
 * (e.g. :core:tracker:ruTracker, :core:tracker:rutor).
 * Pre-wires:
 * - java-library + kotlin.jvm + serialization
 * - dependency on :core:tracker:api
 * - dependency on submodules/tracker_sdk/api + mirror
 * - Jsoup, OkHttp, kotlinx-serialization, kotlinx-coroutines
 * - testImplementation: junit, kotlinx-coroutines-test, mockk,
 *   testing harness
 */
class KotlinTrackerModuleConventionPlugin : Plugin<Project> {
    override fun apply(target: Project) {
        with(target) {
            with(pluginManager) {
                apply("java-library")
                apply("org.jetbrains.kotlin.jvm")
                apply("org.jetbrains.kotlin.plugin.serialization")
                apply(StaticAnalysisConventionPlugin::class.java)
            }

            tasks.withType<JavaCompile>().configureEach {
                sourceCompatibility =
                    JavaVersion.VERSION_17.toString()
                targetCompatibility =
                    JavaVersion.VERSION_17.toString()
            }
            tasks.withType<KotlinJvmCompile>().configureEach {
                compilerOptions {
                    jvmTarget.set(JvmTarget.JVM_17)
                }

                val libs =
                    extensions.getByType<VersionCatalogsExtension>().named("libs")

                dependencies {
                    add("api", project(":core:tracker:api"))
                    // Tracker-SDK submodule via composite/included
                    build
                    add("api", project(":sdk-api")) // alias
                    defined in settings.gradle.kts
                    add("api", project(":sdk-mirror"))

                    add("implementation",
                        libs.findLibrary("jsoup").get())
                }
            }
        }
    }
}

```

```

        add("implementation",
libs.findLibrary("okhttp.core").get())
        add("implementation",
libs.findLibrary("kotlinx.serialization.json").get())
        add("implementation",
libs.findLibrary("kotlinx.coroutines.core").get())

        add("testImplementation",
libs.findLibrary("junit").get())
        add("testImplementation",
libs.findLibrary("kotlinx.coroutines.test").get())
        add("testImplementation",
libs.findLibrary("mockk").get())
        add("testImplementation",
project(":core:tracker:testing"))
        add("testImplementation", project(":sdk-testing"))
    }
}
}
}

```



Step 3: Register the plugin in buildSrc/build.gradle.kts

Read: cat buildSrc/build.gradle.kts Find the gradlePlugin { plugins { ... } } block; add:

```

register("kotlinTrackerModuleConvention") {
    id = "lava.kotlin.tracker.module"
    implementationClass = "KotlinTrackerModuleConventionPlugin"
}

```



Step 4: Add libs.versions.toml entries for any aliases not already present

Verify in gradle/libs.versions.toml:

- jsoup library present (existing for rutracker module)
- okhttp.core library
- mockk library
- kotlinx.coroutines.test library
- kotlinx.coroutines.core library
- kotlinx.serialization.json library

If any are missing, add them. Reference the existing entries for version pinning style.



Step 5: Verify the new plugin compiles

Run: ./gradlew :buildSrc:assemble Expected: BUILD SUCCESSFUL.



Step 6: Commit


```
git add buildSrc/src/main/kotlin/
    KotlinTrackerModuleConventionPlugin.kt \
    buildSrc/build.gradle.kts \
    gradle/libs.versions.toml
git commit -m "sp3a-1.30: lava.kotlin.tracker.module convention
    plugin"
```

Pre-wires :core:tracker:api + Tracker-SDK submodule + Jsoup/
OkHttp/
serialization for tracker plugin modules (RuTracker, RuTor, future
trackers). Eliminates per-module dependency boilerplate.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.31: Wire the Tracker-SDK submodule projects into Lava's settings.gradle.kts (composite build)

Files:

- Modify: settings.gradle.kts



Step 1: Read current settings

Run: `cat settings.gradle.kts | head -60`



Step 2: Add the SDK projects as included builds

The SDK is a separate Gradle project tree. Lava consumes it via Gradle composite builds. Edit settings.gradle.kts:

After the `include(...)` calls, before any closing brace, add:

```
// Tracker-SDK submodule – composite build (pinned via git
    submodule)
includeBuild("submodules/tracker_sdk") {
    dependencySubstitution {
        substitute(module("lava.sdk:api")).using(project(":api"))
        substitute(module("lava.sdk:mirror")).using(project(":mirror"))
        substitute(module("lava.sdk:registry")).using(project(":registry"))
        substitute(module("lava.sdk:testing")).using(project(":testing"))
    }
}
```

This lets Lava modules declare dependencies
{ `implementation("lava.sdk:mirror")` } and Gradle resolves to the local
submodule projects.

(Alternative wiring via `include(":sdk-api")` etc. with `projectDir = ...` does NOT work cleanly across composite builds; `includeBuild` is the established Gradle pattern.)



Step 3: Update the convention plugin to use the new project coordinates

Edit `buildSrc/src/main/kotlin/KotlinTrackerModuleConventionPlugin.kt`:

Change:

```
add("api", project(":sdk-api"))
add("api", project(":sdk-mirror"))
```

to:

```
add("api", "lava.sdk:api")
add("api", "lava.sdk:mirror")
```

And `add("testImplementation", project(":sdk-testing"))` → `"lava.sdk:testing"`.



Step 4: Verify the build resolves SDK dependencies

Create a smoke-test module to verify wiring works (delete after):

```
mkdir -p /tmp/sdk-smoke/src/main/kotlin
cat > /tmp/sdk-smoke/build.gradle.kts <<'EOF'
plugins { kotlin("jvm") version "2.1.0" }
dependencies { implementation("lava.sdk:api") }
EOF
```

Or, more directly, run:

```
./gradlew :app:dependencies --configuration runtimeClasspath |
    grep -i "lava.sdk" || echo "no sdk deps yet – that's
    expected before Phase 2"
```

Expected: no error; the wiring resolves at the build-script level even if no module uses it yet.



Step 5: Commit

```
git add settings.gradle.kts buildSrc/src/main/kotlin/
    KotlinTrackerModuleConventionPlugin.kt
git commit -m "sp3a-1.31: composite-build wire Tracker-SDK into
    Lava"
```

`submodules/tracker_sdk` is now an `includeBuild`; its projects are addressable as `'lava.sdk:<module>'` from any Lava module via the `dependencySubstitution` map.

Section J — :core:tracker:api module (in this repo)

Task 1.32: Create module skeleton + AuthType + TrackerCapability + Protocol re-export

Files:

- Create: core/tracker/api/build.gradle.kts
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/AuthType.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/TrackerCapability.kt



Step 1: Create build.gradle.kts

```
plugins {
    id("lava.kotlin.library")
    id("lava.kotlin.serialization")
}

dependencies {
    api("lava.sdk:api") // re-exports MirrorUrl,
                        Protocol, etc.
    api(libs.kotlinx.coroutines.core)
    api(libs.kotlinx.datetime)

    testImplementation(libs.junit)
    testImplementation(libs.kotlinx.coroutines.test)
    testImplementation(libs.mockk)
}
```



Step 2: Add module to settings.gradle.kts

Edit settings.gradle.kts. After existing include(":core:network:rutracker") line, add (or in a sensible place near other core modules):

```
include(":core:tracker:api")
include(":core:tracker:client")
include(":core:tracker:rutracker") // will exist after Phase 2
                                git mv
include(":core:tracker:rutor")    // will exist after Phase 3
include(":core:tracker:testing")
```

(:core:tracker:rutracker and :core:tracker:rutor are added as anticipations — they don't have build.gradle.kts yet but listing them now keeps the include block organized. They'll be a no-op on ./gradlew tasks until their build files exist. If Gradle complains, comment them out and uncomment in their respective phases.)



Step 3: Implement AuthType

```
// core/tracker/api/src/main/kotlin/lava/tracker/api/AuthType.kt
package lava.tracker.api

import kotlinx.serialization.Serializable

@Serializable
enum class AuthType { NONE, FORM_LOGIN, CAPTCHA_LOGIN, OAUTH,
    API_KEY }
```



Step 4: Implement TrackerCapability

```
// core/tracker/api/src/main/kotlin/lava/tracker/api/
    TrackerCapability.kt
package lava.tracker.api

import kotlinx.serialization.Serializable

@Serializable
enum class TrackerCapability {
    SEARCH, BROWSE, FORUM, TOPIC, COMMENTS, FAVORITES,
    TORRENT_DOWNLOAD, MAGNET_LINK, AUTH_REQUIRED,
    CAPTCHA_LOGIN, RSS, UPLOAD, USER_PROFILE
}
```



Step 5: Tests

```
// core/tracker/api/src/test/kotlin/lava/tracker/api/
    TrackerCapabilityTest.kt
package lava.tracker.api

import org.junit.Assert.assertEquals
import org.junit.Test

class TrackerCapabilityTest {
    @Test
    fun `enum has 13 values matching spec`() {
        assertEquals(13, TrackerCapability.entries.size)
    }
    @Test
    fun `contains all named capabilities`() {
        val names = TrackerCapability.entries.map {
            it.name }.toSet()
        assertEquals(
```

```

        setOf("SEARCH", "BROWSE", "FORUM", "TOPIC", "COMMENTS", "FAVORITES",
            "TORRENT_DOWNLOAD", "MAGNET_LINK", "AUTH_REQUIRED",
            "CAPTCHA_LOGIN", "RSS", "UPLOAD", "USER_PROFILE"),
        names
    )
}
}

```



Step 6: Run tests, confirm pass

Run: `./gradlew :core:tracker:api:test` Expected: PASS — 2 tests green.



Step 7: Commit

```

git add core/tracker/api/build.gradle.kts \
    core/tracker/api/src/main/kotlin/lava/tracker/api/
    AuthType.kt \
    core/tracker/api/src/main/kotlin/lava/tracker/api/
    TrackerCapability.kt \
    core/tracker/api/src/test/kotlin/lava/tracker/api/
    TrackerCapabilityTest.kt \
    settings.gradle.kts
git commit -m "sp3a-1.32: scaffold :core:tracker:api + AuthType +
    TrackerCapability

```

13-value capability enum, 5-value AuthType. Tests assert exact value sets so additions/removals are caught.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.33: TrackerDescriptor, TrackerClient, TrackerFeature interfaces

Files:

- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/TrackerFeature.kt`
- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/TrackerDescriptor.kt`
- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/TrackerClient.kt`



Step 1: Implement TrackerFeature (marker)

package `lava.tracker.api`

```
/** Marker interface for capability-typed feature interfaces. */
```

```
interface TrackerFeature
```



Step 2: Implement TrackerDescriptor

```
package lava.tracker.api
```

```
import kotlinx.serialization.Serializable
```

```
import lava.sdk.api.HasId
```

```
import lava.sdk.api.MirrorUrl
```

```
@Serializable
```

```
interface TrackerDescriptor : HasId {
```

```
    /** Stable identifier, e.g. "rutracker", "rutor". Equal to  
        [id] from HasId. */
```

```
    val trackerId: String
```

```
    override val id: String get() = trackerId
```

```
    /** Human-readable display name shown in UI. */
```

```
    val displayName: String
```

```
    /** Primary + mirror URLs. The first MirrorUrl with  
        isPrimary=true is the canonical address. */
```

```
    val baseUrls: List<MirrorUrl>
```

```
    /** Capabilities this tracker actually supports – not  
        declarations of intent. Constitutional 6.E. */
```

```
    val capabilities: Set<TrackerCapability>
```

```
    /** Authentication mechanism. */
```

```
    val authType: AuthType
```

```
    /** Encoding used by the tracker, e.g. "UTF-8",  
        "Windows-1251". */
```

```
    val encoding: String
```

```
    /** Substring (case-insensitive) that must appear on the  
        tracker's root page for a HEALTHY probe. */
```

```
    val expectedHealthMarker: String
```

```
}
```



Step 3: Implement TrackerClient

```
package lava.tracker.api
```

```
import kotlin.reflect.KClass
```

```
interface TrackerClient : AutoCloseable {
```

```
    val descriptor: TrackerDescriptor
```

```
    /** Lightweight liveness probe. Used by MirrorManager and SDK  
        init. */
```

```

suspend fun healthCheck(): Boolean

/**
 * Returns a feature-interface implementation if the tracker
 * supports the requested
 * capability, or null otherwise. Constitutional clause 6.E
 * (Capability Honesty)
 * requires: capability declared in descriptor ⇒ this returns
 * non-null.
 */
fun <T : TrackerFeature> getFeature(featureClass: KClass<T>):
    T?
}

```



Step 4: Test the interfaces by writing a minimal in-test impl

```

// core/tracker/api/src/test/kotlin/lava/tracker/api/
// TrackerDescriptorContractTest.kt
package lava.tracker.api

import lava.sdk.api.MirrorUrl
import org.junit.Assert.assertEquals
import org.junit.Test
import kotlin.reflect.KClass

class TrackerDescriptorContractTest {
    private object SampleDescriptor : TrackerDescriptor {
        override val trackerId = "sample"
        override val displayName = "Sample Tracker"
        override val baseUrls = listOf(MirrorUrl("https://
sample.example", isPrimary = true))
        override val capabilities =
            setOf(TrackerCapability.SEARCH)
        override val authType = AuthType.NONE
        override val encoding = "UTF-8"
        override val expectedHealthMarker = "Sample"
    }

    private class SampleClient : TrackerClient {
        override val descriptor = SampleDescriptor
        override suspend fun healthCheck() = true
        @Suppress("UNCHECKED_CAST")
        override fun <T : TrackerFeature>
            getFeature(featureClass: KClass<T>): T? = null
        override fun close() {}
    }

    @Test
    fun `descriptor exposes id from HasId equal to trackerId`() {
        assertEquals("sample", SampleDescriptor.id)
        assertEquals(SampleDescriptor.id,
            SampleDescriptor.trackerId)
    }
}

```

```

@Test
fun `client returns null for unsupported feature`() {
    val client = SampleClient()
    // Define a fake feature interface for this test
    interface SampleFeature : TrackerFeature
    val result = client.getFeature(SampleFeature::class)
    assertEquals(null, result)
}
}

```

(Note: the inline interface `SampleFeature` inside a `@Test` method is valid in Kotlin local-class syntax. If your Kotlin version warns, lift it to a top-level type in the test file.)



Step 5: Run, confirm pass

Run: `./gradlew :core:tracker:api:test` Expected: PASS.



Step 6: Commit

```

git add core/tracker/api/src/main/kotlin/lava/tracker/api/
TrackerFeature.kt \
core/tracker/api/src/main/kotlin/lava/tracker/api/
TrackerDescriptor.kt \
core/tracker/api/src/main/kotlin/lava/tracker/api/
TrackerClient.kt \
core/tracker/api/src/test/kotlin/lava/tracker/api/
TrackerDescriptorContractTest.kt
git commit -m "sp3a-1.33: TrackerClient + TrackerDescriptor +
TrackerFeature

```

Capability-based contract replacing monolithic `NetworkApi`.
`getFeature()`
returns null for unsupported capabilities – no Not-Implemented
stubs.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.34: Common data models (TorrentItem, SearchRequest, SearchResult, etc.)

Files:

- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/model/TorrentItem.kt`
- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/model/TorrentFile.kt`
- Create: `core/tracker/api/src/main/kotlin/lava/tracker/api/model/SortField.kt`

- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/SortOrder.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/TimePeriod.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/SearchRequest.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/SearchResult.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/BrowseResult.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/TopicDetail.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/TopicPage.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/CommentsPage.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/Comment.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/ForumTree.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/ForumCategory.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/LoginRequest.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/LoginResult.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/AuthState.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/CaptchaChallenge.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/model/CaptchaSolution.kt



Step 1: Create the enums

```
// SortField.kt
package lava.tracker.api.model
import kotlinx.serialization.Serializable
@Serializable enum class SortField { DATE, SEEDERS, LEECHERS,
    SIZE, RELEVANCE, TITLE }

// SortOrder.kt
package lava.tracker.api.model
import kotlinx.serialization.Serializable
@Serializable enum class SortOrder { ASCENDING, DESCENDING }

// TimePeriod.kt
package lava.tracker.api.model
import kotlinx.serialization.Serializable
@Serializable enum class TimePeriod { LAST_DAY, LAST_WEEK,
    LAST_MONTH, LAST_YEAR, ALL_TIME }
```



Step 2: Create TorrentItem and TorrentFile

```
// TorrentItem.kt
package lava.tracker.api.model

import kotlinx.datetime.Instant
import kotlinx.serialization.Serializable

@Serializable
data class TorrentItem(
    val trackerId: String,
    val torrentId: String,
    val title: String,
    val sizeBytes: Long? = null,
    val seeders: Int? = null,
    val leechers: Int? = null,
    val infoHash: String? = null,
    val magnetUri: String? = null,
    val downloadUrl: String? = null,
    val detailUrl: String? = null,
    val category: String? = null,
    val publishDate: Instant? = null,
    val metadata: Map<String, String> = emptyMap(),
)

// TorrentFile.kt
package lava.tracker.api.model
import kotlinx.serialization.Serializable
@Serializable data class TorrentFile(val name: String, val
    sizeBytes: Long?)
```



Step 3: Create the request/result types

```
// SearchRequest.kt
package lava.tracker.api.model

import kotlinx.serialization.Serializable

@Serializable
data class SearchRequest(
    val query: String,
    val categories: List<String> = emptyList(),
    val sort: SortField = SortField.DATE,
    val sortOrder: SortOrder = SortOrder.DECENDING,
    val author: String? = null,
    val period: TimePeriod? = null,
)

// SearchResult.kt
package lava.tracker.api.model
import kotlinx.serialization.Serializable
@Serializable data class SearchResult(
```

```

        val items: List<TorrentItem>,
        val totalPages: Int,
        val currentPage: Int,
    )

    // BrowseResult.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class BrowseResult(
        val items: List<TorrentItem>,
        val totalPages: Int,
        val currentPage: Int,
        val category: ForumCategory? = null,
    )

```



Step 4: Topic, comments, forum types

```

    // TopicDetail.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class TopicDetail(
        val torrent: TorrentItem,
        val description: String? = null,
        val files: List<TorrentFile> = emptyList(),
    )

    // TopicPage.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class TopicPage(
        val topic: TopicDetail,
        val totalPages: Int,
        val currentPage: Int,
    )

    // Comment.kt
    package lava.tracker.api.model
    import kotlinx.datetime.Instant
    import kotlinx.serialization.Serializable
    @Serializable data class Comment(
        val author: String,
        val timestamp: Instant? = null,
        val body: String,
    )

    // CommentsPage.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class CommentsPage(
        val items: List<Comment>,
        val totalPages: Int,
    )

```

```

        val currentPage: Int,
    )

    // ForumCategory.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class ForumCategory(
        val id: String,
        val name: String,
        val parentId: String? = null,
        val children: List<ForumCategory> = emptyList(),
    )

    // ForumTree.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class ForumTree(val rootCategories:
        List<ForumCategory>)

```



Step 5: Auth-related types

```

    // CaptchaChallenge.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class CaptchaChallenge(val sid: String, val
        code: String, val imageUrl: String)

    // CaptchaSolution.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class CaptchaSolution(val sid: String, val
        code: String, val value: String)

    // AuthState.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable
    sealed class AuthState {
        @Serializable object Authenticated : AuthState()
        @Serializable object Unauthenticated : AuthState()
        @Serializable data class CaptchaRequired(val challenge:
            CaptchaChallenge) : AuthState()
    }

    // LoginRequest.kt
    package lava.tracker.api.model
    import kotlinx.serialization.Serializable
    @Serializable data class LoginRequest(
        val username: String,
        val password: String,
        val captcha: CaptchaSolution? = null,
    )

```

```
// LoginResult.kt
package lava.tracker.api.model
import kotlinx.serialization.Serializable
@Serializable data class LoginResult(
    val state: AuthState,
    val sessionToken: String? = null,
    val captchaChallenge: CaptchaChallenge? = null,
)

```



Step 6: Round-trip serialization tests for the load-bearing types

```
// core/tracker/api/src/test/kotlin/lava/tracker/api/model/
// SerializationRoundTripTest.kt
package lava.tracker.api.model

import kotlinx.datetime.Instant
import kotlinx.serialization.json.Json
import org.junit.Assert.assertEquals
import org.junit.Test

class SerializationRoundTripTest {
    private val json = Json { prettyPrint = false; encodeDefaults = true }

    @Test
    fun `TorrentItem round-trips with all fields`() {
        val original = TorrentItem(
            trackerId = "rutracker",
            torrentId = "12345",
            title = "Ubuntu 24.04",
            sizeBytes = 4_500_000_000L,
            seeders = 100,
            leechers = 5,
            infoHash = "0123456789abcdef0123456789abcdef01234567",
            magnetUri = "magnet:?
xt=urn:btih:0123456789abcdef0123456789abcdef01234567",
            downloadUrl = "https://example/dl/12345",
            detailUrl = "https://example/topic/12345",
            category = "Linux",
            publishDate = Instant.fromEpochSeconds(1714430400),
            metadata = mapOf("rutracker.post.element_type" to
"header"),
        )
        val encoded =
            json.encodeToString(TorrentItem.serializer(), original)
        val decoded =
            json.decodeFromString(TorrentItem.serializer(), encoded)
        assertEquals(original, decoded)
    }

    @Test
    fun `SearchResult round-trips with empty items`() {

```

```

        val original = SearchResult(items = emptyList(),
            totalPages = 0, currentPage = 0)
        val encoded =
            json.encodeToString(SearchResult.serializer(), original)
        val decoded =
            json.decodeFromString(SearchResult.serializer(), encoded)
        assertEquals(original, decoded)
    }

    @Test
    fun `AuthState sealed class round-trips for each variant`() {
        val variants = listOf(
            AuthState.Authenticated,
            AuthState.Unauthenticated,
            AuthState.CaptchaRequired(CaptchaChallenge("sid1",
                "code1", "https://x/cap.png")),
        )
        for (v in variants) {
            val encoded =
                json.encodeToString(AuthState.serializer(), v)
            val decoded =
                json.decodeFromString(AuthState.serializer(), encoded)
            assertEquals(v, decoded)
        }
    }
}

```



Step 7: Run, confirm pass

Run: `./gradlew :core:tracker:api:test` Expected: PASS — all model round-trip tests green.



Step 8: Commit

```

git add core/tracker/api/src/main/kotlin/lava/tracker/api/model/ \
    core/tracker/api/src/test/kotlin/lava/tracker/api/model/
git commit -m "sp3a-1.34: common data model – TorrentItem,
    SearchRequest/Result, etc."

```

Tracker-agnostic data classes replacing the existing RuTracker-specific

DTOs. metadata map carries tracker-specific extras under namespaced keys.

Round-trip serialization tests bind the wire format.

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 1.35: Feature interfaces (Searchable/Browsable/Topic/Comments/Favorites/Authenticatable/Downloadable)

Files:

- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/SearchableTracker.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/BrowsableTracker.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/TopicTracker.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/CommentsTracker.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/FavoritesTracker.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/AuthenticatableTracker.kt
- Create: core/tracker/api/src/main/kotlin/lava/tracker/api/feature/DownloadableTracker.kt



Step 1-7: Implement all seven feature interfaces

// SearchableTracker.kt

```
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
import lava.tracker.api.model.SearchRequest
import lava.tracker.api.model.SearchResult
interface SearchableTracker : TrackerFeature {
    suspend fun search(request: SearchRequest, page: Int = 0):
        SearchResult
}
```

// BrowsableTracker.kt

```
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
import lava.tracker.api.model.BrowseResult
import lava.tracker.api.model.ForumTree
interface BrowsableTracker : TrackerFeature {
    suspend fun browse(category: String?, page: Int): BrowseResult
    /** Returns null when this tracker has no forum tree (e.g.
        RuTor). */
    suspend fun getForumTree(): ForumTree?
}
```

// TopicTracker.kt

```
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
import lava.tracker.api.model.TopicDetail
import lava.tracker.api.model.TopicPage
interface TopicTracker : TrackerFeature {
    suspend fun getTopic(id: String): TopicDetail
    suspend fun getTopicPage(id: String, page: Int): TopicPage
}
```

```
// CommentsTracker.kt
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
import lava.tracker.api.model.CommentsPage
interface CommentsTracker : TrackerFeature {
    suspend fun getComments(topicId: String, page: Int):
        CommentsPage
    /** Returns true on successful add. May trigger
        AuthenticatableTracker.login() upstream. */
    suspend fun addComment(topicId: String, message: String):
        Boolean
}
```

```
// FavoritesTracker.kt
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
import lava.tracker.api.model.TorrentItem
interface FavoritesTracker : TrackerFeature {
    suspend fun list(): List<TorrentItem>
    suspend fun add(id: String): Boolean
    suspend fun remove(id: String): Boolean
}
```

```
// AuthenticatableTracker.kt
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
import lava.tracker.api.model.AuthState
import lava.tracker.api.model.LoginRequest
import lava.tracker.api.model.LoginResult
interface AuthenticatableTracker : TrackerFeature {
    suspend fun login(req: LoginRequest): LoginResult
    suspend fun logout()
    suspend fun checkAuth(): AuthState
}
```

```
// DownloadableTracker.kt
package lava.tracker.api.feature
import lava.tracker.api.TrackerFeature
interface DownloadableTracker : TrackerFeature {
    suspend fun downloadTorrentFile(id: String): ByteArray
    /** Returns null if the magnet URI is not synchronously
        available without an HTTP fetch. */
    fun getMagnetLink(id: String): String?
}
```



Step 8: Run compile + tests

Run: `./gradlew :core:tracker:api:test` Expected: BUILD SUCCESSFUL.



Step 9: Commit


```
git add core/tracker/api/src/main/kotlin/lava/tracker/api/feature/
git commit -m "sp3a-1.35: 7 feature interfaces – Searchable/
    Browseable/Topic/Comments/Favorites/Authenticatable/
    Downloadable"
```

Each is a marker-only interface extending `TrackerFeature`; trackers implement only the features they support. Replaces the 14-method `NetworkApi` with 7 capability-typed contracts.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section K — :core:tracker:registry and :core:tracker:mirror (thin wrappers)

Task 1.36: :core:tracker:registry (Lava-domain wrapper around `lava.sdk:registry`)

Files:

- Create: `core/tracker/registry/build.gradle.kts`
- Create: `core/tracker/registry/src/main/kotlin/lava/tracker/registry/TrackerRegistry.kt`
- Create: `core/tracker/registry/src/main/kotlin/lava/tracker/registry/TrackerClientFactory.kt`
- Create: `core/tracker/registry/src/main/kotlin/lava/tracker/registry/DefaultTrackerRegistry.kt`
- Create: `core/tracker/registry/src/test/kotlin/lava/tracker/registry/DefaultTrackerRegistryTest.kt`



Step 1: Add module to settings

Add to `settings.gradle.kts`: `include(":core:tracker:registry")`



Step 2: Build file

```
plugins { id("lava.kotlin.library") }
dependencies {
    api(project(":core:tracker:api"))
    api("lava.sdk:registry")
    testImplementation(libs.junit)
}
```



Step 3: Implement

```
// TrackerClientFactory.kt
package lava.tracker.registry
```

```

import lava.sdk.api.PluginConfig
import lava.sdk.registry.PluginFactory
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor

interface TrackerClientFactory : PluginFactory<TrackerDescriptor,
    TrackerClient> {
    override val descriptor: TrackerDescriptor
    override fun create(config: PluginConfig): TrackerClient
}

// TrackerRegistry.kt
package lava.tracker.registry

import lava.sdk.registry.PluginRegistry
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor

interface TrackerRegistry : PluginRegistry<TrackerDescriptor,
    TrackerClient> {
    fun trackersWithCapability(capability: TrackerCapability):
        List<TrackerDescriptor>
}

// DefaultTrackerRegistry.kt
package lava.tracker.registry

import lava.sdk.registry.DefaultPluginRegistry
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor

class DefaultTrackerRegistry : TrackerRegistry,
    DefaultPluginRegistry<TrackerDescriptor, TrackerClient>()
{
    override fun trackersWithCapability(capability:
        TrackerCapability): List<TrackerDescriptor> =
        list().filter { capability in it.capabilities }
}

```



Step 4: Test

```

package lava.tracker.registry

import lava.sdk.api.MapPluginConfig
import lava.sdk.api.MirrorUrl
import lava.tracker.api.AuthType
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.TrackerFeature
import org.junit.Assert.assertEquals

```

```

import org.junit.Assert.assertTrue
import org.junit.Test
import kotlin.reflect.KClass

class DefaultTrackerRegistryTest {

    private fun descriptor(id: String, caps:
        Set<TrackerCapability>) = object : TrackerDescriptor {
        override val trackerId = id
        override val displayName = id
        override val baseUrls = listOf(MirrorUrl("https://
            $id.example", isPrimary = true))
        override val capabilities = caps
        override val authType = AuthType.NONE
        override val encoding = "UTF-8"
        override val expectedHealthMarker = id
    }

    private fun fakeFactory(d: TrackerDescriptor) = object :
        TrackerClientFactory {
        override val descriptor = d
        override fun create(config: lava.sdk.api.PluginConfig) =
            object : TrackerClient {
                override val descriptor = d
                override suspend fun healthCheck() = true
                override fun <T : TrackerFeature>
                    getFeature(featureClass: KClass<T>): T? = null
                override fun close() {}
            }
    }

    @Test
    fun `trackersWithCapability filters by declared
        capabilities`() {
        val reg = DefaultTrackerRegistry()
        reg.register(fakeFactory(descriptor("a",
            setOf(TrackerCapability.SEARCH,
            TrackerCapability.BROWSE))))
        reg.register(fakeFactory(descriptor("b",
            setOf(TrackerCapability.BROWSE))))
        val withSearch =
            reg.trackersWithCapability(TrackerCapability.SEARCH)
        assertEquals(1, withSearch.size)
        assertEquals("a", withSearch[0].trackerId)
        val withBrowse =
            reg.trackersWithCapability(TrackerCapability.BROWSE)
        assertEquals(2, withBrowse.size)
    }

    @Test
    fun `register makes the factory retrievable by id`() {
        val reg = DefaultTrackerRegistry()
        reg.register(fakeFactory(descriptor("a", emptySet())))
        val client = reg.get("a", MapPluginConfig())
    }
}

```

```

        assertEquals("a", client.descriptor.trackerId)
        assertTrue(reg.isRegistered("a"))
    }
}

```



Step 5: Run, commit

Run: `./gradlew :core:tracker:registry:test` Expected: PASS.

```

git add core/tracker/registry/ settings.gradle.kts
git commit -m "sp3a-1.36: :core:tracker:registry - Lava-domain
            wrapper around lava.sdk:registry

```

Adds TrackerRegistry interface with trackersWithCapability() – the SDK generic registry doesn't know about TrackerCapability, this Lava wrapper adds the capability-filter helper used by cross-tracker fallback.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 1.37: :core:tracker:mirror (Lava-domain wrapper around lava.sdk:mirror)

Files:

- Create: core/tracker/mirror/build.gradle.kts
- Create: core/tracker/mirror/src/main/kotlin/lava/tracker/mirror/TrackerMirrorManager.kt
- Create: core/tracker/mirror/src/main/kotlin/lava/tracker/mirror/MirrorConfigStore.kt



Step 1: Add to settings

Add: `include(":core:tracker:mirror")` in settings.gradle.kts.



Step 2: Build file

```

plugins {
    id("lava.kotlin.library")
    id("lava.kotlin.serialization")
}
dependencies {
    api(project(":core:tracker:api"))
    api("lava.sdk:api")
    api("lava.sdk:mirror")
    implementation(libs.kotlinx.serialization.json)
    testImplementation(libs.junit)
}

```

```
testImplementation("lava.sdk:testing")
}
```



Step 3: Implement

```
// TrackerMirrorManager.kt
package lava.tracker.mirror

import lava.sdk.mirror.MirrorManager

/** Lava-typed alias for the generic SDK MirrorManager – keeps
    consumer imports tidy. */
typealias TrackerMirrorManager = MirrorManager

// MirrorConfigStore.kt
package lava.tracker.mirror

import kotlinx.serialization.Serializable
import kotlinx.serialization.json.Json
import lava.sdk.api.MirrorUrl

@Serializable
data class TrackerMirrorConfig(val expectedHealthMarker: String,
                              val mirrors: List<MirrorUrl>)

@Serializable
data class MirrorsConfig(val version: Int, val trackers:
                        Map<String, TrackerMirrorConfig>)

class MirrorConfigStore(private val bundledJson: String) {
    private val json = Json { ignoreUnknownKeys = true }
    fun load(): MirrorsConfig =
        json.decodeFromString(MirrorsConfig.serializer(),
                              bundledJson)
}
```



Step 4: Test

```
package lava.tracker.mirror

import org.junit.Assert.assertEquals
import org.junit.Test

class MirrorConfigStoreTest {
    @Test
    fun `parses bundled JSON shape from spec`() {
        val json = """
        {
            "version": 1,
            "trackers": {
                "rutor": {
                    "expectedHealthMarker": "RuTor",

```

```

        "mirrors": [
            {"url": "https://rutor.info", "isPrimary": true,
"priority": 0, "protocol": "HTTPS"},
            {"url": "https://rutor.is", "priority": 1,
"protocol": "HTTPS"}
        ]
    }
}
}
}
"""
.trimIndent()
val config = MirrorConfigStore(json).load()
assertEquals(1, config.version)
val rutor = config.trackers.getValue("rutor")
assertEquals("RuTor", rutor.expectedHealthMarker)
assertEquals(2, rutor.mirrors.size)
assertEquals(true, rutor.mirrors[0].isPrimary)
}
}
}

```



Step 5: Run, commit

```

git add core/tracker/mirror/ settings.gradle.kts
git commit -m "sp3a-1.37: :core:tracker:mirror - typealias +
MirrorConfigStore

```

Lava-domain wrapper exposes TrackerMirrorManager typealias to keep consumer imports under lava.tracker.* and adds MirrorConfigStore for bundled JSON parsing per spec.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section L — :core:tracker:testing (Lava-side test helpers)

Task 1.38: :core:tracker:testing — FakeTrackerClient, builders, fixture loader

Files:

- Create: core/tracker/testing/build.gradle.kts
- Create: core/tracker/testing/src/main/kotlin/lava/tracker/testing/FakeTrackerClient.kt
- Create: core/tracker/testing/src/main/kotlin/lava/tracker/testing/TorrentItemBuilder.kt
- Create: core/tracker/testing/src/main/kotlin/lava/tracker/testing/SearchRequestBuilder.kt
- Create: core/tracker/testing/src/main/kotlin/lava/tracker/testing/LavaFixtureLoader.kt

- Create: core/tracker/testing/src/test/kotlin/lava/tracker/testing/FakeTrackerClientTest.kt



Step 1: Add to settings

Add: include(":core:tracker:testing") in settings.gradle.kts.



Step 2: Build file

```
plugins { id("lava.kotlin.library") }
dependencies {
    api(project(":core:tracker:api"))
    api("lava.sdk:testing")
    api(libs.junit)
    testImplementation(libs.junit)
}
```



Step 3: FakeTrackerClient — configurable client implementing all feature interfaces with hooks

package lava.tracker.testing

```
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.TrackerFeature
import lava.tracker.api.feature.*
import lava.tracker.api.model.*
import kotlin.reflect.KClass
```

```
class FakeTrackerClient(override val descriptor:
    TrackerDescriptor) : TrackerClient {

    var healthy: Boolean = true
    var searchResultProvider: ((SearchRequest, Int) ->
        SearchResult) =
        { _, _ -> SearchResult(emptyList(), 0, 0) }
    var browseResultProvider: ((String?, Int) -> BrowseResult) =
        { _, _ -> BrowseResult(emptyList(), 0, 0) }
    var topicProvider: ((String) -> TopicDetail)? = null
    var loginProvider: ((LoginRequest) -> LoginResult) = {
        LoginResult(AuthState.Unauthenticated) }
    var downloadProvider: ((String) -> ByteArray)? = null

    override suspend fun healthCheck() = healthy

    @Suppress("UNCHECKED_CAST")
    override fun <T : TrackerFeature> getFeature(featureClass:
        KClass<T>): T? {
        // Constitutional clause 6.E: return non-null only for
        // declared capabilities.
```

```

    val declared = descriptor.capabilities
    return when (featureClass) {
        SearchableTracker::class -> if
            (TrackerCapability.SEARCH in declared) (search as T) else
            null
        BrowsableTracker::class -> if
            (TrackerCapability.BROWSE in declared) (browse as T) else
            null
        TopicTracker::class -> if (TrackerCapability.TOPIC in
            declared) (topic as T) else null
        CommentsTracker::class -> if
            (TrackerCapability.COMMENTS in declared) (comments as T)
            else null
        FavoritesTracker::class -> if
            (TrackerCapability.FAVORITES in declared) (favorites as
            T) else null
        AuthenticatableTracker::class -> if
            (TrackerCapability.AUTH_REQUIRED in declared) (auth as T)
            else null
        DownloadableTracker::class -> if
            (TrackerCapability.TORRENT_DOWNLOAD in declared)
            (download as T) else null
        else -> null
    }
}

override fun close() {}

private val search = object : SearchableTracker {
    override suspend fun search(request: SearchRequest, page:
        Int) = searchResultProvider(request, page)
}

private val browse = object : BrowsableTracker {
    override suspend fun browse(category: String?, page: Int)
        = browseResultProvider(category, page)
    override suspend fun getForumTree() = null
}

private val topic = object : TopicTracker {
    override suspend fun getTopic(id: String) =
        topicProvider?.invoke(id)
        ?: error("FakeTrackerClient.topicProvider not
            configured for id=$id")
    override suspend fun getTopicPage(id: String, page: Int) =
        TopicPage(getTopic(id), totalPages = 1, currentPage =
        page)
}

private val comments = object : CommentsTracker {
    override suspend fun getComments(topicId: String, page:
        Int) = CommentsPage(emptyList(), 0, 0)
    override suspend fun addComment(topicId: String, message:
        String) = true
}

private val favorites = object : FavoritesTracker {
    private val store = mutableSetOf<String>()
    override suspend fun list() = store.map {

```



```

        TorrentItem(trackerId = descriptor.trackerId,
            torrentId = it, title = "fav-$it")
    }
    override suspend fun add(id: String) = store.add(id)
    override suspend fun remove(id: String) = store.remove(id)
}

private val auth = object : AuthenticatableTracker {
    private var state: AuthState = AuthState.Unauthenticated
    override suspend fun login(req: LoginRequest) =
        loginProvider(req).also { state = it.state }
    override suspend fun logout() { state =
        AuthState.Unauthenticated }
    override suspend fun checkAuth() = state
}

private val download = object : DownloadableTracker {
    override suspend fun downloadTorrentFile(id: String) =
        downloadProvider?.invoke(id) ?: ByteArray(0)
    override fun getMagnetLink(id: String) = "magnet:?
        xt=urn:btih:fakeinfohash$id"
}
}

```



Step 4: Builders

```

// TorrentItemBuilder.kt
package lava.tracker.testing

import lava.tracker.api.model.TorrentItem

class TorrentItemBuilder {
    var trackerId: String = "test"
    var torrentId: String = "1"
    var title: String = "Sample Torrent"
    var sizeBytes: Long? = null
    var seeders: Int? = 0
    var leechers: Int? = 0
    var infoHash: String? = null
    var magnetUri: String? = null
    var downloadUrl: String? = null
    var detailUrl: String? = null
    var category: String? = null
    var publishDate: kotlin.datetime.Instant? = null
    var metadata: Map<String, String> = emptyMap()
    fun build() = TorrentItem(
        trackerId, torrentId, title, sizeBytes, seeders, leechers,
        infoHash, magnetUri, downloadUrl, detailUrl, category,
        publishDate, metadata
    )
}

fun torrent(block: TorrentItemBuilder.() -> Unit) =
    TorrentItemBuilder().apply(block).build()

```

```
// SearchRequestBuilder.kt
package lava.tracker.testing

import lava.tracker.api.model.*

class SearchRequestBuilder {
    var query: String = ""
    var categories: List<String> = emptyList()
    var sort: SortField = SortField.DATE
    var sortOrder: SortOrder = SortOrder.DECENDING
    var author: String? = null
    var period: TimePeriod? = null
    fun build() = SearchRequest(query, categories, sort,
        sortOrder, author, period)
}

fun searchRequest(block: SearchRequestBuilder.() -> Unit) =
    SearchRequestBuilder().apply(block).build()

// LavaFixtureLoader.kt
package lava.tracker.testing

import lava.sdk.testing.HtmlFixtureLoader

/** Wraps the SDK's loader for Lava-side test resource paths under
    fixtures/<tracker>/<scope>/<file>. */
class LavaFixtureLoader(private val tracker: String) {
    private val sdkLoader = HtmlFixtureLoader(resourceRoot =
        "fixtures/$tracker")
    fun load(scope: String, fileName: String): String =
        sdkLoader.load("$scope/$fileName")
}
```



Step 5: Test the fake's capability-honesty enforcement

```
package lava.tracker.testing

import lava.sdk.api.MirrorUrl
import lava.tracker.api.AuthType
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.feature.SearchableTracker
import org.junit.Assert.*
import org.junit.Test

class FakeTrackerClientTest {

    private fun descriptor(caps: Set<TrackerCapability>) =
        object : TrackerDescriptor {
            override val trackerId = "fake"
            override val displayName = "Fake"
            override val baseUrls = listOf(MirrorUrl("https://
                fake.example", isPrimary = true))
            override val capabilities = caps
        }
}
```

```

        override val authType = AuthType.NONE
        override val encoding = "UTF-8"
        override val expectedHealthMarker = "fake"
    }

    @Test
    fun `getFeature returns non-null when capability is
    declared`() {
        val client =
            FakeTrackerClient(descriptor(setOf(TrackerCapability.SEARCH)))
        assertNotNull(client.getFeature(SearchableTracker::class))
    }

    @Test
    fun `getFeature returns null when capability is NOT
    declared`() {
        val client = FakeTrackerClient(descriptor(emptySet()))
        assertNull(client.getFeature(SearchableTracker::class))
    }
}

```



Step 6: Run, commit

```

git add core/tracker/testing/ settings.gradle.kts
git commit -m "sp3a-1.38: :core:tracker:testing -
    FakeTrackerClient + builders + fixtures

```

FakeTrackerClient enforces capability honesty (clause 6.E):
 getFeature
 returns non-null only for declared capabilities. DSL builders for
 TorrentItem and SearchRequest. LavaFixtureLoader wraps SDK loader.

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Phase 1 acceptance check. Run from repo root:

```
./gradlew :core:tracker:api:test :core:tracker:registry:test :core:tracker:mirror:
```

Expected: BUILD SUCCESSFUL — all five module test suites green.

```
ls submodules/tracker_sdk/api/src/main/kotlin/lava/sdk/api/
```

Expected: MirrorUrl.kt, Protocol.kt, HealthState.kt, MirrorState.kt,
 FallBackPolicy.kt, MirrorUnavailableException.kt, PluginConfig.kt,
 HasId.kt all present.

```
git ls-remote --tags github | grep v0.1.0
```

Expected: v0.1.0 tag visible on the SDK upstream.

Phase 1 done. Phase 2 unblocked.

Phase 2: RuTracker Decoupling (Kotlin only)

Duration: 2.5 weeks. **Tasks:** 36. **Goal:** Refactor the existing `:core:network:rutracker` module behind the new `TrackerClient` contract with **zero behavioral change**, verified by parity tests against the pre-SP-3a baseline. Rewire `SwitchingNetworkApi` to delegate to `LavaTrackerSdk` underneath. **Out of scope:** any modification to `lava-api-go/internal/rutracker/` (deferred to SP-3a-bridge after SP-2 ships).

Acceptance gate: parity test produces byte-for-byte identical output for all 14 `NetworkApi` methods against the pre-SP-3a tag; all existing feature `ViewModel` tests pass without modification; `:core:network:rutracker` removed from `settings.gradle.kts`; `:core:tracker:rutracker` exists with full feature impl set.

Section A — Move and rename the existing rutracker module

Task 2.1: `git mv` the rutracker module to its new location

Files:

- Move: `core/network/rutracker/` → `core/tracker/rutracker/`



Step 1: Verify clean working tree

Run: `git status` Expected: clean. If not, commit or stash first.



Step 2: Perform the move (preserves history)

`git mv core/network/rutracker core/tracker/rutracker`



Step 3: Verify

Run: `ls core/tracker/rutracker/src/main/kotlin/lava/network/`
Expected: `api domain impl model` (the existing package structure, untouched).

Run: `git log --oneline --follow core/tracker/rutracker/build.gradle.kts | head -3` Expected: shows commits from the original `core/network/rutracker/build.gradle.kts` history.



Step 4: Commit the rename

`git commit -m "sp3a-2.1: git mv core/network/rutracker → core/tracker/rutracker"`

History preserved via `git rename` detection. No package or import changes
yet — those land in subsequent tasks. Build is broken between this

commit and Task 2.4 (settings.gradle.kts update).

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Task 2.2: Update settings.gradle.kts to reflect the rename

Files:

- Modify: settings.gradle.kts



Step 1: Remove old include, add new

Edit settings.gradle.kts:

```
// REMOVE this line:  
include(":core:network:rutracker")  
  
// (Already added in Task 1.32; verify present:)  
include(":core:tracker:rutracker")
```



Step 2: Verify build resolves

Run: ./gradlew projects | grep rutracker Expected:
:core:tracker:rutracker only — no :core:network:rutracker.



Step 3: Commit

```
git add settings.gradle.kts  
git commit -m "sp3a-2.2: update settings.gradle.kts for rutracker  
rename"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>"

Task 2.3: Move package paths from lava.network.rutracker.* to lava.tracker.rutracker.*

Files:

- Move: every .kt file under core/tracker/rutracker/src/main/kotlin/lava/network/ to core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/



Step 1: Inspect existing structure

Run: `find core/tracker/rutacker/src/main/kotlin -name "*.kt" | head -20` Note the existing top-level packages: `lava.network.api`, `lava.network.domain`, `lava.network.impl`, `lava.network.model`.



Step 2: Move files

```
cd core/tracker/rutacker/src/main/kotlin
mkdir -p lava/tracker/rutacker
git mv lava/network lava/tracker/rutacker/
cd ../../../../..
```

(After: files live at `core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/api/...`, etc.)



Step 3: Update package declarations in every .kt file

Run a sed across the moved files:

```
find core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker -name "*.kt" \
    -exec sed -i 's|^package lava\.network\.|package
        lava.tracker.rutacker.|g' {} \;
```



Step 4: Update all imports inside the moved files

```
find core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker -name "*.kt" \
    -exec sed -i 's|^import lava\.network\.|(api|domain|impl|
        model\)|import lava.tracker.rutacker.\1|g' {} \;
```

(The negative match — `lava.network.dto.*` and other non-rutacker packages — must remain `lava.network.*`. Imports of types that belong to `:core:network:api` like `lava.network.dto.search.SearchPageDto` are NOT renamed.)



Step 5: Verify compile

Run: `./gradlew :core:tracker:rutacker:compileKotlin` Expected: BUILD SUCCESSFUL. If errors, they'll point to imports that need finer adjustment.



Step 6: Commit

```
git add core/tracker/rutacker/
git commit -m "sp3a-2.3: move package paths to
    lava.tracker.rutacker.*"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 2.4: Update consumer imports across the project

The internal-rutracker types (the RuTrackerInnerApi and any other types under the old `lava.network.rutracker.*` package) are now at `lava.tracker.rutracker.*`. Find and update consumers.

Files:

- Modify: any file in `core/`, `feature/`, `app/` that imports from `lava.network.rutracker.*`



Step 1: Find consumers

```
grep -rln 'import lava\.network\.rutracker\.' core/ feature/ app/
| sort -u
```



Step 2: Bulk rename imports

```
grep -rln 'import lava\.network\.rutracker\.' core/ feature/ app/
| xargs -I {} sed -i 's|^import lava\.network\.rutracker\.|
import lava.tracker.rutracker.|g' {}
```



Step 3: Verify build

Run: `./gradlew :app:assembleDebug` Expected: BUILD SUCCESSFUL.

If failures, hand-fix the remaining import lines reported by the compiler.



Step 4: Commit

```
git add -u core/ feature/ app/
git commit -m "sp3a-2.4: rename consumer imports
lava.network.rutracker.* → lava.tracker.rutracker.*"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 2.5: Update `core/tracker/rutracker/build.gradle.kts` to use the new convention plugin

Files:

- Modify: `core/tracker/rutracker/build.gradle.kts`



Step 1: Read current build file

Run: `cat core/tracker/rutacker/build.gradle.kts` Expected: `lava.kotlin.library` plugin, `:core:network:api` dependency, Ktor and Jsoup.



Step 2: Replace with the tracker module convention

```
plugins {
    id("lava.kotlin.tracker.module") // pre-
    wires :core:tracker:api + Tracker-SDK + Jsoup/OkHttp
}

dependencies {
    // Keep transitional dependency on :core:network:api so the
    // existing
    // RuTrackerInnerApi (legacy HTTP wire) still compiles.
    // Removed in
    // Spec 2 when :proxy retires.
    api(project(":core:network:api"))

    implementation(libs.ktor.client.core)
    // Jsoup is brought in by the convention plugin; do not re-
    // declare.
}
```



Step 3: Verify compile

Run: `./gradlew :core:tracker:rutacker:compileKotlin` Expected: BUILD SUCCESSFUL.



Step 4: Run any existing tests

Run: `./gradlew :core:tracker:rutacker:test` Expected: BUILD SUCCESSFUL (existing tests should still pass; the rename is structural only).



Step 5: Commit

```
git add core/tracker/rutacker/build.gradle.kts
git commit -m "sp3a-2.5: rutacker module switches to
    lava.kotlin.tracker.module plugin"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section A complete. Module renamed and re-wired. Phase 3 (RuTor) may now begin in parallel; Tasks 2.6+ continue building the new RuTracker contract surface.

Section B — RuTrackerDescriptor

Task 2.6: Implement RuTrackerDescriptor

Files:

- Create: core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/RuTrackerDescriptor.kt
- Create: core/tracker/rutacker/src/test/kotlin/lava/tracker/rutacker/RuTrackerDescriptorTest.kt



Step 1: Create descriptor

```
package lava.tracker.rutacker
```

```
import lava.sdk.api.MirrorUrl
import lava.sdk.api.Protocol
import lava.tracker.api.AuthType
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerDescriptor
```

```
object RuTrackerDescriptor : TrackerDescriptor {
    override val trackerId = "rutacker"
    override val displayName = "RuTracker.org"
    override val baseUrls = listOf(
        MirrorUrl("https://rutacker.org", isPrimary = true,
            priority = 0, protocol = Protocol.HTTPS),
        MirrorUrl("https://rutacker.net", priority = 1, protocol
            = Protocol.HTTPS),
        MirrorUrl("https://rutacker.cr", priority = 2, protocol
            = Protocol.HTTPS),
    )
    override val capabilities = setOf(
        TrackerCapability.SEARCH,
        TrackerCapability.BROWSE,
        TrackerCapability.FORUM,
        TrackerCapability.TOPIC,
        TrackerCapability.COMMENTS,
        TrackerCapability.FAVORITES,
        TrackerCapability.TORRENT_DOWNLOAD,
        TrackerCapability.MAGNET_LINK,
        TrackerCapability.AUTH_REQUIRED,
        TrackerCapability.CAPTCHA_LOGIN,
        TrackerCapability.UPLOAD,
        TrackerCapability.USER_PROFILE,
    )
    override val authType = AuthType.CAPTCHA_LOGIN
    override val encoding = "Windows-1251"
    override val expectedHealthMarker = "rutacker"
}
```



Step 2: Test

```
package lava.tracker.rutracker

import lava.tracker.api.TrackerCapability
import org.junit.Assert.*
import org.junit.Test

class RuTrackerDescriptorTest {
    @Test fun `id matches trackerId`() {
        assertEquals("rutracker", RuTrackerDescriptor.id)
    }
    @Test fun `declares all 12 RuTracker capabilities`() {
        assertEquals(12, RuTrackerDescriptor.capabilities.size)
    }
    @Test fun `primary mirror is rutracker_org`() {
        val primary = RuTrackerDescriptor.baseUrls.first {
            it.isPrimary
        }
        assertEquals("https://rutracker.org", primary.url)
    }
    @Test fun `encoding is Windows-1251`() {
        assertEquals("Windows-1251", RuTrackerDescriptor.encoding)
    }
    @Test fun `health marker matches root page substring`() {
        assertTrue(RuTrackerDescriptor.expectedHealthMarker.isNotBlank())
    }
}
```



Step 3: Run, commit

```
./gradlew :core:tracker:rutracker:test --tests
    "*RuTrackerDescriptorTest*"
git add core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/RuTrackerDescriptor.kt \
    core/tracker/rutracker/src/test/kotlin/lava/tracker/
    rutracker/RuTrackerDescriptorTest.kt
git commit -m "sp3a-2.6: RuTrackerDescriptor + tests"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section C — RuTrackerClient + 7 feature implementations

The strategy: RuTrackerClient wraps the existing RuTrackerInnerApi (the current HTTP wire-level client) and exposes the new feature interfaces by delegating each feature method to the existing UseCase classes — preserving zero behavioral change.

Task 2.7: RuTrackerClient skeleton

Files:

- Create: core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerClient.kt



Step 1: Locate the existing RuTrackerInnerApi

Run: `grep -rn "interface RuTrackerInnerApi" core/` (If named differently, search for the rutracker HTTP client under core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/api/.)

Note its package and constructor signature.



Step 2: Implement the client

```
// core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/
// RuTrackerClient.kt
package lava.tracker.rutracker

import javax.inject.Inject
import kotlin.reflect.KClass
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.TrackerFeature
import lava.tracker.api.feature.*
import lava.tracker.rutracker.feature.* // RuTrackerSearch,
// RuTrackerBrowse, etc. (Tasks 2.8–2.13)

class RuTrackerClient @Inject constructor(
    private val search: RuTrackerSearch,
    private val browse: RuTrackerBrowse,
    private val topic: RuTrackerTopic,
    private val comments: RuTrackerComments,
    private val favorites: RuTrackerFavorites,
    private val auth: RuTrackerAuth,
    private val download: RuTrackerDownload,
) : TrackerClient {

    override val descriptor: TrackerDescriptor =
        RuTrackerDescriptor

    override suspend fun healthCheck(): Boolean =
        auth.checkAuthAlive()

    @Suppress("UNCHECKED_CAST")
    override fun <T : TrackerFeature> getFeature(featureClass:
        KClass<T>): T? {
        // Constitutional clause 6.E: declared capability ⇒ non-
        // null feature.
    }
}
```

```

    return when (featureClass) {
        SearchableTracker::class -> if
            (TrackerCapability.SEARCH in descriptor.capabilities)
            search as T else null
        BrowsableTracker::class -> if
            (TrackerCapability.BROWSE in descriptor.capabilities)
            browse as T else null
        TopicTracker::class -> if (TrackerCapability.TOPIC in
            descriptor.capabilities) topic as T else null
        CommentsTracker::class -> if
            (TrackerCapability.COMMENTS in descriptor.capabilities)
            comments as T else null
        FavoritesTracker::class -> if
            (TrackerCapability.FAVORITES in descriptor.capabilities)
            favorites as T else null
        AuthenticatableTracker::class -> if
            (TrackerCapability.AUTH_REQUIRED in
            descriptor.capabilities) auth as T else null
        DownloadableTracker::class -> if
            (TrackerCapability.TORRENT_DOWNLOAD in
            descriptor.capabilities) download as T else null
        else -> null
    }
}

override fun close() {
    // Close any held HTTP clients; for now the inner Ktor
    // client is process-scoped via DI.
}
}

```

(Compile will fail because the seven `RuTracker*` feature classes don't exist yet — they're created in Tasks 2.8–2.13. That's expected.)



Step 3: Commit (build is intentionally broken until Task 2.13)

```

git add core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/RuTrackerClient.kt
git commit -m "sp3a-2.7: RuTrackerClient skeleton – references
    feature impls (Tasks 2.8–2.13)"

```

Build will not succeed until the 7 feature impl classes exist.

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.8: RuTrackerSearch feature implementation

Files:

- Create: `core/tracker/rutacker/src/main/kotlin/lava/tracker/`
`rutacker/feature/RuTrackerSearch.kt`

- Create: core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/mapper/SearchPageMapper.kt (stub — fully populated in Task 2.15)



Step 1: Implement the feature

```
// core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/
// feature/RuTrackerSearch.kt
package lava.tracker.rutacker.feature

import javax.inject.Inject
import lava.tracker.api.feature.SearchableTracker
import lava.tracker.api.model.SearchRequest
import lava.tracker.api.model.SearchResult
import lava.tracker.rutacker.domain.GetSearchPageUseCase
import lava.tracker.rutacker.mapper.SearchPageMapper
import lava.tracker.rutacker.mapper.toLegacySearchParams

class RuTrackerSearch @Inject constructor(
    private val getSearchPage: GetSearchPageUseCase,
    private val mapper: SearchPageMapper,
) : SearchableTracker {

    override suspend fun search(request: SearchRequest, page:
        Int): SearchResult {
        val (sortType, sortOrder, period, categories) =
            request.toLegacySearchParams()
        val dto = getSearchPage(
            searchQuery = request.query,
            categories = categories,
            author = request.author,
            authorId = null,
            sortType = sortType,
            sortOrder = sortOrder,
            period = period,
            page = page,
        )
        return mapper.toResult(dto, currentPage = page)
    }
}
```



Step 2: Stub the mapper for now (filled in Task 2.15)

```
// core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/
// mapper/SearchPageMapper.kt
package lava.tracker.rutacker.mapper

import javax.inject.Inject
import lava.network.dto.search.SearchPageDto
import lava.network.dto.search.SearchPeriodDto
import lava.network.dto.search.SearchSortOrderDto
```

```

import lava.network.dto.search.SearchSortTypeDto
import lava.tracker.api.model.SearchRequest
import lava.tracker.api.model.SearchResult

class SearchPageMapper @Inject constructor() {
    fun toResult(dto: SearchPageDto, currentPage: Int):
        SearchResult {
        TODO("populated in Task 2.15")
    }
}

data class LegacySearchParams(
    val sortType: SearchSortTypeDto?,
    val sortOrder: SearchSortOrderDto?,
    val period: SearchPeriodDto?,
    val categories: String?,
)

fun SearchRequest.toLegacySearchParams(): LegacySearchParams {
    TODO("populated in Task 2.15")
}

```



Step 3: Commit (still broken until 2.15)

```

git add core/tracker/rutrackersrc/main/kotlin/lava/tracker/
    rutrackersfeature/RuTrackerSearch.kt \
    core/tracker/rutrackerssrc/main/kotlin/lava/tracker/
    rutrackersmapper/SearchPageMapper.kt
git commit -m "sp3a-2.8: RuTrackerSearch + SearchPageMapper stub

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.9: RuTrackerBrowse feature implementation

Files:

- Create: core/tracker/rutrackersrc/main/kotlin/lava/tracker/rutrackersfeature/RuTrackerBrowse.kt
- Create: core/tracker/rutrackerssrc/main/kotlin/lava/tracker/rutrackersmapper/ForumDtoMapper.kt
- Create: core/tracker/rutrackerssrc/main/kotlin/lava/tracker/rutrackersmapper/CategoryPageMapper.kt



Step 1: Implement

```

package lava.tracker.rutrackersfeature

```

```

import javax.inject.Inject
import lava.tracker.api.feature.BrowsableTracker

```

```

import lava.tracker.api.model.BrowseResult
import lava.tracker.api.model.ForumTree
import lava.tracker.rutracker.domain.GetCategoryPageUseCase
import lava.tracker.rutracker.domain.GetForumUseCase
import lava.tracker.rutracker.mapper.CategoryPageMapper
import lava.tracker.rutracker.mapper.ForumDtoMapper

class RuTrackerBrowse @Inject constructor(
    private val getCategoryPage: GetCategoryPageUseCase,
    private val getForum: GetForumUseCase,
    private val categoryMapper: CategoryPageMapper,
    private val forumMapper: ForumDtoMapper,
) : BrowsableTracker {

    override suspend fun browse(category: String?, page: Int):
        BrowseResult {
        require(category != null) { "RuTracker browse requires a
            category id" }
        val dto = getCategoryPage(id = category, page = page)
        return categoryMapper.toBrowseResult(dto, currentPage =
            page)
    }

    override suspend fun getForumTree(): ForumTree? {
        val dto = getForum()
        return forumMapper.toForumTree(dto)
    }
}

```



Step 2: Stubs

```

// ForumDtoMapper.kt
package lava.tracker.rutracker.mapper

import javax.inject.Inject
import lava.network.dto.forum.ForumDto
import lava.tracker.api.model.ForumTree

class ForumDtoMapper @Inject constructor() {
    fun toForumTree(dto: ForumDto): ForumTree {
        TODO("populated in Task 2.16")
    }
}

// CategoryPageMapper.kt
package lava.tracker.rutracker.mapper

import javax.inject.Inject
import lava.network.dto.forum.CategoryPageDto
import lava.tracker.api.model.BrowseResult

class CategoryPageMapper @Inject constructor() {

```

```

    fun toBrowseResult(dto: CategoryPageDto, currentPage: Int):
        BrowseResult { TODO("populated in Task 2.17") }
}

```



Step 3: Commit

```

git add core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/feature/RuTrackerBrowse.kt \
    core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/mapper/ForumDtoMapper.kt \
    core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/mapper/CategoryPageMapper.kt
git commit -m "sp3a-2.9: RuTrackerBrowse + ForumDtoMapper/
    CategoryPageMapper stubs

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.10: RuTrackerTopic feature implementation

Files:

- Create: core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/feature/RuTrackerTopic.kt
- Create: core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/mapper/TopicMapper.kt

```

// RuTrackerTopic.kt
package lava.tracker.rutacker.feature

import javax.inject.Inject
import lava.tracker.api.feature.TopicTracker
import lava.tracker.api.model.TopicDetail
import lava.tracker.api.model.TopicPage
import lava.tracker.rutacker.domain.GetTopicUseCase
import lava.tracker.rutacker.domain.GetTopicPageUseCase
import lava.tracker.rutacker.mapper.TopicMapper

class RuTrackerTopic @Inject constructor(
    private val getTopic: GetTopicUseCase,
    private val getTopicPage: GetTopicPageUseCase,
    private val mapper: TopicMapper,
) : TopicTracker {
    override suspend fun getTopic(id: String): TopicDetail =
        mapper.toTopicDetail(getTopic(id = id, page = 0))
    override suspend fun getTopicPage(id: String, page: Int):
        TopicPage =
        mapper.toTopicPage(getTopicPage(id = id, page = page),
            currentPage = page)
}

```



```
// TopicMapper.kt
package lava.tracker.rutracker.mapper
import javax.inject.Inject
import lava.network.dto.topic.ForumTopicDto
import lava.network.dto.topic.TopicPageDto
import lava.tracker.api.model.TopicDetail
import lava.tracker.api.model.TopicPage

class TopicMapper @Inject constructor() {
    fun toTopicDetail(dto: ForumTopicDto): TopicDetail {
        TODO("populated in Task 2.18") }
    fun toTopicPage(dto: TopicPageDto, currentPage: Int):
        TopicPage { TODO("populated in Task 2.18") }
}
```



Commit

```
git add core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/feature/RuTrackerTopic.kt \
    core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/mapper/TopicMapper.kt
git commit -m "sp3a-2.10: RuTrackerTopic + TopicMapper stub"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 2.11: RuTrackerComments feature implementation

```
// core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/
    feature/RuTrackerComments.kt
package lava.tracker.rutracker.feature

import javax.inject.Inject
import lava.tracker.api.feature.CommentsTracker
import lava.tracker.api.model.CommentsPage
import lava.tracker.rutracker.domain.AddCommentUseCase
import lava.tracker.rutracker.domain.GetCommentsPageUseCase
import lava.tracker.rutracker.mapper.CommentsMapper

class RuTrackerComments @Inject constructor(
    private val getCommentsPage: GetCommentsPageUseCase,
    private val addCommentUseCase: AddCommentUseCase,
    private val mapper: CommentsMapper,
) : CommentsTracker {
    override suspend fun getComments(topicId: String, page: Int):
        CommentsPage =
        mapper.toCommentsPage(getCommentsPage(id = topicId, page
        = page), currentPage = page)
    override suspend fun addComment(topicId: String, message:
        String): Boolean =
```

```

        addCommentUseCase(topicId = topicId, message = message)
    }

// core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/
// mapper/CommentsMapper.kt
package lava.tracker.rutacker.mapper

import javax.inject.Inject
import lava.network.dto.topic.CommentsPageDto
import lava.tracker.api.model.CommentsPage

class CommentsMapper @Inject constructor() {
    fun toCommentsPage(dto: CommentsPageDto, currentPage: Int):
        CommentsPage { TODO("populated in Task 2.19") }
}

```



Commit

```

git add core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/feature/RuTrackerComments.kt \
    core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/mapper/CommentsMapper.kt
git commit -m "sp3a-2.11: RuTrackerComments + CommentsMapper stub

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>"

Task 2.12: RuTrackerFavorites and RuTrackerAuth and RuTrackerDownload

Files:

- Create: 3 feature files + 3 mapper stubs

```

// RuTrackerFavorites.kt
package lava.tracker.rutacker.feature

import javax.inject.Inject
import lava.tracker.api.feature.FavoritesTracker
import lava.tracker.api.model.TorrentItem
import lava.tracker.rutacker.domain.AddFavoriteUseCase
import lava.tracker.rutacker.domain.GetFavoritesUseCase
import lava.tracker.rutacker.domain.RemoveFavoriteUseCase
import lava.tracker.rutacker.mapper.FavoritesMapper

class RuTrackerFavorites @Inject constructor(
    private val get: GetFavoritesUseCase,
    private val add: AddFavoriteUseCase,
    private val remove: RemoveFavoriteUseCase,
    private val mapper: FavoritesMapper,
) : FavoritesTracker {

```

```

        override suspend fun list(): List<TorrentItem> =
            mapper.toTorrentItems(get())
        override suspend fun add(id: String): Boolean = add(id)
        override suspend fun remove(id: String): Boolean = remove(id)
    }

```

// RuTrackerAuth.kt

```
package lava.tracker.rutracker.feature
```

```

import javax.inject.Inject
import lava.tracker.api.feature.AuthenticatableTracker
import lava.tracker.api.model.AuthState
import lava.tracker.api.model.LoginRequest
import lava.tracker.api.model.LoginResult
import lava.tracker.rutracker.domain.CheckAuthorisedUseCase
import lava.tracker.rutracker.domain.LoginUseCase
import lava.tracker.rutracker.domain.LogoutUseCase
import lava.tracker.rutracker.mapper.AuthMapper

```

```

class RuTrackerAuth @Inject constructor(
    private val loginUseCase: LoginUseCase,
    private val logoutUseCase: LogoutUseCase,
    private val checkUseCase: CheckAuthorisedUseCase,
    private val mapper: AuthMapper,
) : AuthenticatableTracker {
    override suspend fun login(req: LoginRequest): LoginResult =
        mapper.toLoginResult(loginUseCase(
            username = req.username, password = req.password,
            captchaSid = req.captcha?.sid, captchaCode =
                req.captcha?.code,
            captchaValue = req.captcha?.value
        ))
    override suspend fun logout() { logoutUseCase() }
    override suspend fun checkAuth(): AuthState = if
        (checkUseCase()) AuthState.Authenticated else
        AuthState.Unauthenticated
    /** Used by RuTrackerClient.healthCheck() - checks if the
        persisted auth still resolves. */
    suspend fun checkAuthAlive(): Boolean = checkUseCase()
}

```

// RuTrackerDownload.kt

```
package lava.tracker.rutracker.feature
```

```

import javax.inject.Inject
import lava.tracker.api.feature.DownloadableTracker
import lava.tracker.rutracker.domain.GetMagnetLinkUseCase
import lava.tracker.rutracker.domain.GetTorrentFileUseCase

```

```

class RuTrackerDownload @Inject constructor(
    private val downloadFile: GetTorrentFileUseCase,
    private val getMagnet: GetMagnetLinkUseCase,
) : DownloadableTracker {

```

```

    override suspend fun downloadTorrentFile(id: String):
        ByteArray = downloadFile(id)
    override fun getMagnetLink(id: String): String? =
        getMagnet(id) // sync if cached, else null
}

```

Mapper stubs:

```

// FavoritesMapper.kt
package lava.tracker.rutracker.mapper
import javax.inject.Inject
import lava.network.dto.user.FavoritesDto
import lava.tracker.api.model.TorrentItem
class FavoritesMapper @Inject constructor() {
    fun toTorrentItems(dto: FavoritesDto): List<TorrentItem> {
        TODO("populated in Task 2.20") }
}

// AuthMapper.kt
package lava.tracker.rutracker.mapper
import javax.inject.Inject
import lava.network.dto.auth.AuthResponseDto
import lava.tracker.api.model.LoginResult
class AuthMapper @Inject constructor() {
    fun toLoginResult(dto: AuthResponseDto): LoginResult {
        TODO("populated in Task 2.21") }
}

```

(Note: GetMagnetLinkUseCase and GetTorrentFileUseCase may need to be created or extracted from existing GetTorrentUseCase / download flow. If they don't exist, create them as thin wrappers around the existing inner-API methods and commit them in this task.)



Commit

```

git add core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/feature/ \
    core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/mapper/
git commit -m "sp3a-2.12: RuTracker Favorites/Auth/Download impls
    + mapper stubs

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.13: Resolve compile errors and verify the client compiles

By this point all 7 feature classes exist (with stubbed mappers); RuTrackerClient should compile.



Step 1: Build

Run: `./gradlew :core:tracker:rutracker:compileKotlin` Expected: BUILD SUCCESSFUL.

If compile fails because a UseCase referenced in feature classes doesn't exist with the expected signature, hand-fix by adapting to the actual signature. Common adjustments: LoginUseCase may take Username/Password value classes rather than raw strings — adapt by calling its actual API.



Step 2: Commit any compile-fix tweaks

```
git add -u core/tracker/rutracker/
git commit -m
    "sp3a-2.13: compile-fixes - adapt feature classes to
    actual UseCase signatures"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Section D — Forward DTO → model mappers (full implementations)

The mappers stubbed in Tasks 2.8–2.12 with `TODO()` are now filled in. Each mapper extracts common fields into the new model and stores RuTracker-specific extras under namespaced metadata keys.

Task 2.14: Common helper — LegacySearchParams mapper for SearchRequest

Files:

- Modify: `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/SearchPageMapper.kt`
- Create: `core/tracker/rutracker/src/test/kotlin/lava/tracker/rutracker/mapper/SearchRequestMapperTest.kt`



Step 1: Replace the `toLegacySearchParams` TODO

```
// In SearchPageMapper.kt, replace the TODO-bearing function:
fun SearchRequest.toLegacySearchParams(): LegacySearchParams {
    val sortType = when (sort) {
        SortField.DATE -> SearchSortTypeDto.DATE
        SortField.SEEDERS -> SearchSortTypeDto.SEEDS
        SortField.LEECHERS -> SearchSortTypeDto.LEECHES
        SortField.SIZE -> SearchSortTypeDto.SIZE
        SortField.RELEVANCE -> SearchSortTypeDto.RELEVANCE
        SortField.TITLE -> SearchSortTypeDto.TITLE
    }
    val sortOrderDto = when (sortOrder) {
        SortOrder.ASCENDING -> SearchSortOrderDto.ASCENDING
```

```

        SortOrder.DECENDING -> SearchSortOrderDto.DECENDING
    }
    val periodDto = period?.let {
        when (it) {
            TimePeriod.LAST_DAY -> SearchPeriodDto.LAST_DAY
            TimePeriod.LAST_WEEK -> SearchPeriodDto.LAST_WEEK
            TimePeriod.LAST_MONTH -> SearchPeriodDto.LAST_MONTH
            TimePeriod.LAST_YEAR -> SearchPeriodDto.LAST_YEAR
            TimePeriod.ALL_TIME -> SearchPeriodDto.ALL_TIME
        }
    }
    return LegacySearchParams(
        sortType = sortType,
        sortOrder = sortOrderDto,
        period = periodDto,
        categories = categories.takeIf {
            it.isNotEmpty() }?.joinToString(","),
    )
}

```

(Adapt enum value names to match the actual SearchSortTypeDto enum members in the existing file — verify with `grep -A 30 'enum class SearchSortTypeDto'`. If a sort field has no DTO equivalent, default to DATE.)



Step 2: Test

```
package lava.tracker.rutrackr.mapper
```

```

import lava.network.dto.search.SearchPeriodDto
import lava.network.dto.search.SearchSortOrderDto
import lava.network.dto.search.SearchSortTypeDto
import lava.tracker.api.model.SearchRequest
import lava.tracker.api.model.SortField
import lava.tracker.api.model.SortOrder
import lava.tracker.api.model.TimePeriod
import org.junit.Assert.assertEquals
import org.junit.Test

```

```

class SearchRequestMapperTest {
    @Test
    fun `default request maps to defaults`() {
        val r = SearchRequest(query = "ubuntu")
        val params = r.toLegacySearchParams()
        assertEquals(SearchSortTypeDto.DATE, params.sortType)
        assertEquals(SearchSortOrderDto.DECENDING,
            params.sortOrder)
        assertEquals(null, params.period)
        assertEquals(null, params.categories)
    }
    @Test
    fun `categories list joins with comma`() {

```

```

        val r = SearchRequest(query = "x", categories =
            listOf("1", "2", "3"))
        assertEquals("1,2,3", r.toLegacySearchParams().categories)
    }
    @Test
    fun `period maps`() {
        val r = SearchRequest(query = "x", period =
            TimePeriod.LAST_WEEK)
        assertEquals(SearchPeriodDto.LAST_WEEK,
            r.toLegacySearchParams().period)
    }
}

```



Step 3: Run, commit

```

git add core/tracker/rutacker/src/main/kotlin/lava/tracker/
    rutacker/mapper/SearchPageMapper.kt \
    core/tracker/rutacker/src/test/kotlin/lava/tracker/
    rutacker/mapper/SearchRequestMapperTest.kt
git commit -m "sp3a-2.14: SearchRequest → LegacySearchParams
    mapper + tests

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.15: SearchPageMapper.toResult full implementation

Files:

- Modify: core/tracker/rutacker/src/main/kotlin/lava/tracker/rutacker/mapper/SearchPageMapper.kt
- Create: core/tracker/rutacker/src/test/kotlin/lava/tracker/rutacker/mapper/SearchPageMapperTest.kt



Step 1: Read current SearchPageDto shape

Run: `cat core/network/api/src/main/kotlin/lava/network/dto/search/SearchPageDto.kt` Note the field set: items list with id/title/size/seeder/etc., totalPages, etc.



Step 2: Implement

```

// Replace the TODO-bearing toResult function:
fun toResult(dto: SearchPageDto, currentPage: Int): SearchResult =
    SearchResult(
        items = dto.items.map { item ->
            TorrentItem(
                trackerId = "rutacker",
                torrentId = item.id,
                title = item.title,

```

```

        sizeBytes = item.size,
        seeders = item.seeds,
        leechers = item.leeches,
        infoHash = null, // RuTracker search results
        // don't include hash; fetched on detail
        magnetUri = null,
        downloadUrl = null,
        detailUrl = null,
        category = item.categoryId,
        publishDate = item.date,
        metadata = buildMetadata(item),
    )
},
totalPages = dto.totalPages,
currentPage = currentPage,
)

private fun buildMetadata(item:
    lava.network.dto.search.SearchItemDto): Map<String,
    String> = buildMap {
    item.author?.let { put("rutracker.author", it) }
    item.authorId?.let { put("rutracker.authorId", it) }
    item.commentsCount?.let { put("rutracker.commentsCount",
        it.toString()) }
}

```

(Verify the exact SearchItemDto field names against the existing source. Adapt as needed.)



Step 3: Test

```

package lava.tracker.rutracker.mapper

import lava.network.dto.search.SearchPageDto
import lava.network.dto.search.SearchItemDto
import org.junit.Assert.*
import org.junit.Test

class SearchPageMapperTest {
    @Test
    fun `maps DTO items to TorrentItems with rutracker
        trackerId`() {
        val dto = SearchPageDto(
            items = listOf(
                SearchItemDto(id = "100", title = "Ubuntu", size
                    = 4500000000L,
                        seeds = 50, leeches = 5, categoryId
                    = "Linux",
                        author = "ubuntu-team", authorId =
                    "u100",
                        commentsCount = 10, date = null)
            ),
            totalPages = 5,

```



```

    )
    val result = SearchPageMapper().toResult(dto, currentPage
    = 0)
    assertEquals(1, result.items.size)
    val item = result.items.first()
    assertEquals("rutracker", item.trackerId)
    assertEquals("100", item.torrentId)
    assertEquals("Ubuntu", item.title)
    assertEquals(4500000000L, item.sizeBytes)
    assertEquals("ubuntu-team",
        item.metadata["rutracker.author"])
    assertEquals("10",
        item.metadata["rutracker.commentsCount"])
    assertEquals(5, result.totalPages)
    assertEquals(0, result.currentPage)
}
}

```

(Adapt the test data class construction to match actual DTO constructor signatures.)



Step 4: Run, commit

```

git add core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/mapper/SearchPageMapper.kt \
    core/tracker/rutracker/src/test/kotlin/lava/tracker/
    rutracker/mapper/SearchPageMapperTest.kt
git commit -m "sp3a-2.15: SearchPageMapper.toResult full impl +
    test

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.16: ForumDtoMapper full implementation

Files:

- Modify: core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/ForumDtoMapper.kt
- Create: core/tracker/rutracker/src/test/kotlin/lava/tracker/rutracker/mapper/ForumDtoMapperTest.kt



Step 1: Implement

```
package lava.tracker.rutracker.mapper
```

```

import javax.inject.Inject
import lava.network.dto.forum.ForumDto
import lava.network.dto.forum.SectionDto
import lava.network.dto.forum.CategoryDto
import lava.tracker.api.model.ForumCategory
import lava.tracker.api.model.ForumTree

```

```

class ForumDtoMapper @Inject constructor() {
    fun toForumTree(dto: ForumDto): ForumTree =
        ForumTree(
            rootCategories = dto.sections.map { section ->
                mapSection(section, parentId = null) }
        )

    private fun mapSection(s: SectionDto, parentId: String?):
        ForumCategory =
        ForumCategory(
            id = s.id,
            name = s.name,
            parentId = parentId,
            children = s.categories.map { c -> mapCategory(c,
                parentId = s.id) },
        )

    private fun mapCategory(c: CategoryDto, parentId: String?):
        ForumCategory =
        ForumCategory(id = c.id, name = c.name, parentId =
            parentId, children = emptyList())
}

```

(Verify the SectionDto/CategoryDto field names against the existing source.)



Step 2: Test

```
package lava.tracker.rutracker.mapper
```

```

import lava.network.dto.forum.ForumDto
import lava.network.dto.forum.SectionDto
import lava.network.dto.forum.CategoryDto
import org.junit.Assert.*
import org.junit.Test

```

```

class ForumDtoMapperTest {
    @Test
    fun `maps two-level section-category structure with
        parentId`() {
        val dto = ForumDto(sections = listOf(
            SectionDto(id = "s1", name = "Linux", categories =
                listOf(
                    CategoryDto(id = "c10", name = "Ubuntu"),
                    CategoryDto(id = "c11", name = "Debian"),
                ))
        ))
        val tree = ForumDtoMapper().toForumTree(dto)
        assertEquals(1, tree.rootCategories.size)
        val linux = tree.rootCategories.first()
        assertEquals("Linux", linux.name)
        assertEquals(null, linux.parentId)
        assertEquals(2, linux.children.size)
    }
}

```

```

        assertEquals("s1", linux.children[0].parentId)
    }
}

```



Step 3: Run, commit.

```

git add core/tracker/rutracker/src/main/kotlin/lava/tracker/
      rutracker/mapper/ForumDtoMapper.kt \
      core/tracker/rutracker/src/test/kotlin/lava/tracker/
      rutracker/mapper/ForumDtoMapperTest.kt
git commit -m "sp3a-2.16: ForumDtoMapper full impl + test

```

Co-Authored-By: Claude Opus 4.7 (1M context)
 <noreply@anthropic.com>

Task 2.17: CategoryPageMapper full implementation

Similar shape to SearchPageMapper. Implement toBrowseResult(dto: CategoryPageDto, currentPage: Int): BrowseResult. The category metadata becomes the BrowseResult.category: ForumCategory?.

```

fun toBrowseResult(dto: CategoryPageDto, currentPage: Int):
    BrowseResult =
        BrowseResult(
            items = dto.items.map { /* same TorrentItem shape as
                                   SearchPageMapper */ },
            totalPages = dto.totalPages,
            currentPage = currentPage,
            category = dto.category?.let { ForumCategory(id = it.id,
                name = it.name, parentId = null, children =
                emptyList()) },
        )

```

Test with one normal-shape DTO and one with empty category. Commit with message sp3a-2.17: CategoryPageMapper full impl + test.

Task 2.18: TopicMapper full implementation

Maps ForumTopicDto → TopicDetail (extracts torrent metadata + description) and TopicPageDto → TopicPage (paginated list of posts/comments embedded in the topic).

Implement following the same pattern. Test with a DTO containing files list, magnet URI, and description. Commit sp3a-2.18: TopicMapper full impl + test.

Task 2.19: CommentsMapper full implementation

Maps CommentsPageDto → CommentsPage. Each PostDto becomes a Comment with author = post.author.name, timestamp = post.date, body = post.elements.joinToString { it.text } (flattening the rich post-element structure to plain text for the new model; the original elements survive in metadata keyed "rutracker.post.elements" as JSON).

Test with one normal post, one with embedded quote, one empty. Commit sp3a-2.19: CommentsMapper full impl + test.

Task 2.20: FavoritesMapper full implementation

Maps FavoritesDto → List<TorrentItem>. Same field extraction as search-page mapping. Test. Commit sp3a-2.20: FavoritesMapper full impl + test.

Task 2.21: AuthMapper full implementation

AuthResponseDto is a sealed class with three subtypes (Authorized, Unauthorized, NeedCaptcha). Map:

```
fun toLoginResult(dto: AuthResponseDto): LoginResult = when (dto)
{
    is AuthResponseDto.Authorized -> LoginResult(
        state = AuthState.Authenticated,
        sessionToken = dto.token,
    )
    is AuthResponseDto.Unauthorized -> LoginResult(state =
        AuthState.Unauthenticated)
    is AuthResponseDto.NeedCaptcha -> LoginResult(
        state = AuthState.CaptchaRequired(
            challenge = CaptchaChallenge(sid = dto.sid, code =
            dto.code, imageUrl = dto.imageUrl)
        ),
        captchaChallenge = CaptchaChallenge(sid = dto.sid, code =
        dto.code, imageUrl = dto.imageUrl),
    )
}
```

Test all three branches. Commit sp3a-2.21: AuthMapper full impl + test.

Section E — Reverse mappers for SwitchingNetworkApi

Tasks 2.22–2.27 produce **reverse mappers** that convert new-model types back into legacy DTO types. Used exclusively by the rewired SwitchingNetworkApi to preserve the existing NetworkApi surface for feature ViewModels.

Task 2.22: RuTrackerDtoMappers aggregator type

Files:

- Create: core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/mapper/RuTrackerDtoMappers.kt

```
package lava.tracker.rutracker.mapper
```

```
import javax.inject.Inject
import lava.network.dto.forum.ForumDto
import lava.network.dto.forum.CategoryPageDto
import lava.network.dto.search.SearchPageDto
import lava.network.dto.topic.ForumTopicDto
import lava.network.dto.topic.TopicPageDto
import lava.network.dto.topic.CommentsPageDto
import lava.network.dto.user.FavoritesDto
import lava.network.dto.auth.AuthResponseDto
import lava.tracker.api.model.*

/**
 * Inverse of the forward mappers. Used by SwitchingNetworkApi to
 * translate
 * new SDK results back into legacy DTOs that feature ViewModels
 * still expect.
 * Removed in Spec 2 when feature/* are migrated to consume
 * LavaTrackerSdk directly.
 */
class RuTrackerDtoMappers @Inject constructor() {
    fun searchResultToDto(result: SearchResult): SearchPageDto {
        TODO("Task 2.23")
    }
    fun browseResultToDto(result: BrowseResult): CategoryPageDto {
        TODO("Task 2.24")
    }
    fun forumTreeToDto(tree: ForumTree): ForumDto { TODO("Task 2.25") }
    fun topicDetailToDto(d: TopicDetail): ForumTopicDto {
        TODO("Task 2.26")
    }
    fun topicPageToDto(p: TopicPage): TopicPageDto { TODO("Task 2.26") }
    fun commentsPageToDto(p: CommentsPage): CommentsPageDto {
        TODO("Task 2.26")
    }
    fun favoritesToDto(items: List<TorrentItem>): FavoritesDto {
        TODO("Task 2.27")
    }
    fun loginResultToDto(r: LoginResult): AuthResponseDto {
        TODO("Task 2.27")
    }
}
```

Commit sp3a-2.22: RuTrackerDtoMappers aggregator stub.

Task 2.23: searchResultToDto reverse mapper

Implement the inverse of `SearchPageMapper.toResult`. Each `TorrentItem` becomes a `SearchItemDto` with `metadata["rutracker.author"]` lifted back into `author`, etc. Test with a round-trip: forward-map a fixture DTO, reverse-map the result, assert the reverse equals the forward input. Commit.

Task 2.24: browseResultToDto reverse mapper

Same shape as 2.23 but for `BrowseResult`  `CategoryPageDto`. Round-trip test. Commit.

Task 2.25: forumTreeToDto reverse mapper

Inverse of `ForumDtoMapper.toForumTree`. Test. Commit.

Task 2.26: topicDetailToDto, topicPageToDto, commentsPageToDto reverse mappers

Three inverse mappers for the topic/comments DTOs. Round-trip tests for each. Commit.

Task 2.27: favoritesToDto, loginResultToDto reverse mappers

Two final inverse mappers. Round-trip tests. Commit.

Section F — Registry registration + Hilt DI

Task 2.28: RuTrackerClientFactory

Files:

- Create: `core/tracker/rutracker/src/main/kotlin/lava/tracker/rutracker/RuTrackerClientFactory.kt`

```
package lava.tracker.rutracker
```

```
import javax.inject.Inject
import javax.inject.Provider
import lava.sdk.api.PluginConfig
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.registry.TrackerClientFactory
```

```
class RuTrackerClientFactory @Inject constructor(
```

```

        private val clientProvider: Provider<RuTrackerClient>,
    ) : TrackerClientFactory {
        override val descriptor: TrackerDescriptor =
            RuTrackerDescriptor
        override fun create(config: PluginConfig): TrackerClient =
            clientProvider.get()
    }

```

Commit sp3a-2.28: RuTrackerClientFactory.

Task 2.29: Hilt DI module for :core:tracker:client

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/di/TrackerClientModule.kt
- Modify: core/tracker/client/build.gradle.kts (add Hilt deps)

```

plugins {
    id("lava.android.library") // needed because Hilt-on-JVM-
                             // only is awkward; we keep client module Android-friendly
    id("lava.android.hilt")
    id("lava.kotlin.serialization")
}
android.namespace = "lava.tracker.client"
dependencies {
    api(project(":core:tracker:api"))
    api(project(":core:tracker:registry"))
    api(project(":core:tracker:mirror"))

    api(project(":core:tracker:rutracker")) // ensures
    // factory is on classpath
    implementation(libs.kotlinx.coroutines.core)
}

```

// TrackerClientModule.kt

```
package lava.tracker.client.di
```

```

import dagger.Module
import dagger.Provides
import dagger.hilt.InstallIn
import dagger.hilt.components.SingletonComponent
import javax.inject.Singleton
import lava.tracker.registry.DefaultTrackerRegistry
import lava.tracker.registry.TrackerRegistry
import lava.tracker.rutracker.RuTrackerClientFactory

```

```

@Module
@InstallIn(SingletonComponent::class)
object TrackerClientModule {
    @Provides @Singleton
    fun provideTrackerRegistry(rutrackerFactory:
        RuTrackerClientFactory): TrackerRegistry =

```

```

        DefaultTrackerRegistry().apply {
            register(rutrackerFactory) }
    }
}

```

(After Phase 3 ships, this provideTrackerRegistry will also accept RuTorClientFactory. Documented as a TODO inline.)

Commit sp3a-2.29: Hilt module for TrackerRegistry with RuTracker pre-registered.

Task 2.30: LavaTrackerSdk facade skeleton

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/LavaTrackerSdk.kt
- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/SearchOutcome.kt
- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/BrowseOutcome.kt

```

// LavaTrackerSdk.kt
package lava.tracker.client

import javax.inject.Inject
import javax.inject.Singleton
import kotlinx.coroutines.flow.MutableStateFlow
import kotlinx.coroutines.flow.StateFlow
import lava.sdk.api.MapPluginConfig
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.feature.*
import lava.tracker.api.model.*
import lava.tracker.registry.TrackerRegistry

@Singleton
class LavaTrackerSdk @Inject constructor(
    private val registry: TrackerRegistry,
) {
    private val _activeTrackerId = MutableStateFlow("rutracker")
    val activeTrackerId: StateFlow<String> = _activeTrackerId

    suspend fun switchTracker(trackerId: String) {
        require(registry.isRegistered(trackerId)) { "Unknown
            tracker: $trackerId" }
        _activeTrackerId.value = trackerId
    }

    fun listAvailableTrackers(): List<TrackerDescriptor> =
        registry.list()

    fun getActiveDescriptor(): TrackerDescriptor =

```



```

        getActiveClient().descriptor

private fun getActiveClient(): TrackerClient =
    registry.get(_activeTrackerId.value, MapPluginConfig())

suspend fun search(request: SearchRequest, page: Int = 0):
    SearchOutcome {
    val client = getActiveClient()
    val feature = client.getFeature(SearchableTracker::class)
    ?: return SearchOutcome.Failure(
        cause = IllegalStateException("Active tracker '$
        {client.descriptor.trackerId}' does not support SEARCH"),
        triedTrackers =
        listOf(client.descriptor.trackerId)
    )
    return try {
        val result = feature.search(request, page)
        SearchOutcome.Success(result, viaTracker =
        client.descriptor.trackerId)
    } catch (t: Throwable) {
        // Cross-tracker fallback policy lives in
        // CrossTrackerFallbackPolicy (Task 4.7).
        // For Phase 2, propagate as Failure; the cross-
        // tracker logic lights up in Phase 4.
        SearchOutcome.Failure(t, triedTrackers =
        listOf(client.descriptor.trackerId))
    }
}

suspend fun browse(category: String?, page: Int):
    BrowseOutcome { TODO("Task 2.31") }
suspend fun getTopic(id: String): TopicDetail { TODO("Task
2.31") }
suspend fun getMagnetLink(id: String): String? { TODO("Task
2.31") }
suspend fun downloadTorrent(id: String): ByteArray {
    TODO("Task 2.31") }
}

// SearchOutcome.kt
package lava.tracker.client

import lava.tracker.api.TrackerCapability
import lava.tracker.api.model.SearchResult

sealed class SearchOutcome {
    data class Success(val result: SearchResult, val viaTracker:
        String) : SearchOutcome()
    data class CrossTrackerFallbackProposed(
        val failedTrackerId: String,
        val proposedTrackerId: String,
        val capability: TrackerCapability,
        val resumeWith: suspend () -> SearchOutcome,
    ) : SearchOutcome()
}

```

```

        data class Failure(val cause: Throwable, val triedTrackers:
            List<String>) : SearchOutcome()
    }

    // BrowseOutcome.kt
    package lava.tracker.client

    import lava.tracker.api.model.BrowseResult

    sealed class BrowseOutcome {
        data class Success(val result: BrowseResult, val viaTracker:
            String) : BrowseOutcome()
        data class Failure(val cause: Throwable, val triedTrackers:
            List<String>) : BrowseOutcome()
    }

```

Commit sp3a-2.30: LavaTrackerSdk facade skeleton + SearchOutcome/
BrowseOutcome sealed types.

Task 2.31: Fill remaining LavaTrackerSdk TODO methods

Implement browse, getTopic, getMagnetLink, downloadTorrent following the search pattern. Each uses getActiveClient().getFeature(...), returns success/failure shapes. Tests using FakeTrackerClient. Commit.

Section G — SwitchingNetworkApi rewire

Task 2.32: Read existing SwitchingNetworkApi and design the rewire

Files:

- Read: core/network/impl/src/main/kotlin/lava/network/impl/SwitchingNetworkApi.kt



Step 1: Identify the current 14-method delegation pattern

Run: cat core/network/impl/src/main/kotlin/lava/network/impl/SwitchingNetworkApi.kt Note: each method delegates to one of two backing NetworkApi implementations based on configured endpoint.



Step 2: Plan the rewire — every method gets:

```

override suspend fun XYZ(...): XyzDto {
    val request = buildLavaSdkRequest(...)
    val outcome = sdk.<corresponding-call>(request)
    return mappers.<corresponding-reverse-

```

```
mapper>(outcome.successValue())
}
```

No changes in this task — design only. Commit a short architectural note as a draft inline doc in the existing file (a single comment line):

```
// SP-3a-2.32: SwitchingNetworkApi rewire planned for Task
2.33-2.35;
// rewires each method to delegate to LavaTrackerSdk +
RuTrackerDtoMappers.
```

Commit sp3a-2.32: design note for SwitchingNetworkApi rewire.

Task 2.33: Rewire authentication-related methods (checkAuthorized, login)

```
@Inject constructor(
    private val sdk: LavaTrackerSdk,
    private val ruTrackerMappers: RuTrackerDtoMappers,
) : NetworkApi {

    override suspend fun checkAuthorized(token: String): Boolean {
        val client = sdk.getActiveClient() // expose
        getActiveClient() as internal – visible
        to :core:network:impl
        val auth =
            client.getFeature(AuthenticatableTracker::class) ?:
            return false
        return auth.checkAuth() is AuthState.Authenticated
    }

    override suspend fun login(
        username: String, password: String,
        captchaSid: String?, captchaCode: String?, captchaValue:
        String?,
    ): AuthResponseDto {
        val client = sdk.getActiveClient()
        val auth =
            client.getFeature(AuthenticatableTracker::class)
                ?: error("Active tracker does not support AUTH")
        val captcha = if (captchaSid != null && captchaCode !=
            null && captchaValue != null) {
            CaptchaSolution(captchaSid, captchaCode, captchaValue)
        } else null
        val result = auth.login(LoginRequest(username, password,
            captcha))
        return ruTrackerMappers.loginResultToDto(result)
    }
}
```

(Expose LavaTrackerSdk.getActiveClient() as internal if not already.)

Test using FakeTrackerClient. Commit.

Task 2.34: Rewire forum/category/search/favorites methods

7 more methods: getFavorites, addFavorite, removeFavorite, getForum, getCategory, getSearchPage, addComment. Each follows the same pattern. Test each. Commit as sp3a-2.34.

Task 2.35: Rewire topic/comments/torrent/download methods

Final 5 methods: getTopic, getTopicPage, getCommentsPage, getTorrent, download. Test. Commit sp3a-2.35.

Section H — Parity gate

Task 2.36: Cross-version parity test against pre-SP-3a tag

Files:

- Create: core/network/impl/src/test/kotlin/lava/network/impl/SwitchingNetworkApiParityTest.kt
- Create: .lava-ci-evidence/sp3a-parity/phase2-baseline.json

The strategy: capture the pre-SP-3a behavior of every NetworkApi method as JSON fixtures (one fixture per method × shape), then assert the rewired SwitchingNetworkApi produces byte-for-byte identical JSON output.



Step 1: Generate baseline fixtures from pre-SP-3a tag

```
# Check out the last pre-SP-3a release tag (Lava-
    Android-1.1.4-1014)
git workspace add /tmp/lava-1.1.4 Lava-Android-1.1.4-1014
cd /tmp/lava-1.1.4
# Run a fixture-generating test that exercises every NetworkApi
    method
# against a stubbed HTTP server with known HTML fixtures,
    capturing
# the JSON-serialized DTO output to test resources.
./gradlew :core:network:impl:test --tests
    "*ParityFixtureGenerator*" -PgenerateBaseline=true
# Copy the generated fixtures back to the SP-3a branch
cp -r core/network/impl/build/test-results/parity-baseline /run/
    media/milosvasic/DATA4TB/Projects/Lava/core/network/impl/
    src/test/resources/parity-baseline-1.1.4/
git workspace remove /tmp/lava-1.1.4
```

(If no ParityFixtureGenerator exists yet, this task creates one — write a JUnit test that calls each NetworkApi method with stubbed inputs and writes `Json.encodeToString(...)` of the result to a file under `build/test-results/parity-baseline/<method>.json`.)



Step 2: On SP-3a branch, write the parity test

```
package lava.network.impl

import com.google.common.truth.Truth.assertThat
import kotlinx.coroutines.test.runTest
import kotlinx.serialization.json.Json
import lava.network.dto.search.SearchPageDto
import org.junit.Test
import java.io.File

class SwitchingNetworkApiParityTest {

    private val json = Json { prettyPrint = true; encodeDefaults = true }
    private val baselineDir = File("src/test/resources/parity-baseline-1.1.4")

    @Test
    fun `getSearchPage produces byte-identical output vs 1.1.4 baseline`() = runTest {
        val sdk =
            makeSdkWithStubbedRuTracker() // helper sets up
            SwitchingNetworkApi + LavaTrackerSdk + fixture HTML
        val api = SwitchingNetworkApi(sdk = sdk, ruTrackerMappers = RuTrackerDtoMappers())
        val result = api.getSearchPage(
            token = "test-token", searchQuery = "ubuntu",
            categories = null,
            author = null, authorId = null, sortType = null,
            sortOrder = null,
            period = null, page = 0,
        )
        val produced =
            json.encodeToString(SearchPageDto.serializer(), result)
        val baseline = File(baselineDir,
            "getSearchPage.json").readText()
        assertThat(produced).isEqualTo(baseline)
    }

    // Repeat for all 14 NetworkApi methods – 14 tests total.
}
```



Step 3: Run, confirm pass

Run: `./gradlew :core:network:impl:test --tests "*SwitchingNetworkApiParityTest*" Expected: PASS` — all 14 parity assertions green.

If any test fails, the divergence is real — root-cause it (likely a forward+reverse mapper round-trip dropping a field). Fix the mapper, re-run.



Step 4: Record evidence

```
// .lava-ci-evidence/sp3a-parity/phase2-baseline.json
{
  "task": "sp3a-2.36",
  "subject": "SwitchingNetworkApi parity vs Lava-
    Android-1.1.4-1014",
  "methods_tested": [
    "checkAuthorized", "login", "getFavorites", "addFavorite",
    "removeFavorite",
    "getForum", "getCategory", "getSearchPage", "getTopic",
    "getTopicPage",
    "getCommentsPage", "addComment", "getTorrent", "download"
  ],
  "all_methods_byte_identical": true,
  "baseline_tag": "Lava-Android-1.1.4-1014",
  "ledger_clause": "6.D"
}
```



Step 5: Commit

```
git add core/network/impl/src/test/kotlin/lava/network/impl/
    SwitchingNetworkApiParityTest.kt \
    core/network/impl/src/test/resources/parity-
    baseline-1.1.4/ \
    .lava-ci-evidence/sp3a-parity/phase2-baseline.json
git commit -m "sp3a-2.36: parity gate – SwitchingNetworkApi
    matches 1.1.4 baseline byte-for-byte"
```

All 14 NetworkApi methods produce identical JSON output via the
new
LavaTrackerSdk underneath compared to the pre-SP-3a baseline.
Phase 2
acceptance gate green.

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Phase 2 acceptance check. Run from repo root:

```
./gradlew :app:assembleDebug :core:tracker:rutracker:test :core:network:impl:test
```

Expected: BUILD SUCCESSFUL — debug APK builds, all tests pass.

```
ls -la core/tracker/rutracker/src/main/kotlin/lava/tracker/
    rutracker/feature/
```

Expected: 7 feature impl files (Search, Browse, Topic, Comments, Favorites, Auth, Download).

```
grep -r 'core:network:rutracker' settings.gradle.kts || echo  
"rutracker module no longer at old location - correct"
```

Expected: prints "rutracker module no longer at old location — correct".

Phase 2 done. Phase 3 (RuTor) unblocked. Phase 4 may begin in parallel.

Phase 3: RuTor Implementation

Duration: 2 weeks. **Tasks:** 41. **Goal:** Implement RuTor (rutor.info / rutor.is) as the first additional tracker, validating that the SDK contract is real and not RuTracker-shaped in disguise. Anonymous-by-default per decision 7b-ii.

Acceptance gate: `:core:tracker:rutor:test` green; ≥ 5 HTML fixtures per parser $\times$ 4 parsers (search/browse/topic/login) = 20 fixtures committed; real-tracker integration test passes against rutor.info; RuTorClientFactory registered in DI module alongside RuTracker; `LavaTrackerSdk.switchTracker("rutor")` works end-to-end.

Section A — Module setup

Task 3.1: Create `:core:tracker:rutor` module skeleton

Files:

- Create: `core/tracker/rutor/build.gradle.kts`
- Create: `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/.gitkeep`
- Create: `core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/.gitkeep`
- Create: `core/tracker/rutor/src/test/resources/fixtures/rutor/.gitkeep`



Step 1: `build.gradle.kts`

```
plugins {  
    id("lava.kotlin.tracker.module")  
}  
android.namespace = "lava.tracker.rutor" // remove this line if  
    convention plugin is JVM-only
```

(Note: tracker convention plugin is JVM-only per Phase 1 Task 1.30. The namespace line above is therefore not applicable. Just `plugins { id("lava.kotlin.tracker.module") }` is enough.)



Step 2: Add to `settings.gradle.kts`

Already added in Task 1.32 anticipating this. Verify present: `grep ':core:tracker:rutor' settings.gradle.kts`. If commented, uncomment.



Step 3: Verify

Run: `./gradlew :core:tracker:rutor:compileKotlin` Expected: BUILD SUCCESSFUL.



Step 4: Commit

```
git add core/tracker/rutor/build.gradle.kts \
      core/tracker/rutor/src/main/kotlin/lava/tracker/
      rutor/.gitkeep \
      core/tracker/rutor/src/test/kotlin/lava/tracker/
      rutor/.gitkeep \
      core/tracker/rutor/src/test/resources/fixtures/
      rutor/.gitkeep
git commit -m "sp3a-3.1: scaffold :core:tracker:rutor module"
```

Co-Authored-By: Claude Opus 4.7 (1M context)
<noreply@anthropic.com>

Task 3.2: Add RuTorClient and feature-impl placeholders that compile

Files:

- Create: `core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/RuTorClient.kt` (skeleton — full impl in Task 3.40)



Step 1: Skeleton client

```
package lava.tracker.rutor
```

```
import javax.inject.Inject
import kotlin.reflect.KClass
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.TrackerFeature
```

```
/** Skeleton — full implementation in Task 3.40. */
class RuTorClient @Inject constructor() : TrackerClient {
    override val descriptor: TrackerDescriptor =
        RuTorDescriptor // created in Task 3.6
    override suspend fun healthCheck(): Boolean = false
    override fun <T : TrackerFeature> getFeature(featureClass:
        KClass<T>): T? = null
    override fun close() {}
}
```


(Compile fails until Task 3.6 creates RuTorDescriptor. Expected.)

Commit sp3a-3.2: RuTorClient skeleton (compile blocked until Task 3.6).

Task 3.3: Test resource directory layout for fixtures

Files:

- Create: core/tracker/rutor/src/test/resources/fixtures/rutor/{search,browse,topic,login,comments}/.gitkeep



Step 1:

```
for dir in search browse topic login comments; do
  mkdir -p core/tracker/rutor/src/test/resources/fixtures/rutor/
    $dir
  touch core/tracker/rutor/src/test/resources/fixtures/rutor/
    $dir/.gitkeep
done
```

Commit sp3a-3.3: fixture directory scaffolding for 5 parser scopes.

Task 3.4: README for the rutor module

Files:

- Create: core/tracker/rutor/README.md

```
# :core:tracker:rutor
```

RuTor (rutor.info / rutor.is) tracker plugin. Implements

``TrackerClient``

+ feature interfaces from ``:core:tracker:api``.

Capabilities

SEARCH, BROWSE, TOPIC, COMMENTS, TORRENT_DOWNLOAD, MAGNET_LINK,
RSS,

AUTH_REQUIRED. ****Not implemented:**** FORUM (RuTor has categories
only),

FAVORITES (no list endpoint comparable to RuTracker).

Encoding

UTF-8 throughout. No charset transcoding (unlike RuTracker which
is

Windows-1251).

Login policy (decision 7b-ii)

Anonymous by default. Login is invoked only when the SDK consumer calls `CommentsTracker.addComment()` (or any other authenticated operation) and `checkAuth()` returns Unauthenticated.

Mirrors

See `app/src/main/assets/mirrors.json` for the bundled list. Health probe checks for substring "RuTor" in response body.

Adding a parser fixture

Save HTML to `src/test/resources/fixtures/rutor/<scope>/<name>-<date>.html`. Refresh older than 30 days warns; older than 60 blocks tag. See `docs/refactoring/decoupling/refresh-fixtures.md`.

Commit sp3a-3.4: rutor module README.

Task 3.5: Add module to core/CLAUDE.md scoped doc

Files:

- Modify: core/CLAUDE.md

Add a section pointing readers to the new tracker module pattern (one paragraph). Commit sp3a-3.5: scoped CLAUDE.md note for tracker plugin modules.

Section B — RuTorDescriptor

Task 3.6: Implement RuTorDescriptor

Files:

- Create: core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/RuTorDescriptor.kt
- Create: core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/RuTorDescriptorTest.kt

```
package lava.tracker.rutor
```

```
import lava.sdk.api.MirrorUrl
import lava.sdk.api.Protocol
import lava.tracker.api.AuthType
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerDescriptor
```

```
object RuTorDescriptor : TrackerDescriptor {
```

```

override val trackerId = "rutor"
override val displayName = "RuTor.info"
override val baseUrls = listOf(
    MirrorUrl("https://rutor.info", isPrimary = true,
        priority = 0, protocol = Protocol.HTTPS),
    MirrorUrl("https://rutor.is",
        priority = 1, protocol = Protocol.HTTPS),
    MirrorUrl("https://www.rutor.info",
        priority = 2, protocol = Protocol.HTTPS),
    MirrorUrl("https://www.rutor.is",
        priority = 3, protocol = Protocol.HTTPS),
    MirrorUrl("http://6tor.org",
        priority = 4, protocol = Protocol.HTTP, region = "ipv6-
only"),
)
override val capabilities = setOf(
    TrackerCapability.SEARCH,
    TrackerCapability.BROWSE,
    TrackerCapability.TOPIC,
    TrackerCapability.COMMENTS,
    TrackerCapability.TORRENT_DOWNLOAD,
    TrackerCapability.MAGNET_LINK,
    TrackerCapability.RSS,
    TrackerCapability.AUTH_REQUIRED,
    // No FORUM, no FAVORITES – RuTor lacks these in a
    // comparable form.
)
override val authType = AuthType.FORM_LOGIN // simple POST,
no captcha
override val encoding = "UTF-8"
override val expectedHealthMarker = "RuTor"
}

```

Test asserts: 8 capabilities (no FORUM, no FAVORITES, no CAPTCHA_LOGIN), 5 mirrors, primary is rutor.info, encoding UTF-8.

Commit sp3a-3.6: RuTorDescriptor + tests.

Section C — HTTP client (cookies, semaphore, circuit breaker)

Task 3.7: RuTorHttpClient skeleton

Files:

- Create: core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/http/RuTorHttpClient.kt

```
package lava.tracker.rutor.http
```

```

import java.net.CookieManager
import java.net.CookiePolicy
import java.util.concurrent.TimeUnit

```

```

import javax.inject.Inject
import javax.inject.Singleton
import kotlinx.coroutines.Dispatchers
import kotlinx.coroutines.sync.Semaphore
import kotlinx.coroutines.sync.withPermit
import kotlinx.coroutines.withContext
import okhttp3.FormBody
import okhttp3.JavaNetCookieJar
import okhttp3.OkHttpClient
import okhttp3.Request
import okhttp3.Response

@Singleton
class RuTorHttpClient @Inject constructor() {
    private val cookieJar =
        JavaNetCookieJar(CookieManager().apply {
            setCookiePolicy(CookiePolicy.ACCEPT_ALL) })
    private val client = OkHttpClient.Builder()
        .cookieJar(cookieJar)
        .connectTimeout(10, TimeUnit.SECONDS)
        .readTimeout(15, TimeUnit.SECONDS)
        .followRedirects(true)
        .build()

    /** Per-tracker semaphore: max 4 concurrent requests, per
        spec. */
    private val concurrency = Semaphore(permits = 4)

    suspend fun get(url: String): Response =
        withContext(Dispatchers.IO) {
            concurrency.withPermit {

                client.newCall(Request.Builder().url(url).get().header("User-
Agent", USER_AGENT).build()).execute()
            }
        }

    suspend fun postForm(url: String, fields: Map<String,
String>): Response = withContext(Dispatchers.IO) {
        concurrency.withPermit {
            val body = FormBody.Builder().apply { fields.forEach
            { (k, v) -> add(k, v) } }.build()

            client.newCall(Request.Builder().url(url).post(body).header("User-
Agent", USER_AGENT).build()).execute()
        }
    }

    suspend fun download(url: String): ByteArray =
        withContext(Dispatchers.IO) {
            concurrency.withPermit {

                client.newCall(Request.Builder().url(url).get().header("User-
Agent", USER_AGENT).build()).execute().use {

```

```

        require(it.isSuccessful) {
            "RuTor download failed: HTTP ${it.code} for $url" }
        it.body?.bytes() ?: error("empty response body")
    }
}

fun hasCookie(name: String): Boolean =
    cookieJar.loadForRequest(okhttp3.HttpUrl.Builder().scheme("https").host("rutor"))
        .any { it.name == name }

fun clearCookies() {
    // CookieManager-based cookieJar can't be cleared without
    // reflection; reset by recreating the jar
    // – for SP-3a, we accept that logout requires app
    // restart. Documented in README.
}

private companion object {
    const val USER_AGENT = "Mozilla/5.0 (Linux; Android 14)
    Lava/1.2.0"
}
}

```

Commit sp3a-3.7: RuTorHttpClient with cookie jar + semaphore-bounded concurrency.

Task 3.8: Add circuit breaker around RuTorHttpClient

Use vasic-digital/Recovery submodule's CircuitBreaker if available, otherwise a simple coroutine-based one with thresholds: 3 failures within 30s → OPEN for 30s.

Files:

- Modify: core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/http/RuTorHttpClient.kt

Add a CircuitBreaker field and wrap get/postForm/download calls. If breaker is OPEN, throw CircuitBreakerOpenException (treated as a per-mirror failure by MirrorManager.executeWithFallback).

Commit sp3a-3.8: circuit breaker integrated into RuTorHttpClient (3 failures / 30s).

Task 3.9: Test RuTorHttpClient with MockWebServer

Files:

- Create: core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/http/RuTorHttpClientTest.kt

Tests:

1. GET succeeds, response body returned as expected.
2. Cookies set in response are persisted and sent on next request.
3. Concurrent calls beyond semaphore limit (5+) serialize correctly (max 4 in flight).
4. Circuit breaker opens after 3 consecutive failures, blocks the 4th call.
5. Circuit breaker resets after 30s.

Commit sp3a-3.9: RuTorHttpClient unit tests (cookies, semaphore, breaker).

Task 3.10: Falsifiability rehearsal — semaphore actually bounds concurrency

Files:

- Create: core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/http/RuTorHttpClientFalsifiabilityTest.kt
- Create: .lava-ci-evidence/sp3a-rutor/3.10-semaphore.json

The deliberate break: change `permits = 4` to `permits = 100`; the concurrency-bound test should fail (we'd see 5+ in flight). Revert; passes. Record evidence. Commit.

Section D — Date and size parsers

Task 3.11: RuTorDateParser

Files:

- Create: core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/parser/RuTorDateParser.kt
- Create: core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/parser/RuTorDateParserTest.kt

```
package lava.tracker.rutor.parser
```

```
import java.time.LocalDate
import java.time.ZoneOffset
import java.time.temporal.ChronoUnit
import kotlinx.datetime.Clock
import kotlinx.datetime.Instant
import kotlinx.datetime.toJavaInstant
import kotlinx.datetime.toKotlinInstant
```

```
object RuTorDateParser {
    private val months = mapOf(
        "Янв" to 1, "Фев" to 2, "Мар" to 3, "Апр" to 4, "Май" to
        5, "Июн" to 6,
```

```

        "Июл" to 7, "Авг" to 8, "Сен" to 9, "Окт" to 10, "Ноя" to
        11, "Дек" to 12,
    )
    fun parse(s: String, now: () -> Instant = Clock.System::now):
        Instant? {
        val trimmed = s.trim()
        return when {
            trimmed.equals("Сегодня", ignoreCase = true) ->

            now().toJavaInstant().truncatedTo(ChronoUnit.DAYS).toKotlinInstant()
            trimmed.equals("Вчера", ignoreCase = true) ->

            now().toJavaInstant().truncatedTo(ChronoUnit.DAYS).minus(1,
            ChronoUnit.DAYS).toKotlinInstant()
            else -> {
                val match = Regex("""(\d{1,2})\s+(\S+)\s+
                (\d{2,4})""").find(trimmed) ?: return null
                val (day, monthAbbr, year) = match.destructured
                val month = months[monthAbbr.take(3)] ?: return
                null
                val fullYear = if (year.length == 2) 2000 +
                year.toInt() else year.toInt()
                LocalDate.of(fullYear, month, day.toInt())
                    .atStartOfDay(ZoneOffset.UTC)
                    .toInstant()
                    .toKotlinInstant()
            }
        }
    }
}

```

Tests cover: "12 ЯНВ 24" → 2024-01-12, "Сегодня" → today truncated, "Вчера" → yesterday truncated, "garbage" → null, "5 Дек 2023" → 2023-12-05, all 12 months exercised.

Commit sp3a-3.11: RuTorDateParser with Russian months + Сегодня/Вчера + 14 tests.

Task 3.12: RuTorSizeParser

Files:

- Create: core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/parser/RuTorSizeParser.kt
- Create: core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/parser/RuTorSizeParserTest.kt

```
package lava.tracker.rutor.parser
```

```

object RuTorSizeParser {
    private val pattern = Regex("""(\d+(?:[.,]\d+)?)\s*(GB|MB|KB|
    B|TB)""", RegexOptions.IGNORE_CASE)
    fun parse(s: String): Long? {

```

```

        val m = pattern.find(s) ?: return null
        val (number, unit) = m.destructured
        val value = number.replace(',', '.').toDoubleOrNull() ?:
            return null
        val multiplier = when (unit.uppercase()) {
            "B" -> 1L
            "KB" -> 1_024L
            "MB" -> 1_024L * 1_024L
            "GB" -> 1_024L * 1_024L * 1_024L
            "TB" -> 1_024L * 1_024L * 1_024L * 1_024L
            else -> return null
        }
        return (value * multiplier).toLong()
    }
}

```

Tests: "4.5 GB" → 4_831_838_208, "1,5 GB" (comma decimal) → 1_610_612_736, "1024 MB" → 1_073_741_824, "200 kB" → 204_800, "garbage" → null.

Commit

sp3a-3.12: RuTorSizeParser supports GB/MB/kB/B/TB and comma decimals.

Task 3.13: Falsifiability rehearsal — date parser handles "Сегодня" correctly

Mutate the parser to ignore "Сегодня"; assert the existing test fails; revert; passes. Evidence at [.lava-ci-evidence/sp3a-rutor/3.13-date-parser.json](#). Commit.

Section E — Search parser + 5 fixtures

Task 3.14: RuTorSearchParser skeleton with content-based selectors

Files:

- Create: core/tracker/rutor/src/main/kotlin/lava/tracker/rutor/parser/RuTorSearchParser.kt

```
package lava.tracker.rutor.parser
```

```

import javax.inject.Inject
import lava.tracker.api.model.SearchResult
import lava.tracker.api.model.TorrentItem
import org.jsoup.Jsoup
import org.jsoup.nodes.Element

```

```
class RuTorSearchParser @Inject constructor() {
```



```

fun parse(html: String, currentPage: Int, currentMirror:
String): SearchResult {
    val doc = Jsoup.parse(html)
    val rows = doc.select("tr:has(td:has(a[href^=magnet:?
xt=]))")
    val items = rows.map { row -> parseRow(row,
currentMirror) }
    val totalPages = (doc.select("td.pages a").mapNotNull {
it.text().toIntOrNull() }.maxOrNull() ?: 1)
        .coerceAtLeast(1)
    return SearchResult(items = items, totalPages =
totalPages, currentPage = currentPage)
}

private fun parseRow(row: Element, currentMirror: String):
TorrentItem {
    val titleLink = row.selectFirst("td:nth-of-type(2)
a[href^=/torrent/]")
        ?: error("RuTor row missing title link: $
{row.html().take(200)}")
    val magnet = row.selectFirst("a[href^=magnet:?
xt=]")?.attr("href")
    val downloadHref =
row.selectFirst("a.downgif")?.attr("href")
    // Content-based size selector: tolerates the comments-
count column drift
    val sizeText = row.select("td").firstOrNull { td ->
RuTorSizeParser.parse(td.text()) != null
    }?.text()
    val seeders = row.selectFirst("td
span.green")?.text()?.toIntOrNull()
    val leechers = row.selectFirst("td
span.red")?.text()?.toIntOrNull()
    val date = row.selectFirst("td:nth-of-
type(1)")?.text()?.let(RuTorDateParser::parse)
    val infoHash = magnet?.let { Regex("""[A-Fa-f0-9]
{40}""").find(it)?.value }
    val torrentId = titleLink.attr("href").substringAfter("/
torrent/").substringBefore("/")
    return TorrentItem(
        trackerId = "rutor",
        torrentId = torrentId,
        title = titleLink.text(),
        sizeBytes = sizeText?.let(RuTorSizeParser::parse),
        seeders = seeders,
        leechers = leechers,
        infoHash = infoHash,
        magnetUri = magnet,
        downloadUrl = downloadHref?.let {
resolveAgainst(currentMirror, it) },
        detailUrl = resolveAgainst(currentMirror,
titleLink.attr("href")),
        category = null,
        publishDate = date,
        metadata = emptyMap(),

```

```

    )
}

private fun resolveAgainst(base: String, relative: String):
    String =
    if (relative.startsWith("http")) relative else
    base.trimEnd('/') + relative
}

```

Commit sp3a-3.14: RuTorSearchParser skeleton with content-based size selector.

Task 3.15: Search fixture — `search-normal-2026-04-30.html`

Files:

- Create: `core/tracker/rutor/src/test/resources/fixtures/rutor/search/search-normal-2026-04-30.html`

Scrape a real RuTor search page (e.g., `/search/0/0/000/0/ubuntu`) using `curl` from a local machine, save the response. The fixture must contain at least 10 torrent rows with: magnet links, sizes (mix of GB/MB), seeders, leechers, dates (mix of "Сегодня"/"Вчера"/explicit), infohashes derivable from magnets.

Commit sp3a-3.15: `search-normal-2026-04-30.html` (10 torrent rows scraped from `rutor.info`).

Task 3.16: Search fixture — `search-empty-2026-04-30.html`

Scrape `/search/0/0/000/0/zzzzzznoresults`. Should produce a page with no torrent rows but valid HTML structure.

Commit sp3a-3.16: `search-empty-2026-04-30.html`.

Task 3.17: Search fixture — `search-edge-columns-2026-04-30.html`

Locate (or construct from a real scrape) a search result with rows missing the comments-count column. This tests the content-based size selector.

Commit sp3a-3.17: `search-edge-columns-2026-04-30.html` (variable column hazard).

Task 3.18: Search fixture — `search-cyrillic-2026-04-30.html`

Scrape `/search/0/0/000/0/война` (Russian query). Results should have Cyrillic titles, Cyrillic in `Сегодня/Вчера` date cells, possibly `Ё/ё` characters.

Commit sp3a-3.18: search-cyrillic-2026-04-30.html.

Task 3.19: Search fixture — search-malformed-2026-04-30.html

Hand-construct a malformed HTML file (e.g., truncated mid-row, unclosed tags, missing magnet link in a row). Used to verify parser gracefully throws or skips bad rows without crashing.

Commit sp3a-3.19: search-malformed-2026-04-30.html (hand-crafted edge case).

Task 3.20: RuTorSearchParserTest — 5 fixtures × assertions

Files:

- Create: core/tracker/rutor/src/test/kotlin/lava/tracker/rutor/parser/RuTorSearchParserTest.kt

```
package lava.tracker.rutor.parser
```

```
import com.google.common.truth.Truth.assertThat
import lava.tracker.testing.LavaFixtureLoader
import org.junit.Test
```

```
class RuTorSearchParserTest {
    private val parser = RuTorSearchParser()
    private val fixtures = LavaFixtureLoader("rutor")

    @Test
    fun `normal page parses 10+ items with non-null core fields`() {
        val html = fixtures.load("search", "search-normal-2026-04-30.html")
        val result = parser.parse(html, currentPage = 0,
            currentMirror = "https://rutor.info")

        assertThat(result.items).hasSize(10) // adapt N to actual
        fixture row count
        for (item in result.items) {
            assertThat(item.title).isNotEmpty()
            assertThat(item.magnetUri).isNotNull()
            assertThat(item.infoHash).matches("[A-Fa-f0-9]{40}")
            assertThat(item.sizeBytes).isNotNull()
            assertThat(item.seeders).isNotNull()
        }
    }

    @Test
    fun `empty page returns zero items`() {
        val html = fixtures.load("search", "search-empty-2026-04-30.html")
    }
}
```

```

        val result = parser.parse(html, currentPage = 0,
            currentMirror = "https://rutor.info")
        assertThat(result.items).isEmpty()
    }

    @Test
    fun `edge_columns fixture - content-based selector finds size
    despite column drift`() {
        val html = fixtures.load("search", "search-edge-
            columns-2026-04-30.html")
        val result = parser.parse(html, currentPage = 0,
            currentMirror = "https://rutor.info")
        for (item in result.items) {
            assertThat(item.sizeBytes).isNotNull() // load-
            bearing assertion: content-based selector worked
        }
    }

    @Test
    fun `cyrillic fixture parses without character corruption`() {
        val html = fixtures.load("search", "search-
            cyrillic-2026-04-30.html")
        val result = parser.parse(html, currentPage = 0,
            currentMirror = "https://rutor.info")
        // Assert at least one item has a Cyrillic character in
        the title
        assertThat(result.items.any { it.title.any { c -> c.code
            in 0x0400..0x04FF } }).isTrue()
    }

    @Test
    fun `malformed fixture does not crash - gracefully returns
    whatever it could parse`() {
        val html = fixtures.load("search", "search-
            malformed-2026-04-30.html")
        val result = parser.parse(html, currentPage = 0,
            currentMirror = "https://rutor.info")
        // Acceptance: did not throw. Empty or partial results are
        fine.
        assertThat(result.items).isNotNull()
    }
}

```

Run: ./gradlew :core:tracker:rutor:test --tests
 "*RuTorSearchParserTest*". All 5 pass.

Commit sp3a-3.20: RuTorSearchParserTest (5 fixtures × assertions).

Section F — Browse parser + 5 fixtures

Task 3.21: RuTorBrowseParser skeleton

Same shape as RuTorSearchParser but parses `/browse/{page}/{cat}/...` URLs. Categories live in a top bar; current category extracted into `BrowseResult.category`.

Commit `sp3a-3.21: RuTorBrowseParser skeleton`.

Tasks 3.22–3.25: Browse fixtures + test

- Task 3.22: `browse-normal-2026-04-30.html` (scraped from `/browse/0/0/000/0`)
- Task 3.23: `browse-empty-2026-04-30.html` (scraped from a known-empty category)
- Task 3.24: `browse-deep-pagination-2026-04-30.html` (scraped from page 50+)
- Task 3.25: `RuTorBrowseParserTest` with 4 fixtures × assertions (3.24 covers `single-result` + malformed via composition)

Commit each.

Section G — Topic parser + 5 fixtures

Task 3.26: RuTorTopicParser skeleton

Parses `/torrent/{id}/{slug}` pages — extracts title, description, magnet, file list, comment count.

Commit `sp3a-3.26: RuTorTopicParser skeleton`.

Tasks 3.27–3.30: Topic fixtures + test

- Task 3.27: `topic-normal-2026-04-30.html`
- Task 3.28: `topic-with-files-2026-04-30.html`
- Task 3.29: `topic-with-long-description-2026-04-30.html`
- Task 3.30: `RuTorTopicParserTest` with 5 fixtures × assertions (combine `no-magnet` + malformed under one fixture each)

Commit each.

Section H — Comments parser + Login parser + fixtures

Task 3.31: RuTorCommentsParser

Comments are embedded in the topic page; this parser extracts the `<div id="comments">` block.

Commit sp3a-3.31: RuTorCommentsParser.

Task 3.32: RuTorLoginParser

Parses the response of `POST /users.php?login`. Detects: success (redirect with `userid` cookie), wrong password (login form re-rendered with error text), account locked (specific error variant).

Commit sp3a-3.32: RuTorLoginParser.

Tasks 3.33–3.35: Login fixtures + test

- Task 3.33: `login/success-with-userid-cookie-2026-04-30.html` (the redirect-target home page)
- Task 3.34: `login/failure-wrong-password-2026-04-30.html`
- Task 3.35: `RuTorLoginParserTest` with 5 fixtures (success, wrong-password, account-locked, redirect-to-home, malformed)

Commit each.

Section I — Feature implementations

Task 3.36: RuTorSearch feature impl

```
package lava.tracker.rutor.feature
```

```
import javax.inject.Inject
import lava.tracker.api.feature.SearchableTracker
import lava.tracker.api.model.SearchRequest
import lava.tracker.api.model.SearchResult
import lava.tracker.rutor.http.RuTorHttpClient
import lava.tracker.rutor.parser.RuTorSearchParser
```

```
class RuTorSearch @Inject constructor(
    private val http: RuTorHttpClient,
    private val parser: RuTorSearchParser,
) : SearchableTracker {
    override suspend fun search(request: SearchRequest, page:
        Int): SearchResult {
        val mirror = "https://rutor.info" // resolved via
        MirrorManager in Phase 4; for now hardcoded
```

```

        val url = buildString {
            append(mirror)
            append("/search/").append(page)
            append("/0") // category 0 = all
            append("/000") // method-scope-method composed
            append("/0") // sort

            append("/").append(java.net.URLEncoder.encode(request.query,
                "UTF-8"))
        }
        val response = http.get(url)
        val html = response.use { it.body?.string() ?:
            error("empty body") }
        return parser.parse(html, currentPage = page,
            currentMirror = mirror)
    }
}

```

Commit sp3a-3.36: RuTorSearch feature impl.

Task 3.37: RuTorBrowse, RuTorTopic, RuTorComments feature impls

Three parallel files following the same pattern. Commit sp3a-3.37: RuTor Browse/Topic/Comments feature impls.

Task 3.38: RuTorAuth feature impl

```

class RuTorAuth @Inject constructor(
    private val http: RuTorHttpClient,
    private val parser: RuTorLoginParser,
) : AuthenticatableTracker {
    override suspend fun login(req: LoginRequest): LoginResult {
        val response = http.postForm("https://rutor.info/
            users.php?login", mapOf(
                "nick" to req.username, "password" to req.password,
                "login" to "Bxoq"
            ))
        val html = response.use { it.body?.string().orEmpty() }
        return parser.parseLoginResponse(html, sessionToken = if
            (http.hasCookie("userid")) http.cookieValue("userid")
            else null)
    }
    override suspend fun logout() = http.clearCookies()
    override suspend fun checkAuth(): AuthState =
        if (http.hasCookie("userid")) AuthState.Authenticated
        else AuthState.Unauthenticated
}

```

Commit

sp3a-3.38: RuTorAuth – anonymous-by-default policy enforced upstream by LavaTrackerSdk.

Task 3.39: RuTorDownload feature impl

```
class RuTorDownload @Inject constructor(private val http:
    RuTorHttpClient) : DownloadableTracker {
    override suspend fun downloadTorrentFile(id: String):
        ByteArray =
            http.download("https://d.rutor.info/download/$id")
    override fun getMagnetLink(id: String): String? =
        null // not synchronously available; consumer fetches
            detail
}
```

Commit sp3a-3.39: RuTorDownload feature impl.

Section J — RuTorClient + Factory + DI registration

Task 3.40: RuTorClient full implementation + RuTorClientFactory + Hilt registration

```
// RuTorClient.kt – full impl replacing the Task 3.2 skeleton
package lava.tracker.rutor

import javax.inject.Inject
import kotlin.reflect.KClass
import lava.tracker.api.TrackerCapability
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.api.TrackerFeature
import lava.tracker.api.feature.*
import lava.tracker.rutor.feature.*
import lava.tracker.rutor.http.RuTorHttpClient

class RuTorClient @Inject constructor(
    private val http: RuTorHttpClient,
    private val search: RuTorSearch,
    private val browse: RuTorBrowse,
    private val topic: RuTorTopic,
    private val comments: RuTorComments,
    private val auth: RuTorAuth,
    private val download: RuTorDownload,
) : TrackerClient {
    override val descriptor: TrackerDescriptor = RuTorDescriptor

    override suspend fun healthCheck(): Boolean =
        runCatching { http.get("https://rutor.info").use {
            it.code == 200 } }.getOrDefault(false)

    @Suppress("UNCHECKED_CAST")
    override fun <T : TrackerFeature> getFeature(featureClass:
        KClass<T>): T? {
```



```

        val declared = descriptor.capabilities
        return when (featureClass) {
            SearchableTracker::class -> if
                (TrackerCapability.SEARCH in declared) search as T else
                null
            BrowseableTracker::class -> if
                (TrackerCapability.BROWSE in declared) browse as T else
                null
            TopicTracker::class -> if (TrackerCapability.TOPIC in
                declared) topic as T else null
            CommentsTracker::class -> if
                (TrackerCapability.COMMENTS in declared) comments as T
                else null
            AuthenticatableTracker::class -> if
                (TrackerCapability.AUTH_REQUIRED in declared) auth as T
                else null
            DownloadableTracker::class -> if
                (TrackerCapability.TORRENT_DOWNLOAD in declared) download
                as T else null
            // FavoritesTracker, BrowseableTracker.getForumTree ->
            null (RuTor has no forum, no favorites)
            else -> null
        }
    }

    override fun close() {}
}

// RuTorClientFactory.kt
package lava.tracker.rutor

import javax.inject.Inject
import javax.inject.Provider
import lava.sdk.api.PluginConfig
import lava.tracker.api.TrackerClient
import lava.tracker.api.TrackerDescriptor
import lava.tracker.registry.TrackerClientFactory

class RuTorClientFactory @Inject constructor(private val
    provider: Provider<RuTorClient>) : TrackerClientFactory {
    override val descriptor: TrackerDescriptor = RuTorDescriptor
    override fun create(config: PluginConfig): TrackerClient =
        provider.get()
}

```

Update core/tracker/client/src/main/kotlin/lava/tracker/client/di/TrackerClientModule.kt to register both factories:

```

@Provides @Singleton
fun provideTrackerRegistry(
    rutrackerFactory: RuTrackerClientFactory,
    rutorFactory: RuTorClientFactory,
): TrackerRegistry = DefaultTrackerRegistry().apply {
    register(rutrackerFactory)
}

```

```

        register(rutorFactory)
    }

```

Commit sp3a-3.40: RuTorClient + RuTorClientFactory + Hilt registers RuTor alongside RuTracker.

Section K — Real-tracker integration test

Task 3.41: Real-tracker integration test against rutor.info

Files:

- Create: core/tracker/rutor/src/integrationTest/kotlin/lava/tracker/rutor/RealRuTorIntegrationTest.kt (annotated @RealTracker so default test runs skip it)
- Modify: core/tracker/rutor/build.gradle.kts to define an integrationTest source set

```

@Tag("RealTracker")
class RealRuTorIntegrationTest {
    @Test fun `real search for ubuntu returns >=3 results`() =
        runTest {
            val client = RuTorClient(/* DI-built with real
                                     RuTorHttpClient */)
            val search =
                client.getFeature(SearchableTracker::class) ?:
                error("SEARCH not declared")
            val result = search.search(SearchRequest(query =
                "ubuntu"))
            assertThat(result.items.size).isAtLeast(3)
            for (item in result.items) {
                assertThat(item.infoHash).matches("[A-Fa-f0-9]{40}")
                assertThat(item.sizeBytes).isNotNull()
            }
        }

    @Test fun `real login with test credentials produces userid
        cookie`() = runTest {
        val username = System.getenv("LAVA_TEST_USERNAME") ?:
            "<redacted-per-§6.H>"
        val password = System.getenv("LAVA_TEST_PASSWORD") ?:
            "<redacted-per-§6.H>"
        val auth =
            client.getFeature(AuthenticatableTracker::class) ?:
            error("AUTH not declared")
        val result = auth.login(LoginRequest(username, password))
        assertThat(result.state).isEqualTo(AuthState.Authenticated)
        assertThat(result.sessionToken).isNotNull()
    }
}

```

Run only when invoked explicitly: ./
 gradlew :core:tracker:rutor:integrationTest -PrealTrackers=true.

Record in `.lava-ci-evidence/sp3a-rutor/3.41-real-tracker-smoke.json` after a successful local run.

Commit `sp3a-3.41: real-tracker integration test (login + search)` against `rutor.info`.

Phase 3 acceptance check.

```
./gradlew :core:tracker:rutor:test
ls core/tracker/rutor/src/test/resources/fixtures/rutor/search/ |
    wc -l    # expect 5
ls core/tracker/rutor/src/test/resources/fixtures/rutor/browse/ |
    wc -l    # expect 5
ls core/tracker/rutor/src/test/resources/fixtures/rutor/topic/ |
    wc -l    # expect 5
ls core/tracker/rutor/src/test/resources/fixtures/rutor/login/ |
    wc -l    # expect 5
```

Plus a working real-tracker smoke run recorded in evidence.

Phase 3 done.

Phase 4: Mirror Health + Cross-Tracker Fallback + :feature:tracker_settings UI

Duration: 1 week. **Tasks:** 20. **Goal:** Wire the SDK's `MirrorManager` into the running app (periodic `WorkManager` probes, Room persistence), implement the cross-tracker fallback policy with one-tap modal UX (decision 7a-ii), and ship the `:feature:tracker_settings` Compose UI for tracker selection + custom mirrors.

Acceptance gate: `WorkManager` periodic worker probes both trackers' mirrors every 15 min; mirror health persisted to Room and survives app restart; cross-tracker fallback emits `CrossTrackerFallbackProposed` outcome which the UI renders as a modal; user can add/edit/remove custom mirrors per tracker in settings.

Section A — Persistence schema

Task 4.1: Room schema migration for mirror health and user mirrors

Files:

- Create: `core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/MirrorHealthEntity.kt`
- Create: `core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/MirrorHealthDao.kt`
- Create: `core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/UserMirrorEntity.kt`
- Create: `core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/UserMirrorDao.kt`

- Modify: core/database/src/main/kotlin/.../LavaDatabase.kt to add new entities
- Create: core/database/schemas/lava.database.LavaDatabase/<next>.json (Room schema export)

```
// MirrorHealthEntity.kt
@Entity(
    tableName = "tracker_mirror_health",
    primaryKeys = ["tracker_id", "mirror_url"],
)
data class MirrorHealthEntity(
    @ColumnInfo("tracker_id") val trackerId: String,
    @ColumnInfo("mirror_url") val mirrorUrl: String,
    @ColumnInfo("state") val state: String,           // HEALTHY/
    DEGRADED/UNHEALTHY/UNKNOWN
    @ColumnInfo("last_check_at") val lastCheckAt:
    Long?, // epoch millis
    @ColumnInfo("consecutive_failures") val consecutiveFailures:
    Int,
)

// UserMirrorEntity.kt
@Entity(
    tableName = "tracker_mirror_user",
    primaryKeys = ["tracker_id", "url"],
)
data class UserMirrorEntity(
    @ColumnInfo("tracker_id") val trackerId: String,
    @ColumnInfo("url") val url: String,
    @ColumnInfo("priority") val priority: Int,
    @ColumnInfo("protocol") val protocol: String,    // HTTP/HTTPS/
    HTTP3
    @ColumnInfo("added_at") val addedAt: Long,      // epoch millis
)
```

DAOs with the standard Room CRUD + Flow observers for the active tracker.
Migration in Room: LavaDatabase version bump.

Commit sp3a-4.1: Room schema for tracker_mirror_health +
tracker_mirror_user.

Task 4.2: MirrorHealthRepository + UserMirrorRepository

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/MirrorHealthRepository.kt
- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/UserMirrorRepository.kt

Implementations with Hilt-injected DAOs. Provide methods:
loadAll(trackerId), upsert(state), flowOf(trackerId),
clear(trackerId). Tests using in-memory Room DB.

Commit sp3a-4.2: MirrorHealthRepository + UserMirrorRepository + tests.

Task 4.3: MirrorConfigLoader — bundle + user merge

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/persistence/MirrorConfigLoader.kt
- Create: app/src/main/assets/mirrors.json (bundled config from spec §5.5)

```
class MirrorConfigLoader @Inject constructor(
    @ApplicationContext private val context: Context,
    private val userRepo: UserMirrorRepository,
) {
    suspend fun loadFor(trackerId: String): List<MirrorUrl> {
        val bundled = parseBundledJson()[trackerId]?.mirrors ?:
            emptyList()
        val user = userRepo.loadAll(trackerId).map {
            it.toMirrorUrl() }
        // User entries supersede bundled at same URL
        return (bundled.associateBy { it.url } + user.associateBy
            { it.url }).values
            .sortedBy { it.priority }
    }

    private fun parseBundledJson(): Map<String,
        BundledTrackerConfig> {
        val raw =
            context.assets.open("mirrors.json").bufferedReader().use
            { it.readText() }
        return Json { ignoreUnknownKeys = true }
            .decodeFromString<MirrorsConfig>(raw)
            .trackers
    }
}
```

mirrors.json content matches spec §5.5 verbatim (RuTracker + RuTor entries).

Commit sp3a-4.3: MirrorConfigLoader + bundled mirrors.json.

Section B — WorkManager periodic health worker

Task 4.4: MirrorHealthCheckWorker

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/work/MirrorHealthCheckWorker.kt
- Modify: app's WorkManager initializer to schedule the worker

```

@HiltWorker
class MirrorHealthCheckWorker @AssistedInject constructor(
    @Assisted appContext: Context,
    @Assisted workerParams: WorkerParameters,
    private val sdk: LavaTrackerSdk,
    private val healthRepo: MirrorHealthRepository,
) : CoroutineWorker(appContext, workerParams) {
    override suspend fun doWork(): Result {
        val trackers = sdk.listAvailableTrackers().map {
            it.trackerId
        }
        for (id in trackers) {
            try {
                sdk.probeMirrorsFor(id) // delegates to
                MirrorManager.probeAll
                val states = sdk.observeMirrorHealth(id).first()
                healthRepo.upsertAll(id, states)
            } catch (t: Throwable) {
                // Log but don't fail the whole worker
            }
        }
        return Result.success()
    }
}

```

Periodic schedule: 15 minutes, network-connected constraint, no charging requirement.

Commit sp3a-4.4: MirrorHealthCheckWorker + 15-min periodic schedule.

Task 4.5: Wire LavaTrackerSdk to load + persist mirror health

Add to LavaTrackerSdk:

```

suspend fun probeMirrorsFor(trackerId: String) =
    mirrorManager.probeAll(trackerId)
fun observeMirrorHealth(trackerId: String):
    Flow<List<MirrorState>> =
    mirrorManager.observeHealth(trackerId)

// On init: rehydrate state from healthRepo
suspend fun initialize() {
    for (descriptor in registry.list()) {
        val persisted = healthRepo.loadAll(descriptor.trackerId)
        // Seed mirrorManager states from persisted before the
        // first probe runs
        ...
    }
}

```

Tests: LavaTrackerSdk.initialize() rehydrates state correctly; subsequent observeMirrorHealth reflects the rehydrated values.

Commit sp3a-4.5: LavaTrackerSdk persists mirror health via MirrorHealthRepository.

Section C — Cross-tracker fallback policy

Task 4.6: CrossTrackerFallbackPolicy

Files:

- Create: core/tracker/client/src/main/kotlin/lava/tracker/client/CrossTrackerFallbackPolicy.kt

```
class CrossTrackerFallbackPolicy @Inject constructor(  
    private val registry: TrackerRegistry,  
) {  
    /**  
     * When the active tracker has exhausted all mirrors and a  
     * configured  
     * alternative tracker supports the same capability, returns  
     * the alternative  
     * descriptor. Returns null when no fallback should be  
     * proposed.  
     */  
    fun proposeFallback(  
        failedTrackerId: String,  
        capability: TrackerCapability,  
        userOptedIn: Boolean = true,  
    ): TrackerDescriptor? {  
        if (!userOptedIn) return null  
        return registry.trackersWithCapability(capability)  
            .firstOrNull { it.trackerId != failedTrackerId }  
    }  
}
```

Tests using FakeTrackerClient. Commit sp3a-4.6:
CrossTrackerFallbackPolicy.

Task 4.7: Wire fallback into LavaTrackerSdk.search + sibling methods

Modify LavaTrackerSdk.search (Phase 2 Task 2.30 stubbed this as Failure propagation):

```
suspend fun search(request: SearchRequest, page: Int = 0):  
    SearchOutcome {  
    val active = getActiveClient()  
    val feature = active.getFeature(SearchableTracker::class)  
    ?: return SearchOutcome.Failure(...)  
    return try {  
        SearchOutcome.Success(  

```

```

        mirrorManager.executeWithFallback(active.descriptor.trackerId)
        { _ ->
            feature.search(request, page)
        },
        viaTracker = active.descriptor.trackerId
    )
} catch (e: MirrorUnavailableException) {
    val proposal =
        fallbackPolicy.proposeFallback(active.descriptor.trackerId,
            TrackerCapability.SEARCH)
    if (proposal != null) {
        SearchOutcome.CrossTrackerFallbackProposed(
            failedTrackerId = active.descriptor.trackerId,
            proposedTrackerId = proposal.trackerId,
            capability = TrackerCapability.SEARCH,
            resumeWith = {
                val alt = registry.get(proposal.trackerId,
                    MapPluginConfig())
                val altFeature =
                    alt.getFeature(SearchableTracker::class)
                ?: return@CrossTrackerFallbackProposed
                SearchOutcome.Failure(...)

                SearchOutcome.Success(altFeature.search(request, page),
                    viaTracker = proposal.trackerId)
            },
        )
    } else {
        SearchOutcome.Failure(e, triedTrackers =
            listOf(active.descriptor.trackerId))
    }
}
}
}

```

Apply the same pattern to `browse`, `getTopic`, etc.

Tests with `FakeTrackerClients`: simulate all `RuTracker` mirrors `UNHEALTHY`, assert outcome is `CrossTrackerFallbackProposed`; simulate user accepting → success via `RuTor`; simulate user dismissing → `Failure`.

Commit

sp3a-4.7: `LavaTrackerSdk` emits `CrossTrackerFallbackProposed` when all mirrors fail.

Task 4.8: Falsifiability rehearsal — cross-tracker fallback contract

Mutate `proposeFallback` to always return `null`; assert the test that expects a fallback proposal fails. Revert; passes. Evidence at `.lava-ci-evidence/sp3a-phase4/4.8-fallback.json`. Commit.

Section D — :feature:tracker_settings Compose UI

Task 4.9: Module skeleton

Files:

- Create: feature/tracker_settings/build.gradle.kts
- Create: feature/tracker_settings/src/main/AndroidManifest.xml

```
// build.gradle.kts
plugins { id("lava.android.feature") }
android.namespace = "lava.feature.tracker_settings"
dependencies {
    implementation(project(":core:tracker:api"))
    implementation(project(":core:tracker:client"))
    implementation(project(":core:designsystem"))
    implementation(project(":core:navigation"))
}
```

Add to settings.gradle.kts: include(":feature:tracker_settings").

Commit sp3a-4.9: scaffold :feature:tracker_settings module.

Task 4.10: TrackerSettingsState, TrackerSettingsAction, TrackerSettingsSideEffect

Files:

- Create: feature/tracker_settings/src/main/kotlin/lava/feature/tracker_settings/TrackerSettingsState.kt
- Create: .../TrackerSettingsAction.kt
- Create: .../TrackerSettingsSideEffect.kt

```
data class TrackerSettingsState(
    val loading: Boolean = false,
    val activeTrackerId: String = "rutracker",
    val availableTrackers: List<TrackerDescriptor> = emptyList(),
    val mirrorHealthByTracker: Map<String, List<MirrorState>> = emptyMap(),
    val customMirrors: Map<String, List<MirrorUrl>> = emptyMap(),
    val showAddMirrorDialog: Boolean = false,
    val addMirrorTargetTracker: String? = null,
    val error: String? = null,
)

sealed class TrackerSettingsAction {
    object Load : TrackerSettingsAction()
    data class SwitchActive(val trackerId: String) : TrackerSettingsAction()
    data class OpenAddMirrorDialog(val trackerId: String) : TrackerSettingsAction()
    object DismissAddMirrorDialog : TrackerSettingsAction()
```

```

    data class AddCustomMirror(val trackerId: String, val url:
        String, val priority: Int, val protocol: Protocol) :
        TrackerSettingsAction()
    data class RemoveCustomMirror(val trackerId: String, val url:
        String) : TrackerSettingsAction()
    data class ProbeNow(val trackerId: String) :
        TrackerSettingsAction()
}
sealed class TrackerSettingsSideEffect {
    data class ShowToast(val message: String) :
        TrackerSettingsSideEffect()
}

```

Commit sp3a-4.10: TrackerSettings MVI state/action/side-effect.

Task 4.11: TrackerSettingsViewModel

```

@HiltViewModel
class TrackerSettingsViewModel @Inject constructor(
    private val sdk: LavaTrackerSdk,
    private val userMirrorRepo: UserMirrorRepository,
) : ContainerHost<TrackerSettingsState,
    TrackerSettingsSideEffect>, ViewModel() {

    override val container = container<TrackerSettingsState,
        TrackerSettingsSideEffect>(TrackerSettingsState())

    init { intent { reduce { state.copy(loading = true) };
        load() } }

    private fun load() = intent {
        val trackers = sdk.listAvailableTrackers()
        val active = sdk.activeTrackerId.value
        val health = trackers.associate { it.trackerId to
            sdk.observeMirrorHealth(it.trackerId).first() }
        val custom = trackers.associate { it.trackerId to
            userMirrorRepo.loadAll(it.trackerId).map { e ->
                e.toMirrorUrl() } }
        reduce { state.copy(loading = false, availableTrackers =
            trackers, activeTrackerId = active, mirrorHealthByTracker
            = health, customMirrors = custom) }
    }

    fun onAction(action: TrackerSettingsAction) = intent {
        when (action) {
            TrackerSettingsAction.Load -> load()
            is TrackerSettingsAction.SwitchActive -> {
                sdk.switchTracker(action.trackerId)
                reduce { state.copy(activeTrackerId =
                    action.trackerId) }
            }
        }

        postSideEffect(TrackerSettingsSideEffect.ShowToast("Switched
            to ${action.trackerId}"))
    }
}

```



```

        )
        HorizontalDivider()
    }
}
}

```

Commit sp3a-4.12: TrackerSelectorList Compose component.

Task 4.13: MirrorListSection Compose component

Renders mirror health for a tracker: each `MirrorUrl` with a colored health indicator (green=HEALTHY, yellow=DEGRADED, red=UNHEALTHY, gray=UNKNOWN), priority, protocol badge, optional "remove" button for user-supplied mirrors.

Commit sp3a-4.13: MirrorListSection.

Task 4.14: HealthIndicator Compose component

A small colored dot + label. Pure stateless renderer.

Commit sp3a-4.14: HealthIndicator.

Task 4.15: AddCustomMirrorDialog Compose component

Dialog with: URL field (validated as `https?://...`), priority slider, protocol radio (HTTP/HTTPS/HTTP3), Add/Cancel buttons.

Commit sp3a-4.15: AddCustomMirrorDialog.

Task 4.16: TrackerSettingsScreen — top-level composable

Composes the four components above + scaffolding (TopAppBar, Snackbar host for side effects).

Commit sp3a-4.16: TrackerSettingsScreen top-level composable.

Task 4.17: Cross-tracker fallback modal Compose component

Files:

- Create: `feature/search_result/src/main/kotlin/lava/feature/search_result/components/CrossTrackerFallbackModal.kt`

(Lives in `:feature:search_result` because that's where the modal is consumed; `:feature:tracker_settings` only has the settings UI.)

```

@Composable
fun CrossTrackerFallbackModal(
    failedTracker: String,
    proposedTracker: String,
    onAccept: () -> Unit,
    onDismiss: () -> Unit,
) {
    AlertDialog(
        onDismissRequest = onDismiss,
        title = { Text("$failedTracker is unavailable") },
        text = {
            Text("All known mirrors of $failedTracker are
                unreachable. " +
                "Try the same search on $proposedTracker?")
        },
        confirmButton = {
            TextButton(onClick = onAccept) { Text("Try
                $proposedTracker") }
        },
        dismissButton = {
            TextButton(onClick = onDismiss) { Text("Cancel") }
        },
    )
}

```

Commit sp3a-4.17: CrossTrackerFallbackModal
in :feature:search_result.

Task 4.18: Wire modal into :feature:search_result ViewModel

SearchResultViewModel consumes LavaTrackerSdk.search(). When the outcome is CrossTrackerFallbackProposed, emit a side effect that the screen renders as the modal. On user accept → call outcome.resumeWith(). On user dismiss → propagate the original failure as a Snackbar.

Commit sp3a-4.18: SearchResultViewModel renders
CrossTrackerFallbackModal.

Task 4.19: Navigation entry to :feature:tracker_settings

Add addTrackerSettings() and openTrackerSettings() to :core:navigation. Wire from :feature:menu (the existing menu/account screen) as a new menu item.

Commit sp3a-4.19: navigation entry – Settings → Trackers.

Task 4.20: Phase 4 integration smoke test

Run on a real Android emulator (or device): launch app → menu → Trackers → see RuTracker + RuTor listed → tap RuTor → home re-renders for RuTor → search for "ubuntu" → results from RuTor render. Capture as a video or screenshot sequence.

Record in `.lava-ci-evidence/sp3a-phase4/4.20-integration-smoke.json` with attached screenshots paths.

Commit `sp3a-4.20`: Phase 4 integration smoke verified on real device.

Phase 4 done. Phase 5 unblocked (or runs concurrent).

Phase 5: Constitutional Updates + 8 Challenge Tests + Tag Gate

Duration: 0.5 weeks. **Tasks:** 25. **Goal:** Land all eight cascaded constitutional doc updates, eight Challenge Tests with falsifiability rehearsals, `scripts/ci.sh` + `scripts/tag.sh` updates with new gates, pre-push hook, real-device verification, tag `Lava-Android-1.2.0-1020`.

Acceptance gate: `scripts/tag.sh` `Lava-Android-1.2.0-1020` succeeds; `.lava-ci-evidence/Lava-Android-1.2.0-1020/` contains complete evidence pack; tag visible on all four upstreams with verified per-mirror SHA convergence.

Section A — Cascaded constitutional doc updates

Task 5.1: Add clauses 6.D, 6.E, 6.F to root `CLAUDE.md`

Files:

- Modify: `CLAUDE.md`

Insert after the existing Sixth Law section the three new clauses with the exact body wording from spec §9.1. Commit `sp3a-5.1`: constitutional clauses 6.D + 6.E + 6.F added to root `CLAUDE.md`.

Task 5.2: Update `core/CLAUDE.md` with tracker capability honesty rule

Add a paragraph referencing 6.E and listing the binding requirement: every new feature interface in `core/tracker/api/` must have an enum entry in `TrackerCapability`, a behavioral test, and a doc mention.

Commit `sp3a-5.2`: `core/CLAUDE.md` scoped clause for new feature interfaces.

Task 5.3: Update feature/CLAUDE.md with Challenge Test requirement

Add: every feature ViewModel that consumes LavaTrackerSdk must have a Challenge Test covering the same UI path; the Challenge Test's falsifiability rehearsal must be recorded in the same PR.

Commit

sp3a-5.3: feature/CLAUDE.md scoped clause for SDK-consuming ViewModels.

Task 5.4: Update lava-api-go/CLAUDE.md and AGENTS.md with bridge expectations

Add a section: "When SP-2 ships, the Go-side rutracker refactor must satisfy 6.D and 6.E. The bridge plan will be written then; this clause makes the requirement explicit now so the Go work doesn't drift."

Commit sp3a-5.4: lava-api-go/{CLAUDE,AGENTS}.md SP-3a-bridge expectations.

Task 5.5: Create submodules/tracker_sdk/CLAUDE.md

Already created in Phase 1 Task 1.7 — verify present and matches the body in spec §9.2. If missing, create from the Phase 1 spec content.

Commit (no-op if already done) or sp3a-5.5: verify Tracker-SDK constitution present.

Task 5.6: Create submodules/tracker_sdk/CONSTITUTION.md and AGENTS.md

Already created in Phase 1 Task 1.7 — verify present.

Commit sp3a-5.6: verify Tracker-SDK CONSTITUTION + AGENTS present.

Task 5.7: Update root AGENTS.md with SDK module map

Extend the existing module-map section with the new core/tracker/* modules and the submodules/tracker_sdk/ pin policy. Add a pointer to docs/sdk-developer-guide.md (created in Task 5.8).

Commit sp3a-5.7: root AGENTS.md updated with SDK module map.

Task 5.8: Create docs/sdk-developer-guide.md (partial draft)

Per spec §14 success criterion 10: a partial draft outlining the steps for adding a third tracker, validated by paper-tracing through the existing RuTor module.

Sections:

1. SDK Architecture overview
2. Adding a new tracker (7-step recipe — Section 6.1 of the source PDF)
3. Mirror configuration
4. Testing requirements

Commit sp3a-5.8: docs/sdk-developer-guide.md partial draft.

Section B — 8 Challenge Tests

Task 5.9: Challenge Test C1 — App launch and tracker selection

Files:

- Create: app/src/androidTest/kotlin/lava/app/challenges/C1_AppLaunchAndTrackerSelectionTest.kt

Compose UI test: launch MainActivity, navigate to Settings → Trackers, assert RuTracker and RuTor are listed, tap RuTracker, assert active state changes to RuTracker.

Falsifiability rehearsal: comment out the registry registration for RuTor in the Hilt module → test asserts only one tracker present → fails. Revert.

Evidence: .lava-ci-evidence/sp3a-challenges/C1-<sha>.json with attached screenshot.

Commit sp3a-5.9: Challenge Test C1 (app launch + tracker selection) + falsifiability rehearsal.

Task 5.10: Challenge Test C2 — Authenticated search on RuTracker

Compose UI test: with credentials (<redacted-per-§6.H>/<redacted-per-§6.H>), perform a search for "ubuntu" on RuTracker, assert ≥1 result row renders with parseable size and seeders text.

Falsifiability rehearsal: throw inside RuTrackerSearch.search → test asserts the error UI is shown. Revert.

Commit sp3a-5.10: Challenge Test C2 + falsifiability rehearsal.

Task 5.11: Challenge Test C3 — Anonymous search on RuTor

Compose UI test: switch active tracker to RuTor, search for "ubuntu", assert ≥ 1 result row with parseable size and seeders. (No login required per decision 7b-ii.)

Falsifiability rehearsal: mutate the RuTor descriptor capabilities to drop SEARCH → test asserts the search UI shows "not supported" → fails when capability check is honored. Revert.

Commit sp3a-5.11: Challenge Test C3 + rehearsal.

Task 5.12: Challenge Test C4 — Switch tracker and re-search

After C2 search, switch to RuTor in settings, return to search, assert results re-render from RuTor (different items than RuTracker).

Falsifiability rehearsal: break `LavaTrackerSdk.switchTracker` (don't update `_activeTrackerId.value`) → test asserts results stay RuTracker → fails. Revert.

Commit sp3a-5.12: Challenge Test C4 + rehearsal.

Task 5.13: Challenge Test C5 — View topic detail

Tap a search result, assert `TopicDetail` screen renders with title, description, file list, magnet URI button.

Falsifiability rehearsal: mutate `TopicMapper.toTopicDetail` to drop the description field → test asserts description is shown → fails. Revert.

Commit sp3a-5.13: Challenge Test C5 + rehearsal.

Task 5.14: Challenge Test C6 — Download .torrent file

From topic detail, tap Download → assert a file is written to the app's downloads directory and the bytes parse as a valid bencoded torrent (`info.pieces` field present).

Falsifiability rehearsal: short-circuit `RuTorDownload.downloadTorrentFile` to return empty → test asserts file is non-empty → fails. Revert.

Commit sp3a-5.14: Challenge Test C6 + rehearsal.

Task 5.15: Challenge Test C7 — Cross-tracker fallback modal accept

Pre-condition: simulate all RuTracker mirrors UNHEALTHY (use a test-only injection that pre-marks the mirrors as UNHEALTHY in the in-memory state).

Test: perform a search; assert `CrossTrackerFallbackModal` is rendered; tap "Try RuTor"; assert results from RuTor render with the "results from RuTor" banner.

Falsifiability rehearsal: mutate `LavaTrackerSdk.search` to never emit `CrossTrackerFallbackProposed` (always return `Failure`) → test asserts modal is shown → fails. Revert.

Commit

sp3a-5.15: Challenge Test C7 (cross-tracker fallback accept path) + rehearsal.

Task 5.16: Challenge Test C8 — Cross-tracker fallback modal dismiss

Same setup as C7, tap "Cancel" → assert explicit failure UI renders (Snackbar with "Search failed" or equivalent), no silent fallback.

Falsifiability rehearsal: mutate the dismiss handler to silently retry on RuTor anyway → test asserts no RuTor results render → fails. Revert.

Commit sp3a-5.16: Challenge Test C8 (cross-tracker fallback dismiss) + rehearsal.

Section C — `scripts/ci.sh` + `scripts/tag.sh` + pre-push hook

Task 5.17: Create `scripts/ci.sh`

Files:

- Create: `scripts/ci.sh`

Single entry point implementing the gate described in spec §8.6. Driven by `--changed-only` flag for the pre-push hook subset, `--full` for tag-time runs.

```
#!/usr/bin/env bash
# scripts/ci.sh – local-only CI gate for Lava
set -euo pipefail
cd "$(dirname "$0")/.."

MODE="${1:---full}"
EVIDENCE_DIR=".lava-ci-evidence/${date +%Y-%m-%dT%H-%M-%S}"
mkdir -p "$EVIDENCE_DIR"

echo "==> Hosted-CI forbidden-files check"
forbidden=$(find . \
  \( -path './.git' -o -path './build' -o -path './node_modules' \) -prune -o \
  \( -path '*.github/workflows/*' -o -name '.gitlab-ci.yml' -o \
    -name '.circleci' -o \
    -name 'azure-pipelines.yml' -o -name 'bitbucket-pipelines.yml' -o -name 'Jenkinsfile' \) \
  -print 2>/dev/null || true)
```

```

if [[ -n "$forbidden" ]]; then echo "FORBIDDEN HOSTED-CI FILES:
    $forbidden" >&2; exit 1; fi

echo "==> Host-power forbidden-command regex check"
viol=$(grep -rE --exclude-dir=.git --exclude-dir=build \
    '(systemctl\s+(suspend|hibernate|poweroff|halt|reboot|kill-user|
        kill-session)|loginit\s+(suspend|hibernate|poweroff|
        reboot|kill-user|kill-session|terminate-user|terminate-
        session)|pm-suspend|pm-hibernate|shutdown\s+(-h|-r|-P|-H|
        now|--halt|--poweroff|--reboot)|
        org\.freedesktop\.login1\.Manager\.(Suspend|Hibernate|
        HybridSleep|PowerOff|Reboot))' \
    scripts/ docs/ buildSrc/ 2>/dev/null || true)
if [[ -n "$viol" ]]; then echo "FORBIDDEN HOST-POWER COMMAND:
    $viol" >&2; exit 1; fi

echo "==> Spotless"
./gradlew spotlessCheck

echo "==> Unit tests"
./gradlew :core:tracker:api:test :core:tracker:client:test :core:tracker:registry:
:core:tracker:mirror:test :core:tracker:rutracker:test :core:tracker:rutracker:
\
:core:tracker:testing:test :core:network:impl:test :core:preferences:tes

if [[ "$MODE" == "--full" ]]; then
    echo "==> SwitchingNetworkApi parity gate"
    ./gradlew :core:network:impl:test --tests "*ParityTest*"

    echo "==> Mutation tests (PITest)"
    ./gradlew :core:tracker:rutor:pitest :core:tracker:rutracker:pitest

    echo "==> Fixture freshness check"
    ./scripts/check-fixture-freshness.sh

    echo "==> Compose UI Challenge Tests"
    ./gradlew :app:connectedDebugAndroidTest --tests
        "lava.app.challenges.*"
fi

echo "==> Constitutional doc parser"
./scripts/check-constitution.sh

echo "==> All gates passed"
echo "$MODE" > "$EVIDENCE_DIR/mode"
git rev-parse HEAD > "$EVIDENCE_DIR/sha"

```

Commit sp3a-5.17: scripts/ci.sh local-only CI gate.

Task 5.18: Create scripts/check-fixture-freshness.sh

```
#!/usr/bin/env bash
set -euo pipefail
THIRTY_DAYS_AGO=$(date -d '30 days ago' +%s)
SIXTY_DAYS_AGO=$(date -d '60 days ago' +%s)
warn=0
for f in $(find core/tracker/*/src/test/resources/fixtures -name
    '*.html'); do
    fname=$(basename "$f")
    date_in_name=$(echo "$fname" | grep -oE '[0-9]{4}-[0-9]{2}-[0-9]
        {2}' || echo "")
    if [[ -z "$date_in_name" ]]; then continue; fi
    ts=$(date -d "$date_in_name" +%s 2>/dev/null || echo 0)
    if [[ "$ts" -lt "$SIXTY_DAYS_AGO" ]]; then echo "BLOCK: $f >60
        days old" >&2; exit 1; fi
    if [[ "$ts" -lt "$THIRTY_DAYS_AGO" ]]; then echo "WARN: $f >30
        days old"; warn=1; fi
done
exit 0
```

Commit sp3a-5.18: scripts/check-fixture-freshness.sh.

Task 5.19: Create scripts/check-constitution.sh

```
#!/usr/bin/env bash
set -euo pipefail
# Verify that root CLAUDE.md contains clauses 6.D, 6.E, 6.F
required=(
    "6.D - Behavioral Coverage Contract"
    "6.E - Capability Honesty"
    "6.F - Anti-Bluff Submodule Inheritance"
)
for clause in "${required[@]"; do
    if ! grep -qF "$clause" CLAUDE.md; then
        echo "MISSING constitutional clause: $clause" >&2
        exit 1
    fi
done
# Verify Tracker-SDK CLAUDE.md exists
test -f submodules/tracker_sdk/CLAUDE.md || { echo "MISSING
    submodules/tracker_sdk/CLAUDE.md" >&2; exit 1; }
echo "Constitution check passed"
```

Commit sp3a-5.19: scripts/check-constitution.sh.

Task 5.20: Create .github/hooks/pre-push and enable

```
mkdir -p .github/hooks
cat > .github/hooks/pre-push <<'EOF'
```

```
#!/usr/bin/env bash
set -euo pipefail
exec ./scripts/ci.sh --changed-only
EOF
chmod +x .github/hooks/pre-push
git config core.hooksPath .github/hooks
```

Commit sp3a-5.20: pre-push hook (non-bypassable local CI gate).

Task 5.21: Update `scripts/tag.sh` with new evidence-pack gate

Files:

- Modify: `scripts/tag.sh`

Add precondition: refuse to tag unless `.lava-ci-evidence/<TAG>/` exists with required subfiles (`ci.sh.json`, `challenges/`, `bluff-audit/`, `mirror-smoke/`, `per-mirror SHA`). Reuse existing four-upstream push + per-mirror SHA convergence (clause 6.C); add a check that all 8 Challenge Tests in `.lava-ci-evidence/sp3a-challenges/` ran since the commit being tagged.

Commit sp3a-5.21: `scripts/tag.sh` enforces SP-3a evidence-pack gate.

Section D — Real-device verification + tag

Task 5.22: Real-device verification on a physical Android phone

Per Sixth Law clause 5: human (or scripted runner) uses each Challenge-Tested feature on a real device against real RuTracker and real RuTor. Outcome recorded in `.lava-ci-evidence/Lava-Android-1.2.0-1020/real-device-verification.md` with timestamps + screenshots.

Commit sp3a-5.22: real-device verification artifact.

Task 5.23: Update `CHANGELOG.md` with 1.2.0 entry

Lava-Android-1.2.0-1020 (2026-MM-DD)

Added

- RuTor (`rutor.info` / `rutor.is`) tracker support.
- New tracker selection UI in Settings → Trackers.
- Custom mirror entry per tracker.
- Cross-tracker fallback modal: when all mirrors of the active tracker fail, the app proposes the alternative tracker for the same operation.

Changed

- Internal: RuTracker implementation now fully decoupled behind the multi-tracker SDK.

- New ``vasic-digital/Tracker-SDK`` submodule mounted at ``submodules/tracker_sdk/``.

Constitutional

- Added clauses 6.D (Behavioral Coverage Contract), 6.E (Capability Honesty), 6.F (Anti-Bluff Submodule Inheritance) to root CLAUDE.md and cascaded.

Fixed

- (none – this release is feature-additive)

Commit sp3a-5.23: CHANGELOG entry for Lava-Android-1.2.0-1020.

Task 5.24: Bump version codes in app/build.gradle.kts

```
versionCode = 1020
versionName = "1.2.0"
```

(Verify file path: usually a release.properties or app/build.gradle.kts block.)

Commit sp3a-5.24: bump app version to 1.2.0 / 1020.

Task 5.25: Tag Lava-Android-1.2.0-1020 and mirror

Run: `./scripts/tag.sh Lava-Android-1.2.0-1020`

The script enforces all the Phase 5 gates, pushes to all four upstreams, verifies per-mirror SHA convergence. Creates the canonical evidence pack at `.lava-ci-evidence/Lava-Android-1.2.0-1020/`.

If any gate fails, fix and re-run. Do NOT bypass with `--no-verify` or environment overrides — that would violate the Local-Only CI/CD constitutional rule.

Final commit sp3a-5.25: SP-3a complete – tagged Lava-Android-1.2.0-1020.

Phase 5 done. SP-3a complete.

Final acceptance check

```
git tag --list | grep Lava-Android-1.2.0-1020
ls -la .lava-ci-evidence/Lava-Android-1.2.0-1020/
git ls-remote --tags github | grep Lava-Android-1.2.0-1020
git ls-remote --tags gitflic | grep Lava-Android-1.2.0-1020
git ls-remote --tags gitlab | grep Lava-Android-1.2.0-1020
git ls-remote --tags gitverse | grep Lava-Android-1.2.0-1020
```

All checks pass: tag visible locally, evidence pack present, four upstreams converged.

The SP-3a-bridge follow-up (Go-side rutracker refactor + parity tests) runs after SP-2 ships its last release tag — that's a separate plan, not part of this one.

Plan Self-Review

Spec coverage:

- §0 Decision log → all 7 decisions traced to specific tasks: scope decomposition is the very shape of this spec; hybrid extraction → Tasks 1.1–1.27; Kotlin-first → Phase 2 explicitly excludes Go; behavioral coverage → Task 0.7 + 5.17; in-flight bluff audit → Phase 0; bundled+user mirrors → Tasks 4.1–4.3; one-tap fallback → Tasks 4.6–4.8 + 4.17; anonymous-by-default → Task 3.38; four-upstream mirror → Tasks 1.1–1.5 + 1.25–1.27.
- §1 Goals G1–G8 → all addressed; G1 by Phase 1, G2 by Phase 2, G3 by Phase 3, G4 by Phase 4, G5 by Phase 1 SDK extraction, G6 by Phase 5 §A, G7 by Phase 0, G8 by Phase 5 §D.
- §2 Architecture → file structure section + Phase 1 module creation tasks.
- §3 Tracker-SDK submodule → Phase 1 §B–§G.
- §4 Core interfaces & data model → Tasks 1.32–1.35.
- §5 Mirror & cross-tracker fallback → Phase 4.
- §6 RuTor implementation → Phase 3.
- §7 Mappers + adapter + backward compat → Phase 2 §D, §E, §G.
- §8 Testing strategy → Phase 0, Phase 5 §B; coverage exemption ledger → Task 0.7.
- §9 Constitutional updates → Phase 5 §A.
- §10 Phase shape → matches.
- §11 Versioning + rollout → Tasks 5.23–5.25.

Placeholder scan: no "TBD", "TODO" outside of intentionally-stubbed code blocks (mappers stubbed in Tasks 2.8–2.12 with explicit "populated in Task 2.X" pointers — that's a forward reference, not a planning gap).

Type consistency: `LavaTrackerSdk.search()` returns `SearchOutcome` consistently across Tasks 2.30, 4.7, 5.10–5.16.

`TrackerCapability.AUTH_REQUIRED` (not `AUTH`) used consistently. `MirrorUrl` from `lava.sdk.api`, not `lava.tracker.api` — consistent.

No spec section unaddressed.

End of plan.